

Chapter 36. Zvwm Family of Integrated Matrix Extensions

36.1. Introduction

High-performance computing and machine learning workloads depend critically on matrix multiply-accumulate over a wide range of data types and precisions. Dedicated matrix-multiply accelerators often require new register state—separate matrix register files—to achieve competitive throughput, introducing substantial architectural complexity and binary interface disruption.

The Zvwm family of Integrated Matrix extensions (Zvmmm, Zvvfmm, Zvmtls, Zvmttls) takes a different approach: it accelerates matrix multiplication using *only* the $32 \times \text{VLEN}$ architected vector registers already defined by the RISC-V "V" vector extension. By interpreting vector register groups as two-dimensional matrix tiles, the Zvwm family of Integrated Matrix extensions delivers high arithmetic density without introducing any new architected state.

For orientation, **VLEN** is the vector-register width in bits, **SEW** is the accumulator element width, **LMUL** is the vector register-group multiplier for the A and B input operands, **W** is the per-instruction widening factor, and λ is the IME tile-layout parameter introduced below.

We focus, in particular, on matrix multiply-accumulate operations of the form $C \leftarrow A \times B^T + C$, using matrix tiles held in vector register groups. A ($\sigma \times K_{\text{eff}}$) and B ($\sigma \times K_{\text{eff}}$) are the input panels and C ($\sigma \times \sigma$) is the accumulator panel, with $\sigma = \text{VLEN} \div (\text{SEW} \times \lambda)$ and $K_{\text{eff}} = \lambda \times W \times \text{LMUL}$. A, B, and C are row-major matrix panels. The detailed tile geometry, including the roles of **VLEN**, **SEW**, λ , **W**, **LMUL**, **K_{eff}**, and **EMUL_C**, is defined in [Section 36.2.1](#).

The extensions are designed to support implementations spanning a wide range of microarchitectures and performance points: from small, embedded in-order cores targeting low-power and area-constrained applications, to large, high-performance out-of-order implementations targeting HPC and AI workloads. A key architectural goal is that the same executable can run correctly across this entire range without recompilation. Achieving near-peak arithmetic throughput may require the executable to contain multiple code paths specialized for different supported tile geometries or **EMUL_C** values, with the appropriate path selected at run time.

36.2. Overview

The Zvwm family of Integrated Matrix extensions consists of the following extensions:

- Zvmmm - [Matrix-multiplication instructions \(integer\)](#): multiply-accumulate instructions for integer matrix tiles
- Zvvfmm - [Matrix-multiplication instructions \(floating-point\)](#): multiply-accumulate instructions for floating-point matrix tiles
- Zvmtls - [Load/store instructions for matrix tiles](#): two-dimensional load/store instructions for moving data between memory and vector register groups interpreted as matrix tiles
- Zvmttls - [Transposing load/store instructions for matrix tiles](#): two-dimensional transposing load/store instructions for moving data between memory and vector register groups interpreted as matrix tiles



*Reader orientation: this chapter first defines the common tile geometry and new **vtype** fields, then the integer and floating-point multiply-accumulate semantics, then microscaling, tile load/store instructions, C intrinsics, instruction details, and finally the encoding maps. The encoding maps are normative for instruction legality; instruction descriptions that refer to an*

encoding-map row rely on the tables in [Section 36.13](#).



Unless explicitly stated otherwise, instruction descriptions, encoding tables, exception lists, **vtype** field definitions, operation pseudocode, and required disclosure rules are normative.

Notes, examples, rationale paragraphs, and software-usage discussions are informative. They explain intent, expected usage, implementation strategies, or performance considerations, but do not add architectural requirements beyond the normative text they reference.

36.2.1. Matrix tile multiplication geometry

The geometry of the multiplier and the tiles is defined by the new **lambda** (λ) field, which is encoded in 3 bits in the **vtype** CSR, in combination with the per-instruction widening factor **W** and the standard vector-configuration values **LMUL**, **SEW**, and **VLEN**.

The Zvwm family uses the widening factor **W** to describe both the arithmetic widening ratio and the corresponding packing of input elements within each **SEW**-wide element group. The elements of accumulator **C** always have width **SEW** and corresponding element group size (EGS) of 1.

When **W** = 1, the instruction is *non-widening*: each input element has width **SEW**, no packing occurs, and the element group sizes of inputs **A** and **B** are both 1.

When **W** > 1, the instruction is *widening*: each input element has width **SEW**/**W**, and **W** such narrow input elements are packed into an **SEW**-wide vector element group. Correspondingly, element group sizes of both **A** and **B** are **W**.

Thus, "packing" refers to the storage layout of the inputs in the vector register groups, whereas "widening" refers to the arithmetic relationship between the input element width and the accumulator element width. In this specification, these two views are coupled by the same parameter **W**.

The packed layout of **W** narrow input elements within one **SEW**-wide storage position is an *element group* in the sense of the base RISC-V V vector specification, with an element-group size (**EGS**) of **W**.

Matrix tiles are represented using the existing RISC-V V register file and its configuration state. The three matrices in the multiply-accumulate operation $C \leftarrow A \times B^T + C$ are stored as follows:

- The *accumulator* **C** is stored in a vector register group with element width **SEW**. The register-group multiplier of the **C** register group is **EMUL_C**, computed from the tile geometry: $\text{EMUL_C} = (\text{VLEN} / \text{SEW}) / \lambda^2$, where λ is the tile-layout parameter decoded from the **lambda**[2:0] field in **vtype**. λ^2 must divide $(\text{VLEN} / \text{SEW})$, giving $\text{EMUL_C} \in \{1, 2, 4, 8, 16\}$. The **C** base register index must be a multiple of **EMUL_C**.



EMUL_C plays the same role for the **C** operand that the per-operand **EMUL** plays for ordinary **V** instructions: it is the operand's effective register-group multiplier and it dictates the operand's alignment. However, **EMUL_C** is derived from the tile geometry (**VLEN**, **SEW**, λ) rather than from an **EEW** / **SEW** ratio, and it is independent of **vtype.LMUL**. In particular, **EMUL_C** = 16 lies outside the value set permitted for the per-operand **EMUL** of base **V** instructions; the **C** operand is therefore aligned and indexed analogously to a **V** **EMUL**-grouped operand but with an extended value range.



For the C-language intrinsic interface, **EMUL_C** = 16 is represented by IME-only **m16** accumulator types. These types do not imply that **LMUL=16** is a legal **vtype** setting, and they are not ordinary RVV vector intrinsic types. See [Section 36.10.1.1](#).

- The input matrices **A** and **B** are stored in vector register groups with element width determined by the

instruction: equal to SEW for non-widening variants ($W=1$), and equal to SEW/2, SEW/4, or SEW/8 for widening variants ($W=2$, $W=4$, or $W=8$), with the narrower input elements packed within each SEW-wide storage position. The effective K dimension of the multiply (K_{eff}) equals $\lambda \times W \times \text{LMUL}$, where W is 1 for non-widening instructions, 2 for $2\times$ widening instructions, 4 for $4\times$ widening instructions, and 8 for $8\times$ widening instructions. LMUL scales the A and B register groups along the K dimension only and does not affect C. Only integer values of LMUL are supported by the Zvwm family of Integrated Matrix extensions: $\text{LMUL} \in \{1, 2, 4, 8\}$. Fractional LMUL settings ($\text{LMUL} < 1$) are reserved. Executing any Zvwm instruction when `vtype.vlmul` does not encode an integer LMUL value in $\{1, 2, 4, 8\}$ raises an illegal-instruction exception.

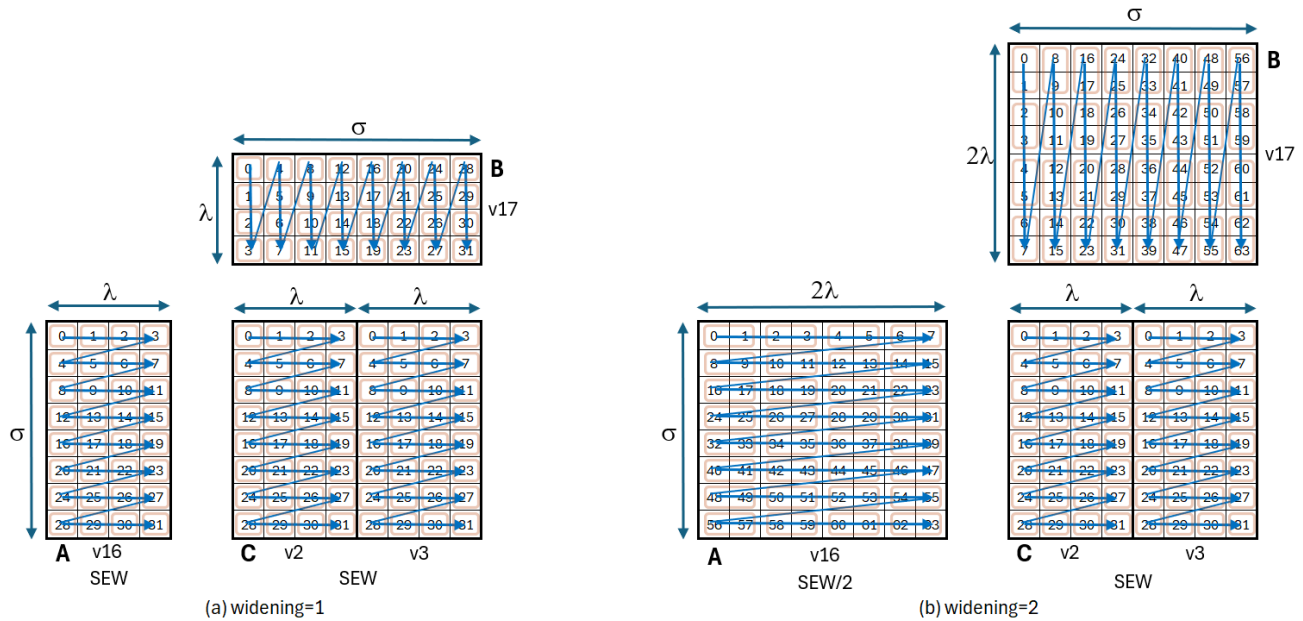


Figure 8. Geometry of matrix tiles and element ordering for VLEN/SEW (vector length in C element width) = 32 and $\lambda=4$. Vector register groups are interpreted as 2D tiles. Vector element indices show the tile element order. (a) Non-widening case ($W=1$) with A, B, and C having the same SEW. (b) Double-widening case ($W=2$) with A and B at half the SEW of C, with two narrow input elements packed into each SEW-wide storage position.

The K-dimension of the multiplication (shared inner dimension of A and B^T) is determined by λ from `vtype`, scaled by a per-instruction widening factor W and further multiplied by LMUL:

$$K_{\text{eff}} = \lambda \times W \times \text{LMUL}$$

A and B tiles have elements of width SEW/W ; the accumulator C has elements of width SEW (see table below). When operating on integer data types, the signedness of each input is controlled independently by `altfmt_A` and `altfmt_B` (0 = signed, 1 = unsigned; see Section 36.4.4); the accumulator C is always signed. Integer accumulation wraps modulo 2^{SEW} .

Table 72 show how the tile dimensions follow from the widening, λ , and LMUL settings.

Table 72. Tile dimensions K_{eff} , M , and N derived from the settings.

Parameter	Value	description
K_{eff}	$\lambda \times W \times \text{LMUL}$	number of input elements along the inner (K) dimension
$M = N$	$(\text{VLEN} \div \text{SEW}) \div \lambda$	rows = columns of the square accumulator tile



EMUL_C is independent of both W and LMUL; the C register group always occupies EMUL_C registers regardless of how A and B are configured.

The elements in each vector register are contiguous in the inner (**K**) direction, as depicted in Figure 8. Tiles A, B, and C elements are stored in row-major order.

The architectural input tiles A_{tile} and B_{tile} are both stored in row-major register layout. The tile A_{tile} has shape $M \times K_{\text{eff}}$, and B_{tile} has shape $N \times K_{\text{eff}}$. For every active accumulator element, the instruction computes:

$$C_{i,j} \leftarrow C_{i,j} + \sum_{k=0}^{K_{\text{eff}}-1} A_{\text{tile}}[i,k] B_{\text{tile}}^T[k,j]$$

Equivalently, the matrix operation is:

$$C \leftarrow C + A_{\text{tile}} \times B_{\text{tile}}^T$$

The contents of the vs2 register group have two equivalent matrix interpretations:

- B_{tile} has shape $N \times K_{\text{eff}}$ and is stored in row-major register layout.
- B_{tile}^T has shape $K_{\text{eff}} \times N$ and is stored in column-major register layout.

The two views refer to the same register contents:

$$B_{\text{tile}}^T[k,j] = B_{\text{tile}}[j,k]$$

Descriptions involving output-column selection and dot products generally use the B_{tile}^T view, because column j of B_{tile}^T contributes directly to column j of C .

Consequently, if $A_{\text{tile}} = B_{\text{tile}}$, the instruction computes:

$$C \leftarrow C + A_{\text{tile}} A_{\text{tile}}^T$$

To compute a conventional matrix product $C \leftarrow C + A_{\text{matrix}} B_{\text{matrix}}$, where B_{matrix} has shape $K_{\text{eff}} \times N$, software populates the architectural B tile such that:

$$B_{\text{tile}}[j,k] = B_{\text{matrix}}[k,j]$$

Thus, B_{tile} contains the transpose of the corresponding B_{matrix} panel:

$$B_{\text{tile}} = B_{\text{matrix}}^T$$

If B_{matrix} is stored in column-major order in memory, its columns are contiguous and may be loaded directly into the rows of B_{tile} using an order-preserving tile load. If B_{matrix} is stored in row-major order, a transposing tile load may be used to construct B_{tile} .

This register-tile interpretation permits matrix multiplication to be implemented using either inner-product or outer-product microarchitectures and naturally supports higher-rank updates expressed as sums of outer products.

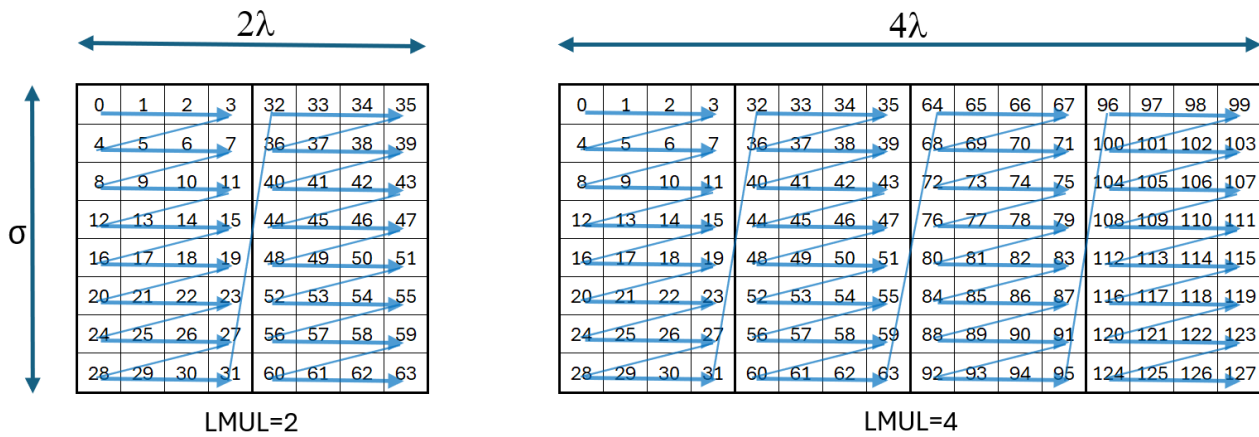


Figure 9. LMUL scaling of matrix tiles for LMUL=2 (left) and LMUL=4 (right).

LMUL scales the tile along the K dimension only, increasing the effective K dimension of the tile by a factor of LMUL. The blue arrows indicate the contiguous elements in the vector register groups that form the tiles in the ordering of element-wise 1D vector operations.



Implementation note: Because LMUL extends only the K dimension of the A and B input tiles, it does not change the physical C accumulator tile shape or EMUL_C. This allows an implementation to reuse the same C accumulator datapath across different A/B register-group sizes.

For example, an implementation can process an LMUL > 1 operation as an ordered sequence of LMUL=1 steps, while preserving the architectural result defined by this specification.

36.3. Extensions in Zvmm

36.3.1. Naming conventions

The Zvmm family uses two levels of naming.

Family-level extensions group related instructions and are named by function: **Zvmmm** (integer multiply-accumulate), **Zvvfmm** (floating-point multiply-accumulate), **Zvmmtls** (tile load/store), and **Zvmmtts** (transposing tile load/store). Implementing a family-level extension requires support for all instructions in that family.

Individual extensions identify a specific input/output type combination and follow the scheme **Zvv** + *input-type* + *output-type* + **mm**, where the output-type is omitted when it is the same as the input type. The type tokens are:

- Integer: **i4**, **i8**, **i16**, **i32**, **i64**
- Floating-point: **ofp4**, **ofp8**, **fp16**, **bf16**, **fp32**, **fp64**
- Microscaling (E8M0-scaled, block size 32): prefix **x** — e.g., **xofp8**, **xi8**
- Microscaling (E8M0-scaled, block size 16): prefix **xn** — e.g., **xnofp8**, **xni8**

For first-read clarity, the type tokens used in extension names have the following meanings:

Token	Meaning
ofp4	4-bit floating-point input format OFP4, currently E2M1.

Token	Meaning
ofp8	8-bit floating-point input format OFP8, either E4M3 or E5M2.
bf16	BFloat16.
fp16, fp32, fp64	IEEE binary16, binary32, and binary64.
x...	E8MO-microscaled form with block size 32.
xn...	E8MO-microscaled form with block size 16.
MXFP4, MXFP8	E8MO-scaled OFP4 or OFP8 data.
MXINT4, MXINT8	E8MO-scaled signed 4-bit or 8-bit integer data.

Examples: **Zvvi8i32mm** (Int8 inputs, Int32 accumulator), **Zvvfp16fp32mm** (FP16 inputs, FP32 accumulator), **Zvvxofp8fp16mm** (MXFP8 inputs with BS=32, FP16 accumulator). When the accumulator type matches the input type (e.g., **Zvvi16mm**, **Zvvfp32mm**), the output token is dropped.

36.3.2. Computational extensions

Table [Table 73](#) lists the computational extensions in the Zvvm family which are not related to microscaling. The extensions specific to microscaling are listed separately in [Section 36.12](#).

Table 73. Computational extensions in the Zvvm family of Integrated Matrix extensions.

Extension	Dependencies	Multiplicand Types	Accumulator Type
Zvvi4i8mm	Zve32x	[U]Int4, [U]Int4	Int8
Zvvi4i16mm	Zve32x	[U]Int4, [U]Int4	Int16
Zvvi4i32mm	Zve32x	[U]Int4, [U]Int4	Int32
Zvvi8mm	Zve32x	[U]Int8, [U]Int8	Int8
Zvvi8i16mm	Zve32x	[U]Int8, [U]Int8	Int16
Zvvi8i32mm	Zve32x	[U]Int8, [U]Int8	Int32
Zvvi8i64mm	Zve64x	[U]Int8, [U]Int8	Int64
Zvvi16mm	Zve32x	[U]Int16, [U]Int16	Int16
Zvvi16i32mm	Zve32x	[U]Int16, [U]Int16	Int32
Zvvi16i64mm	Zve64x	[U]Int16, [U]Int16	Int64
Zvvi32mm	Zve32x	[U]Int32, [U]Int32	Int32
Zvvi32i64mm	Zve64x	[U]Int32, [U]Int32	Int64
Zvvi64mm	Zve64x	[U]Int64, [U]Int64	Int64
Zvvofp4ofp8mm	Zve32f	OFP4 (E2M1), OFP4 (E2M1)	OFP8
Zvvofp4fp16mm	Zve32f	OFP4 (E2M1), OFP4 (E2M1)	IEEE binary16
Zvvofp4bf16mm	Zve32f	OFP4 (E2M1), OFP4 (E2M1)	BFloat16
Zvvofp4fp32mm	Zve32f	OFP4 (E2M1), OFP4 (E2M1)	IEEE binary32

Extension	Dependencies	Multiplicand Types	Accumulator Type
Zvvofp8mm	Zve32f	OFP8 (E4M3 or E5M2), OFP8 (E4M3 or E5M2)	OFP8
Zvvofp8fp16mm	Zve32f	OFP8 (E4M3 or E5M2), OFP8 (E4M3 or E5M2)	IEEE binary16
Zvvofp8bf16mm	Zve32f	OFP8 (E4M3 or E5M2), OFP8 (E4M3 or E5M2)	BFloat16
Zvvofp8fp32mm	Zve32f	OFP8 (E4M3 or E5M2), OFP8 (E4M3 or E5M2)	IEEE binary32
Zvvofp8fp64mm	Zve64d	OFP8 (E4M3 or E5M2), OFP8 (E4M3 or E5M2)	IEEE binary64
Zvvfp16mm	Zve32f	IEEE binary16, IEEE binary16	IEEE binary16
Zvvfp16fp32mm	Zve32f	IEEE binary16, IEEE binary16	IEEE binary32
Zvvfp16fp64mm	Zve64d	IEEE binary16, IEEE binary16	IEEE binary64
Zvvbf16mm	Zve32f	BFloat16, BFloat16	BFloat16
Zvvbf16fp32mm	Zve32f	BFloat16, BFloat16	IEEE binary32
Zvvbf16fp64mm	Zve64d	BFloat16, BFloat16	IEEE binary64
Zvvfp32mm	Zve32f	IEEE binary32, IEEE binary32	IEEE binary32
Zvvfp32fp64mm	Zve64d	IEEE binary32, IEEE binary32	IEEE binary64
Zvvfp64mm	Zve64d	IEEE binary64, IEEE binary64	IEEE binary64

36.4. New fields in the Vector Type (**vtype**) Register

The Zvvm family of Integrated Matrix extensions defines the following additional fields in the **vtype** CSR:

- **lambda[2:0]** at **vtype[XLEN-2:XLEN-4]** is the Selected Lambda
- **bs** at **vtype[XLEN-5]** is the Block Size selector for microscaling
- **altfmt_A** at **vtype[XLEN-6]** is the Alternate Format for input A
- **altfmt_B** at **vtype[XLEN-7]** is the Alternate Format for input B

These IME fields occupy the high end of **vtype**, immediately below **vill** (**vtype[XLEN-1]**). They are distinct from the base **altfmt** field defined by Zvfbfa — which selects the floating-point format of the output accumulator C in conjunction with **vsew** (see [Section 36.4.3](#)) and resides in the low part of **vtype** at the bit position assigned by Zvfbfa. **altfmt** (output format, Zvfbfa) does not collide with the IME-defined **altfmt_A** / **altfmt_B** (input formats, high bits).



*On reset, and whenever **vtype.vill** = 1, the IME **vtype** fields read as **lambda[2:0]** = 000, **bs** = 0, **altfmt_A** = 0, and **altfmt_B** = 0. This is the unconfigured state in which **lambda[2:0]** = 000 is observable; see [\[integrated-matrix-lambda\]](#). A configuration instruction on an IME-legal (**VLEN**, **SEW**) configuration always leaves **lambda[2:0]** holding a supported nonzero value. Outside the IME-legal domain, no nonzero lambda is permissible and the field remains 000.*

36.4.1. Interaction with vector configuration instructions

The IME fields **lambda**[2:0], **bs**, **altfmt_A**, and **altfmt_B** are outside the **vtypei** immediate field of **vsetvli** and **vsetivli**.

The **vsetvli** and **vsetivli** instructions write the standard Vector fields encoded in **vtypei**, including **vsew**, **vlmul**, **vta**, and **vma**, and set **vl** according to the rules of the base Vector extension. They do not directly write the IME fields **lambda**[2:0], **bs**, **altfmt_A**, or **altfmt_B**.

On execution of **vsetvli** or **vsetivli**, the **bs**, **altfmt_A**, and **altfmt_B** fields retain their previous values.

On execution of **vsetvli** or **vsetivli**, the current value of **lambda**[2:0] is evaluated against the resulting **VLEN** and **SEW** configuration. If the current nonzero lambda value is supported for the resulting configuration, it is retained. Otherwise, if the resulting configuration is in the IME-legal domain (see [\[integrated-matrix-lambda\]](#)), the implementation shall select the largest supported nonzero lambda value for that configuration; the supported set is non-empty there, so the selection yields a nonzero value. If the resulting configuration is outside the IME-legal domain (no nonzero lambda is permissible), **lambda**[2:0] is set to 000.

LMUL does not affect whether a lambda value is supported. Therefore, a change to **LMUL** alone shall not change **lambda**[2:0].

The **vsetvl** instruction writes a complete **vtype** value from **rs2**, including the IME fields **lambda**[2:0], **bs**, **altfmt_A**, and **altfmt_B**. Therefore, **vsetvl** is the architectural mechanism for setting or restoring IME **vtype** fields that are outside the **vsetvli** and **vsetivli** immediate field.

The **lambda**[2:0] bits supplied to **vsetvl** in **rs2** are interpreted as a lambda write request for the resulting **VLEN** and **SEW** configuration.

A nonzero request is processed according to the nonzero-write rule in [\[integrated-matrix-lambda\]](#). A zero request is processed according to the preserve-or-initialize rule in that section; it is not necessarily written as a literal zero.

The **bs**, **altfmt_A**, and **altfmt_B** fields are written as supplied by **rs2**. The legality of particular **bs**, **altfmt_A**, and **altfmt_B** combinations is checked by the IME instruction that uses those fields.

*The IME **vtype** fields fall into two WARL classes, which is why they behave differently across a **vsetvli**/**vsetivli**:*



- **lambda**[2:0] is geometry-constrained: its set of supported values is a function of (**VLEN**, **SEW**), so a change to **SEW** can invalidate the current value. It therefore follows a preserve-or-initialize rule. Within the IME-legal domain, a configuration instruction always leaves it holding a supported nonzero value. Outside the IME-legal domain, no nonzero lambda is permissible and the field remains 000.
- **bs**, **altfmt_A**, and **altfmt_B** are not geometry-constrained by **VLEN**/**SEW**; any encoding is representable. They are therefore retained verbatim, and the legality of a particular combination is deferred to the IME instruction that consumes them.

*Because the two classes are validated at different points, a **vtype** in which an auto-selected **lambda** and retained **bs/altfmt_*** form an unsupported combination is not detected at configuration time; it is rejected (illegal-instruction) by the IME multiply-accumulate or load/store instruction that uses the combination.*



*The base Vector specification describes **vsetvl** as the configuration instruction used for context restore because it takes a complete **vtype** value from **rs2**. In IME, **vsetvl** is also the*

*standard configuration instruction used to write IME **vtype** fields that are above the **vtypei** immediate field of **vsetvli** and **vsetivli**.*

36.4.2. Selected lambda (**lambda[2:0]**) field

The **lambda[2:0]** field is a 3-bit WARL read/write field in the **vtype** CSR that holds the dynamic selected lambda. Lambda determines the geometry of the accumulator tile C and, together with **W** and **LMUL**, determines:

$$K_{\text{eff}} = \text{lambda} * W * \text{LMUL}$$

The set of supported nonzero lambda values is an implementation-defined function of **VLEN** and **SEW**. **LMUL** does not affect whether a lambda value is supported.

A nonzero lambda is *architecturally permissible* for a given (**VLEN**, **SEW**) only when it yields a valid C register group, i.e. when $\text{EMUL_C} = \text{VLEN} \div (\text{SEW} \times \lambda^2)$ is an integer in $\{1, 2, 4, 8, 16\}$ (equivalently, when the resulting tile has $M = N = \text{VLEN} \div (\text{SEW} \times \lambda) \geq 1$). The *IME-legal configuration domain* is the set of (**VLEN**, **SEW**) for which at least one architecturally permissible nonzero lambda exists. Note that lambda = 1 is permissible only when $\text{SEW} \leq \text{VLEN} \leq 16 \times \text{SEW}$; for $\text{VLEN} > 16 \times \text{SEW}$ the minimum permissible lambda is larger than 1, and for $\text{VLEN} < \text{SEW}$ no lambda is permissible (the configuration lies outside the IME-legal domain).

Implementation requirement. For every (**VLEN**, **SEW**) in the IME-legal configuration domain, an implementation shall support at least one architecturally permissible nonzero lambda value. Consequently, within that domain the set of supported nonzero lambda values is non-empty, **lambda[2:0]** always holds a supported nonzero value after a configuration instruction, and **000** is never produced as a selected value.

For a (**VLEN**, **SEW**) outside the IME-legal domain, no nonzero lambda is permissible; **lambda[2:0]** remains **000** and any IME instruction that requires a dynamic lambda raises an illegal-instruction exception.

One discovery mechanism is to attempt to program each nonzero lambda encoding and read back the value that is retained.



*This **vtype**-write discovery path WARL-clamps an unsupported nonzero request to a supported value (see the write rules below). The instruction-immediate lambda field of a tile load/store behaves oppositely: an unsupported nonzero immediate is not clamped and raises an illegal-instruction exception (see [Section 36.8.3.1](#)). For discovery, the useful path is the **vtype**-write path, because only that path reads back a clamped value.*

Table 74. **lambda[2:0]** (selected lambda) encoding

lambda[2:0]			Selected lambda
0	0	0	<i>no selected lambda</i>
0	0	1	1
0	1	0	2
0	1	1	4
1	0	0	8
1	0	1	16
1	1	0	32
1	1	1	64

A write of **000** is a preserve-or-initialize request:

- If the current nonzero lambda value is supported for the resulting (**VLEN**, **SEW**) configuration, that value is retained.
- Otherwise, if the configuration is in the IME-legal domain, the implementation shall select the largest supported nonzero lambda value for the resulting configuration; if the configuration is outside the IME-legal domain, **lambda[2:0]** is set to **000**.

Within the IME-legal domain the supported set is non-empty, so a **000** write results in a supported nonzero selected value.

A nonzero write requests the corresponding lambda value.

If the requested value is supported for the resulting (**VLEN**, **SEW**) configuration, that value is retained.

If the requested value is not supported but the configuration is in the IME-legal domain, the implementation shall select the largest supported nonzero lambda value that is less than or equal to the requested value; if no supported value is less than or equal to the request, it shall select the smallest supported nonzero lambda value.

If the configuration is outside the IME-legal domain, no nonzero lambda is permissible and **lambda[2:0]** is set to **000**.

A configuration instruction on a configuration in the IME-legal domain never selects **lambda[2:0] = 000**; the encoding is selected only outside that domain, and is otherwise observable only in an as-yet-unconfigured **vtype**. Base Vector instructions may execute when **lambda[2:0] = 000**; an IME instruction that requires a dynamic lambda raises an illegal-instruction exception when **lambda[2:0] = 000**.

*The **000** lambda encoding carries two distinct meanings, depending on where it appears:*



- As a **vtype** write request (the **lambda** bits supplied to **vsetvl**, or the value written by **__riscv_vsetlambda(0)**): it is a preserve-or-initialize request. Within the IME-legal domain, this request preserves a valid current lambda or selects the largest supported nonzero lambda. Outside the IME-legal domain, no nonzero lambda is permissible and the field remains **000**.
- As a tile load/store instruction immediate: it means "use **vtype.lambda`**" for that instruction. It does not write **vtype.lambda** and is distinct from a **vtype** write of zero.

*Thus, **000** is not a selected nonzero lambda value. It denotes that no selected lambda is available, which can occur in an unconfigured **vtype**, when **vtype.vill = 1**, or outside the IME-legal (**VLEN**, **SEW**) domain.*

36.4.3. Alternate floating-point format for the output (**altfmt**)

The Zvbm Integrated Matrix floating-point extensions use the **altfmt** field (as defined by Zvfbfa) to select the floating-point format of the elements in the output accumulator matrix C, in conjunction with **vsew**.

Table 75. Output data type encoding (**altfmt** × **vsew[2:0]**)

altfmt	vsew[2:0]			FP format
0	0	0	0	E4M3 (OFP8)
1	0	0	0	E5M2 (OFP8)

altfmt	vsew[2:0]			FP format
0	0	0	1	IEEE binary16
1	0	0	1	BFloat16
0	0	1	0	IEEE binary32
1	0	1	0	<i>reserved</i>
0	0	1	1	IEEE binary64
1	0	1	1	<i>reserved</i>
X	1	X	X	<i>reserved</i>

36.4.4. Alternate formats for inputs (**altfmt_A**, **altfmt_B**)

The **altfmt_A** and **altfmt_B** fields are 1-bit read/write fields added to the **vtype** CSR by the Zvwm family of Integrated Matrix extensions. They select the interpretation of elements in input matrices A and B, respectively. Their meaning depends on whether the instruction is a floating-point or integer multiply-accumulate.

The **altfmt_A** and **altfmt_B** bits are located at **vtype**[XLEN-6] and **vtype**[XLEN-7], immediately below the **bs** field. These positions are outside the **vtypei** immediate field of **vsetvli** and **vsetivli**; they cannot be written directly by those instructions. They are written by **vsetvl**, which supplies a full **vtype** value in **rs2**, and are preserved by **vsetvli** and **vsetivli** as described in [Section 36.4.1](#).

36.4.4.1. Floating-point instructions

For floating-point multiply-accumulate instructions, **altfmt_A** and **altfmt_B** select the floating-point format of the corresponding input operand. These fields are only significant for sub-word floating-point input operands (OFP8, 16-bit FP); for IEEE binary32 and IEEE binary64 inputs they are ignored.

Table 76. FP input data type encoding (*altfmt_A or altfmt_B × input element width*)

altfmt_A / altfmt_B	Input element width	FP format
0	4-bit	E2M1 (OFP4)
1	4-bit	<i>reserved</i>
0	8-bit	E4M3 (OFP8)
1	8-bit	E5M2 (OFP8)
0	16-bit	IEEE binary16
1	16-bit	BFloat16

36.4.4.2. Integer instructions

For integer multiply-accumulate instructions, **altfmt_A** and **altfmt_B** select whether the corresponding input operand is interpreted as signed or unsigned.

Table 77. Integer input signedness encoding (*altfmt_A or altfmt_B*)

altfmt_A / altfmt_B	Interpretation
0	Signed
1	Unsigned

36.4.5. Microscaling block size (bs)

The **bs** field is a 1-bit read/write field in the **vtype** CSR that selects the block size used for microscaling operations (**vm=0**). When **vm=1**, the **bs** field is ignored.

Table 78. Block size encoding

bs	Block size
0	32 elements
1	16 elements

The **bs** bit is located at **vtype[XLEN-5]**, immediately below **lambda[2:0]**. Like **altfmt_A** and **altfmt_B**, this position is outside the **vsetvli** immediate field.

36.5. Storage formats

36.5.1. Element packing in input tiles

For non-widening multiply-accumulate instructions (**W=1**), the input elements A and B have the same width as the accumulator (**SEW**), and no packing occurs.

For widening multiply-accumulate instructions (**W>1**), each input register group holds elements at the effective input element width $EEW = SEW \div W$. Because tile load instructions always transfer data at **SEW** granularity, every loaded **SEW**-bit storage position contains **W** contiguous packed narrow elements that the multiply-accumulate instruction consumes as a sub-dot-product.

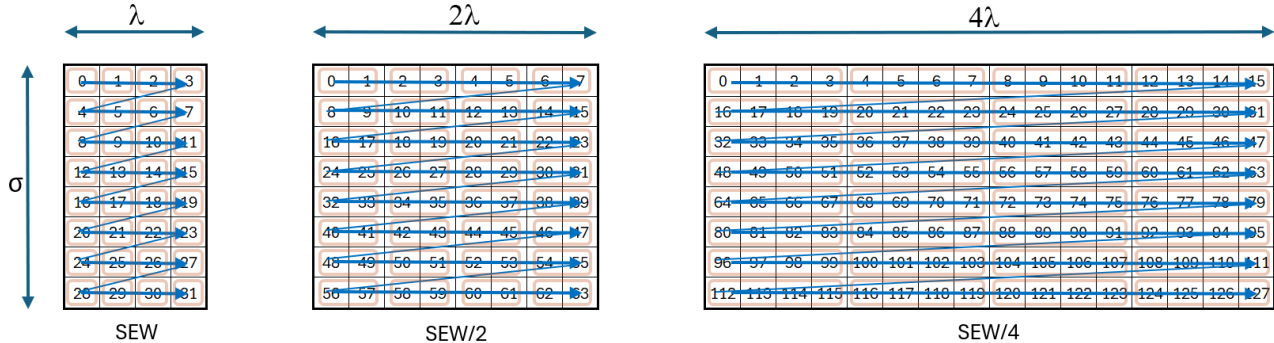


Figure 10. Element distribution and tile geometry example for $L=32$, **SEW** wide elements (left), two **SEW/2** wide elements packed per **SEW** (middle), and four **SEW/4** wide elements per **SEW** (right). Widening by a factor **W** (with the corresponding packing of **W** narrow elements per **SEW**-wide slot) increases the effective **K** dimension of the tile by a factor of **W**.

36.5.1.1. Byte-sized and wider elements ($EEW \geq 8$)

When the input element width is 8 bits or wider ($EEW \in \{8, 16, 32, 64\}$), the elements within the input register group follow standard RISC-V **V** element ordering: element k at width **EEW** occupies bits $[k \times EEW + EEW - 1 : k \times EEW]$ of the register group. This is identical to the layout produced by a vector load at the same element width (e.g., **vle8.v** for **EEW=8**) and requires no special data preparation beyond the tile load itself.

36.5.1.2. Sub-byte elements ($EEW = 4$)

Four-bit input elements (**OFP4** (**E2M1**), **Int4**) are packed two per byte in little-endian nibble order:

- The *even-indexed* element (element $2n$) occupies the **lower** nibble [3:0] of byte n .
- The *odd-indexed* element (element $2n + 1$) occupies the **upper** nibble [7:4] of byte n .

This extends the standard RISC-V V element-indexing convention to sub-byte widths: element k at width 4 occupies bits $[4k + 3 : 4k]$ of the register group, consistent with the formula for all element widths.

Because tile loads operate at SEW (≥ 8 bits), each loaded byte already contains a naturally ordered pair of 4-bit elements. Software preparing input data in memory must pack adjacent elements within each byte accordingly.

36.6. Zvmm: Extension for matrix multiplication on vector register groups interpreted as 2D integer matrix tiles

The Zvmm family of extensions provides instructions that perform matrix multiply-accumulate on integer data, computing $C \leftarrow C + A \times B^T$, where A, B, and C are matrix tiles held in the vector register file.

The architectural A and B input tiles are denoted A_{tile} and B_{tile} . Both are stored in row-major register layout. The instructions compute:

$$C \leftarrow C + A_{\text{tile}} \times B_{\text{tile}}^T$$

The following table lists the integer matrix tile multiplication instructions. The arguments are:

- **vd**: Destination vector register group containing the row-major architectural C matrix tile.
- **vs1**: Source vector register group containing the row-major architecture A matrix tile.
- **vs2**: Source vector register group containing the row-major architectural B matrix tile.

Mnemonic	W	A/B element width	C element width
vmacc.vv vd, vs1, vs2	1	SEW	SEW
vmmacc.vv vd, vs1, vs2	2	SEW/2	SEW
vqmmacc.vv vd, vs1, vs2	4	SEW/4	SEW
v8wmmacc.vv vd, vs1, vs2	8	SEW/8	SEW

Vector masking is not supported on matrix multiply-accumulate instructions.

For **vmacc.vv**, **vm=0** is *reserved*.

For **vmmacc.vv**, **vqmmacc.vv**, and **v8wmmacc.vv**, **vm=1** selects the integer-accumulate instruction, while **vm=0** encodes **vwimmacc.vv**, **vfqimmacc.vv**, and **vf8wimmacc.vv**, respectively; these are the integer-input, floating-point-accumulate microscaled forms described in [Section 36.7.3](#) and [Section 36.13.3](#).

A **vm=0** integer-input encoding is legal only when the corresponding row and selected MX block-size cell in [Section 36.13.3](#) name the required microscaling extension and that extension is implemented. Otherwise, an illegal-instruction exception is raised.

Matrix multiply-accumulate instructions cannot be interrupted and resumed: they require **vstart** = 0 at the start of execution, and if **vstart** is non-zero when such an instruction begins, an illegal-instruction exception is raised.

36.6.1. Arithmetic Considerations

36.6.1.1. Integer accumulation

For the unscaled integer multiply-accumulate instructions, all intermediate results are reduced modulo 2^{SEW} . Because modular addition is both associative and commutative, the final result is uniquely defined regardless of the accumulation order or grouping factor.

The microscaled integer-input forms (`vfwimmac.vv`, `vfqimmac.vv`, `vf8wimmac.vv`) use the separately specified exact integer block-dot-product path. For each microscaling block, the integer products in that block are summed exactly. The exact integer block sum is then converted to the C accumulator format `fmt_C` using the dynamic rounding mode `frm`.

The paired block scales are decoded into `fmt_C`, and their product is formed using the accumulator-format floating-point multiplication specified by the operation pseudocode. The converted block sum is multiplied by that combined block scale, again in `fmt_C`, and the scaled value is accumulated into the floating-point C accumulator using `frm`.

This floating-point conversion, scaling, and accumulation may raise floating-point exception flags. The result may depend on `frm`, `fmt_C`, the scale values, the selected block size, and the specified NaN/exception behavior. It is not governed by the floating-point-input grouping and partial-sum rounding model defined later in [Section 36.7.2.2](#); in particular, it is not controlled by the `G`, `psm`, or `rnd` parameters of that model.

36.7. Zvfmfm: Extension for matrix multiplication on vector register groups interpreted as 2D floating-point matrix tiles

The `Zvfmfm` family of extensions provides floating-point matrix multiply-accumulate instructions, computing $C \leftarrow C + A \times B^T$. A, B, and C are matrix tiles held in the vector register file.

The architectural A and B input tiles are denoted A_{tile} and B_{tile} . Both are stored in row-major register layout. The instructions compute:

$$C \leftarrow C + A_{\text{tile}} \times B_{\text{tile}}^T$$

The following table lists the floating-point matrix tile multiplication instructions. The arguments are:

- **vd**: Destination vector register group containing the C matrix tile.
- **vs1**: Source vector register group containing the A matrix tile.
- **vs2**: Source vector register group containing the B matrix tile.
- **v0.scale**: Block scales for A and B operands in MX-scaling mode.

The operand spelling `v0.scale` denotes the use of vector register `v0` as the source of paired microscaling block-scale values. The layout and semantics of these scale values are defined in [Section 36.7.3](#).

Mnemonic	W	A/B element width	C element width	Remark
<code>vfmacc.vv vd, vs1, vs2</code>	1	SEW	SEW	floating-point A/B tiles only
<code>vfwmmacc.vv vd, vs1, vs2[, v0.scale]</code>	2	SEW/2	SEW	floating-point and MX-scaled A/B tiles

Mnemonic	W	A/B element width	C element width	Remark
vfqmmacc.vv <i>vd, vs1, vs2[, v0.scale]</i>	4	SEW/4	SEW	floating-point and MX-scaled A/B tiles
vf8wmmacc.vv <i>vd, vs1, vs2[, v0.scale]</i>	8	SEW/8	SEW	floating-point and MX-scaled A/B tiles
vfwmacc.vv <i>vd, vs1, vs2, v0.scale</i>	2	SEW/2	SEW	only MX-scaled integer A/B tiles
vfqimmacc.vv <i>vd, vs1, vs2, v0.scale</i>	4	SEW/4	SEW	only MX-scaled integer A/B tiles
vf8wimmacc.vv <i>vd, vs1, vs2, v0.scale</i>	8	SEW/8	SEW	only MX-scaled integer A/B tiles

Vector masking is not supported on matrix multiply-accumulate instructions.

For **vfmmacc.vv**, *vm=0* is reserved.

For **vfwmacc.vv**, **vfqmmacc.vv**, and **vf8wmmacc.vv**, *vm=1* selects normal unscaled operands and *vm=0* requests microscaled MX operands using *v0.scale*.

A *vm=0* encoding is legal only when the corresponding row and selected MX block-size cell in [Table 87](#) name the required microscaling extension or extensions and all such extensions are implemented. Otherwise, an illegal-instruction exception is raised. See [Section 36.7.3](#) for the arithmetic and scale-layout semantics.

Matrix multiply-accumulate instructions cannot be interrupted and resumed: they require *vstart* = 0 at the start of execution, and if *vstart* is non-zero when such an instruction begins, an illegal-instruction exception is raised.

The floating-point format of each operand tile is controlled by fields in *vtype*:

- *altfmt_A* and *altfmt_B* select the FP format for A and B input elements, interpreted in conjunction with the input element width $SEW \div W$ (see [Section 36.4.4](#)). The A and B input tiles may use different formats (*altfmt_A* $\neq$ *altfmt_B*); see [Section 36.7.1](#) for extension requirements.
- *altfmt* (as defined by *Zvfbfa*) selects the FP format for the C accumulator elements, interpreted in conjunction with SEW (see [Section 36.4.3](#)).

36.7.1. Mixed-format inputs (*altfmt_A* $\neq$ *altfmt_B*)

Mixed-format inputs (*altfmt_A* $\neq$ *altfmt_B*) are permitted for all floating-point multiply-accumulate instructions. Each $A \times B$ product is defined by the exact mathematical product of the corresponding input values, and the floating-point result of the overall matrix multiply-accumulate operation is determined according to [Section 36.7.2](#). The valid mixed-format combinations are listed in [Table 87](#).

36.7.1.1. Mixed-format multiplication semantics

Each $A \times B$ element product is defined by the exact mathematical product of the corresponding input values, regardless of whether *fmt_A* equals *fmt_B*. The floating-point result of the overall matrix multiply-accumulate operation, including any intermediate rounding, grouping, and partial-sum formation, is then determined according to [Section 36.7.2](#). In particular, the implementation-defined mapping from (SEW, W, *lambda*) to (G, *psm*, *rnd*) determines how exact products are grouped into partial sums and whether those

partial sums are rounded before accumulation into C.

The exact product of two floating-point values with p_A and p_B significand bits has at most $p_A + p_B$ significand bits. An implementation may compute the product in any internal format with at least that many significand bits.

Table 79. Significand widths (p) of supported formats

Format	p (significand bits)
E2M1 (OFP4)	2
E4M3 (OFP8)	4
E5M2 (OFP8)	3
IEEE binary16	11
BFloat16	8
IEEE binary32	24
IEEE binary64	53

When $p_A + p_B \leq p_C$, the exact product is representable in **fmt_C**. Therefore, if the accumulation model later requires a value containing that product to be rounded to the C accumulator format, the product itself does not introduce additional rounding error. This holds for all mixed-format widening combinations. For example, $E4M3 \times E5M2 \rightarrow FP16$ ($4 + 3 = 7 \leq 11$) and $FP16 \times BF16 \rightarrow FP32$ ($11 + 8 = 19 \leq 24$).

An implementation need not use a standard IEEE 754 type for the internal product; any representation that captures the exact product (e.g., an integer significand-exponent pair) suffices, provided the overall matrix multiply-accumulate result matches the semantics defined in [Section 36.7.2](#).

All floating-point matrix multiply-accumulate instructions, including mixed-format and microscaling variants, shall follow the normative rules in [Section 36.7.2](#) for grouping, partial-sum formation, optional rounding of partial sums, and final accumulation into C.

Final accumulation into the C accumulator format uses the dynamic rounding mode from **frm**. Any intermediate grouping, partial-sum formation, and optional rounding of partial sums follow [Section 36.7.2](#). Floating-point exception flags accumulate in **fflags**.

The extension requirements for mixed-format operation depend on the input element width:

36.7.1.2. Sub-word inputs (OFP4, OFP8)

For 4-bit inputs, only OFP4 (E2M1) is supported (**altfmt_A** = **altfmt_B** = 0). The encoding **altfmt_A** = 1 or **altfmt_B** = 1 is reserved for 4-bit inputs.

Likewise, for 8-bit inputs, **altfmt_A** and **altfmt_B** independently select E4M3 or E5M2. All four combinations — including mixed $E4M3 \times E5M2$ — are covered by the same OFP8 extension for the given output format and are permitted on all four instructions (**vfmmacc.vv**, **vfwmacc.vv**, **vfqmmacc.vv**, **vf8wmacc.vv**). Each $A \times B$ product is defined by the exact mathematical product of the corresponding input values. Grouping, partial-sum formation, optional rounding of partial sums, and final accumulation into C follow [Section 36.7.2](#).

36.7.1.3. 16-bit inputs (IEEE binary16, BFloat16)

For 16-bit inputs, `altfmt_A` and `altfmt_B` independently select IEEE binary16 or BFloat16. When both are the same, a single extension suffices (`Zvvp16mm*` for IEEE binary16, `Zvvpbf16mm*` for BFloat16), and all three instructions (`vfmacc.vv`, `vfwmacc.vv`, `vfqmmacc.vv`) are available.

When `altfmt_A` $\neq$ `altfmt_B` (one input is IEEE binary16 and the other is BFloat16), the mixed combination is permitted on all three instructions (`vfmacc.vv`, `vfwmacc.vv`, `vfqmmacc.vv`) — the two input formats share the same element width, so no additional encoding space or hardware extension is required. Each $A \times B$ product is defined by the exact mathematical product of the corresponding input values. Grouping, partial-sum formation, optional rounding of partial sums, and final accumulation into C follow [Section 36.7.2](#).

For the mixed cases, *both* the IEEE binary16 and BFloat16 extensions for the same instruction and output format are required. The instruction is legal only when both extensions are present; if either is missing, the instruction raises an illegal-instruction exception.

Table 80. Extension requirements for mixed FP16/BF16 inputs

Instruction	Output format	Required extensions (both must be present)
<code>vfmacc.vv</code>	FP16 (SEW=16)	<code>Zvvp16mm</code> AND <code>Zvvpbf16mm</code>
<code>vfwmacc.vv</code>	BF16 (SEW=16)	<code>Zvvp16mm</code> AND <code>Zvvpbf16mm</code>
<code>vfmacc.vv</code>	FP32 (SEW=32)	<code>Zvvp16fp32mm</code> AND <code>Zvvpbf16fp32mm</code>
<code>vfqmmacc.vv</code>	FP64 (SEW=64)	<code>Zvvp16fp64mm</code> AND <code>Zvvpbf16fp64mm</code>

If the required extensions are not both present, the instruction raises an illegal-instruction exception.

36.7.1.4. 32-bit and wider inputs

For IEEE binary32 and IEEE binary64 inputs, `altfmt_A` and `altfmt_B` are ignored (there is only one format per width), so mixed-format operation does not apply.

Final accumulation into the C accumulator format uses the dynamic rounding mode from `frm`. Any intermediate grouping, partial-sum formation, and optional rounding of partial sums follow [Section 36.7.2](#). Floating-point exception flags are accumulated into `fflags`.

The K-dimension, tile-dimension formulae, `EMUL_C`, and instruction-to-widening-factor mapping are the same as for the integer family (see [Matrix-multiplication instructions \(integer\)](#)); the floating-point mnemonics are `vfmacc.vv` (W=1), `vfwmacc.vv` (W=2), `vfqmmacc.vv` (W=4), and `vf8wmacc.vv` (W=8).

As with the integer multiply-accumulate instructions, vector masking is not supported. When `vm=0`, the instruction does not apply a vector mask; instead, `v0` supplies paired block-scale factors for microscaling (see [Section 36.7.3](#)).

36.7.1.5. Mixed-format inputs arithmetic considerations

Whenever mixed formats are used in the computational instructions, one must follow the guidelines in [Section 36.7.2](#).

36.7.2. Arithmetic considerations

Each multiply-accumulate instruction computes, for every output element $C_{i,j}$:

$$C_{i,j} \leftarrow C_{i,j} + \sum_{k=0}^{K_{\text{eff}}-1} A_{\text{tile}}[i,k] \times B_{\text{tile}}^T[k,j]$$

where $K_{\text{eff}} = \lambda \times W \times \text{LMUL}$. This section specifies how the K_{eff} product terms may be grouped and when intermediate rounding is permitted.

36.7.2.1. Sub-dot-products

When $W > 1$, the W narrow input-element pairs packed within one accumulator-width (SEW -bit) position form a natural computational unit called a *sub-dot-product*. The W multiplications of $(\text{SEW} \div W)$ -bit elements are performed and their products summed.

There are $K_{\text{eff}} \div W = \lambda \times \text{LMUL}$ sub-dot-products per output element.

When $W = 1$, each product of two SEW -bit values is exact at $2 \times \text{SEW}$ bits; a sub-dot-product consists of a single product term.

36.7.2.2. Accumulation and rounding model (floating-point)

The resulting value of element $C_{i,j}$ for any given LMUL must match the value computed by applying a series of multiply-accumulate instructions with $\text{LMUL}=1$, in increasing order of vector-register indices. Therefore, it is enough to specify the result of the computation

$$C_{i,j} \leftarrow C_{i,j} + \sum_{k=0}^{\lambda W - 1} A_{\text{tile}}[i,k] \times B_{\text{tile}}^T[k,j]$$

An implementation partitions the λ sub-dot-products for each output element into consecutive groups of G sub-dot-products. The implementation-defined grouping factor G is applied separately within each $\text{LMUL}=1$ step, so that a group shall not cross the boundary between two consecutive $\text{LMUL}=1$ steps. For unscaled floating-point multiply-accumulate instructions, every group within an $\text{LMUL}=1$ step has exactly G sub-dot-products, and the number of groups in the computation of $C_{i,j}$ above is $\lambda \div G$. For microscaled instructions, additional microscaling block boundaries apply; see [Section 36.7.3.2.2](#).

G must satisfy:

- G is a power of two;
- $1 \leq G \leq \lambda$. Since both λ and G are powers of two and $1 \leq G \leq \lambda$, G divides λ .
- For unscaled instructions, each group contains exactly G sub-dot-products and therefore defines a dot product of two vectors with $G \times W$ elements. For microscaled instructions, a group contains the actual group length g_{len} defined in [Section 36.7.3.2.2](#) and therefore defines a dot product of two vectors with $g_{\text{len}} \times W$ elements. In either case, the group dot product is reduced into a partial sum S according to an implementation-defined parameter $\text{psm} \in \{0, 1\}$.
 - If $\text{psm}=0$, the partial sum S is formed using exact computation: the contributing products and sums are computed in sufficiently precise internal form, without rounding to the C accumulator format, until the next rounding step defined by this specification.
 - If $\text{psm}=1$, the partial sum S is formed using an *implementation-defined* non-exact reduction of the group's products. The reduction may use any internal representation chosen by the implementation. An implementation that supports $\text{psm}=1$ for a given (SEW, W, λ) shall publish, alongside its $(G, \text{psm}, \text{rnd})$ disclosure, the specific reduction algorithm and the values of any parameters it takes; two implementations are bit-exactly compatible at $\text{psm}=1$ only if both disclose the same algorithm with the same parameters.

- Once the partial sum **S** is computed, and before it is accumulated into the result **C**, it is treated according to an implementation-defined parameter **rnd**.
 - If **rnd=frm**, the partial sum **S** is rounded to the **C** accumulator format using the rounding mode from **frm**.
 - If **rnd=rto**, the partial sum **S** is rounded to the **C** accumulator format using round-to-odd.
 - If **rnd=xct**, no rounding to the **C** accumulator format is applied to the partial sum **S** before the final accumulation into **C**.
- After each group, the partial sum **S**, or the rounded value derived from **S**, is accumulated into the running value of $C_{i,j}$ by computing

$$C_{i,j} \leftarrow \text{round}_{\text{frm}}(C_{i,j} + S).$$

The accumulation rounding is performed with the rounding mode from **frm**. If **rnd=xct**, the partial sum **S** is retained in sufficient internal precision so that the only rounding step in the update of $C_{i,j}$ by that group is the final $\text{round}_{\text{frm}}(C_{i,j} + S)$.

*The three **rnd** values are not numerically interchangeable.*

*When a group partial sum **S** is first rounded to the **C** accumulator format and then accumulated into $C_{i,j}$, the computation has two rounding points: one when the partial sum is rounded, and another when the accumulation*

$$\text{round}_{\text{frm}}(C_{i,j} + S)$$

is performed. This two-stage rounding can differ from an accumulation performed with the exact partial sum.



*When **rnd=xct**, the partial sum **S** is not rounded to the **C** accumulator format before accumulation. The exact internal partial sum is accumulated directly into $C_{i,j}$, so the intermediate rounding point is eliminated.*

*When **rnd=frm** or **rnd=rto**, the partial sum **S** is rounded to the **C** accumulator format before accumulation, so the two-stage rounding remains. However, **rnd=rto** preserves information about discarded nonzero bits in the intermediate rounded value. This makes **rnd=rto** less prone than **rnd=frm** to double-rounding artifacts under the subsequent accumulation, although it can still lose accuracy relative to **rnd=xct**.*

*Therefore, implementations should prefer **rnd=xct** when they can support exact internal partial sums through the final accumulation. When **rnd=xct** is not practical, **rnd=rto** is generally preferable to **rnd=frm** for reducing double-rounding effects.*

Supported values of **G**, **psm**, and **rnd** by an implementation may depend on **SEW**, **W**, and **lambda**.

An implementation must disclose all supported combinations by publishing a table of its mappings from (**SEW**, **W**, **lambda**) to (**G**, **psm**, **rnd**).

For a given (**SEW**, **W**, **lambda**) entry, the disclosed (**G**, **psm**, **rnd**) tuple applies uniformly to every legal floating-point input-format combination, **C** accumulator format, and unscaled or microscaled operation having that geometry. The **bs** setting does not select a different tuple.

This mapping applies only to floating-point-input instructions evaluated through the floating-point grouping and partial-sum model. The integer-input microscaled instructions (**vfwimmacc.vv**, **vfqimmacc.vv**, and **vf8wimmacc.vv**) use the separately specified exact integer block-dot-product path and are not governed by **G**, **psm**, or **rnd**.

Microscaling block boundaries may shorten an actual group to `g_len < G`, as described in [Section 36.7.3.2.2](#), but they do not change the disclosed `(G, psm, rnd)` tuple.

For each entry with `psm=1`, the disclosure must additionally identify the reduction algorithm and the values of any parameters it takes.

For each `(SEW, W, Lambda)` entry with `psm=1`, the implementation-specific architectural disclosure is complete only when it includes a SAIL fragment that provides a concrete definition of `fp_disclosed_reduce_group` and of any companion functions referenced by it for that entry.

The SAIL fragment shall cover every legal combination of `fmt_A`, `fmt_B`, and `fmt_C` represented by that entry, and both unscaled and microscaled execution where applicable. Any format-dependent reduction parameters shall be defined or derived unambiguously by the disclosure.

The disclosed reduction shall define behavior for every actual group length that can occur for that entry. For unscaled instructions, the actual group length is always `G`. For microscaled instructions, the actual group length may be any value `g_len` such that $1 \leq g_len \leq G$, because groups are shortened at microscaling block boundaries as described in [Section 36.7.3.2.2](#).

When composed with the shared SAIL pseudocode of this specification, the published fragment shall reproduce, bit-exactly, both the result that the implementation computes and the floating-point exception flags that the implementation accrues for every legal input. The published fragment shall identify the version of this specification against which it was authored.

A SAIL fragment in this form constitutes the machine-executable disclosure of the reduction algorithm. Natural-language text and parameter tables may describe the algorithm, but they do not by themselves constitute a complete architectural disclosure for a `psm=1` entry.

The flexibility that `psm=1` grants — choosing the reduction algorithm and its parameters — means that this specification alone does not fully determine the numerical result of a `psm=1` implementation.



The ISA extension defines the shared instruction semantics and the required points of implementation disclosure. The implementation-specific SAIL fragment completes the architectural description of the implementation's chosen `psm=1` behaviour.

The conformance and certification programmes are the venue in which complete machine-executable disclosure is checked. A certification programme may therefore require a SAIL fragment for each disclosed `psm=1` entry and may require that the fragment reproduce the hardware bit-exactly when composed with the shared SAIL of this specification.

The SAIL fragment serves three purposes that natural-language disclosure cannot:



1. **Unambiguous semantics.** A SAIL definition fixes operand widths, alignment, rounding, ordering, and parameter values without the interpretation overhead of prose.
2. **Direct integration with conformance testing.** The architectural model in this specification is itself SAIL-based; a published implementation fragment plugs into the same model with no glue code, so an architectural test suite can exercise the implementor's own definition of `fp_disclosed_reduce_group` to verify match against hardware.
3. **Reproducible cross-implementation comparison.** Two implementors disclosing nominally the same algorithm can settle disputes by diffing their SAIL fragments rather than re-deriving equivalence from prose.

An implementation that discloses the RISC-V Bulk Normalization Algorithm (RVBNA),

*introduced below, at the standard parameters may satisfy the SAIL-fragment portion of its complete architectural disclosure by referencing the SAIL fragment published alongside the RVBNA specification itself, provided that fragment composes with the shared SAIL of this specification and reproduces the implementation's result bits and accrued **fflags** bit-exactly.*

The floating-point result and accrued exception flags of a matrix multiply-accumulate instruction are fully architecturally determined once the implementation's supported mapping from **(SEW, W, λ)** to **(G, psm, rnd)** is known and, for every entry with **psm=1**, the implementation's complete architectural disclosure for the reduction algorithm is available.

For unscaled instructions, the number of group updates per output element of C is $(\lambda \times \text{LMUL}) \div \text{G}$. Each group update includes one final accumulation rounded according to **frm**. If **rnd=frm** or **rnd=rto**, one additional rounding of the partial sum S occurs before that final accumulation.

For microscaled instructions, the number of group updates is the number of actual groups produced by the microscaling-block and **LMUL=1**-step intersections described in [Section 36.7.3.2.2](#). It need not equal $(\lambda \times \text{LMUL}) \div \text{G}$. In addition, when **rnd=frm** or **rnd=rto**, scaling the rounded partial sum with the accumulator-format floating-point multiply is a separate rounding point.

*A common choice of **psm=1** reduction is the RISC-V Bulk Normalization Algorithm (RVBNA), defined as part of the Zvdota extension. An implementation adopting RVBNA discloses it as the **psm=1** algorithm and publishes the standard RVBNA parameters (per-factor significand and biased-exponent widths **p** / **e**, result significand and biased-exponent widths **q** / **f**, and the per-group accumulation count **n**). For Zvfmfm instructions, the natural binding for a group of actual length **g_len** is $n = \text{g_len} \times \text{W}$, with **q/f** determined by the output format and **p/e** by the input format(s). Thus, unscaled and other full-length groups use $n = \text{G} \times \text{W}$, while a shortened microscaled group uses the corresponding smaller value $n = \text{g_len} \times \text{W}$.*



*The standard RVBNA description includes a terminal round-to-odd step that produces a value in the result format. When an implementation discloses RVBNA as its **psm=1** algorithm, that terminal round-to-odd is taken only if it also discloses **rnd=rto** for the same **(SEW, W, λ)**; otherwise the rounding implied by the disclosed **rnd** value applies in its place. Most implementations that adopt RVBNA are therefore expected to also disclose **rnd=rto**.*

*Implementations are free to declare any other **psm=1** reduction (for example, a Kulisch-style fixed-point accumulator, or a different bulk-normalised scheme with different parameter choices) provided they publish the algorithm and its parameters as required by the disclosure rule above.*

*A Zvfm implementation can produce the exact same results as the analogous Zvfm implementation when operating on 16- and 8-bit inputs by disclosing **psm=1** with RVBNA at the standard parameters described above, **rnd=rto**, and a suitable architectural value of **G**: **G=2** for 16-bit inputs with 32-bit outputs, and **G=1** for 8-bit inputs with 32-bit outputs. For input element widths of 32 bits or greater, compatibility with the analogous Zvfm instruction is obtained with **G=1**, **psm=0**, and **rnd=frm** so that each group contains a single product and the partial sum S is that product rounded according to **frm**.*



*In a **systolic-array** datapath, **G** is typically 1: each sub-dot-product is rounded and added to C immediately, yielding $\lambda \times \text{LMUL}$ rounding additions per output element.*

*In an **outer-product** datapath, **G** is typically on the order of λ (e.g., λ , $\lambda \div 2$, $\lambda \div 4$, or $\lambda \div 8$): multiple sub-dot-products are accumulated at extended internal precision before a single rounding and C addition. This significantly reduces the number of expensive full-precision additions.*

Portable software must not assume a particular mapping from (SEW, W, Lambda) to (G, psm, rnd) unless that mapping has been disclosed by the implementation.

The rounding of partial sum S is optional in order to match established practices in industry and other RISC-V matrix extensions.

Two conforming implementations may produce floating-point results that differ for identical inputs. Bit-exact reproducibility of floating-point matrix multiply-accumulate results and accrued exception flags across different implementations is therefore guaranteed only when both implementations support the same (SEW, W, Lambda) to (G, psm, rnd) mapping and, at any psm=1 entry, also disclose the same reduction algorithm with the same parameters and the same exception-flag behavior.

36.7.2.3. Floating-point special values and exception flags

For floating-point matrix multiply-accumulate instructions, the architectural result of an instruction consists of both:

- the bit pattern written to each active C accumulator element; and
- the accrued floating-point exception flags written to **fflags**.

For a given implementation disclosure, including the disclosed (SEW, W, Lambda) to (G, psm, rnd) mapping and any complete psm=1 architectural disclosure, the final destination register contents and the final accrued **fflags** value are architecturally determined.

The temporal order in which individual floating-point exception conditions arise during one matrix multiply-accumulate instruction is implementation-defined. However, the final accrued **fflags** value shall be the bitwise OR of all floating-point exception flags raised by the helper operations in the specified or disclosed evaluation of all active output elements.

C tile tail elements, inactive columns, and any element positions not processed by the instruction do not contribute floating-point exception flags.

When microscaling forces an output element to the default NaN because a combined block scale (the product **scale_A** × **scale_B**) is NaN, that element's evaluation terminates at the block that produced the NaN scale. The flags raised while decoding and multiplying the scales of that block, and all flags raised by earlier blocks of the same element (their scales, products, and accumulations), are accrued as usual. The products and accumulation of the NaN-producing block and every later block of that element are not evaluated and contribute no flags. The bitwise-OR rule above therefore applies to the evaluation actually performed for each active output element, which for a forced-NaN element ends at the NaN-producing block.

The helper operations in the SAIL-like pseudocode define where floating-point exception flags may be raised:

- **fp_mul_exact** forms the exact internal product of two FP operands. For finite operands, including subnormal operands and signed zeros, no rounding to the C accumulator format is performed at this point. Special values are handled according to the rules of the operand formats. For example, signaling NaN operands and invalid operations such as zero multiplied by infinity raise the invalid-operation flag and produce an internal NaN value. The payload of an internal NaN value is not architecturally significant.
- **fp_internal_add** forms the exact internal sum of two internal FP values. For finite operands, no rounding to the C accumulator format is performed at this point. Special values are handled according to the rules of the participating FP values. For example, adding infinities of opposite signs raises the invalid-operation flag and produces an internal NaN value. The payload of an internal NaN value is not architecturally significant.

- **fp_round_to_frm** rounds an internal FP value to the C accumulator format using the dynamic rounding mode **frm**. This operation may raise inexact, overflow, underflow, or other exception flags defined for conversion to the C accumulator format.
- **fp_round_to_rto** rounds an internal FP value to the C accumulator format using round-to-odd. This operation may raise the same exception flags that would be raised by converting the internal value to the C accumulator format under the specified round-to-odd rule.
- **fp_add_internal** adds an internal FP value directly to a C accumulator element and rounds the final result according to **frm**. This operation may raise the floating-point exception flags associated with the final addition and rounding.
- **fp_add** and **fp_mul** operate on values already represented in the C accumulator format and follow the floating-point rules for that format, including special-value handling and exception-flag behavior.
- **decode_scale** decodes a scale-format value into the C accumulator format. If that decoding requires conversion to the C accumulator format, the conversion may raise the floating-point exception flags defined by the scale format and the C accumulator format.

NaN payloads are not architecturally significant for IME floating-point operations. An internal value represented by **fp_internal** may use any internal NaN representation, but whenever a shared floating-point helper materializes a NaN result in a concrete floating-point format, it shall return the default canonical NaN for that format, as defined by **fp_defaultNaN**.

This rule applies both to NaNs propagated from input operands and to NaNs created by invalid operations, scale decoding, scale arithmetic, partial-sum rounding, or final accumulation. Canonicalization of the NaN result does not change the exception-flag conditions specified above.

For **psm=0**, the shared SAIL-like pseudocode and the helper operations above define both the result bits and the accrued exception flags.

For **psm=1**, the implementation-specific SAIL fragment that completes the architectural disclosure shall define both the result value produced by the non-exact reduction and any floating-point exception flags raised by that reduction. When composed with the shared SAIL-like pseudocode of this specification, the implementation-specific fragment shall reproduce both the hardware result bits and the hardware accrued **fflags** value for every legal input.



*This rule follows the same architectural principle as ordinary vector floating-point instructions: floating-point exceptions from active computations are accrued into **fflags**, while inactive element positions do not contribute exception flags. IME matrix multiply-accumulate instructions define their active computations in terms of active C tile elements rather than ordinary one-dimensional vector element positions.*

36.7.3. Microscaling support (**v0.scale**)

Microscaling floating-point formats (as defined by the *OCF Microscaling Formats (MX) Specification* ^[1]) represent tensors using narrow per-element values (e.g., OFP4 or OFP8) combined with a coarser shared scale factor applied to a *block* of consecutive elements along the K dimension. The standard OCF MX block size is 32 elements; however, recent research (Lee et al., "MX+: A Robust Framework for Finer-Grained MX Quantization", MICRO '25) demonstrates that a block size of 16 significantly improves accuracy for activation tensors with outliers.

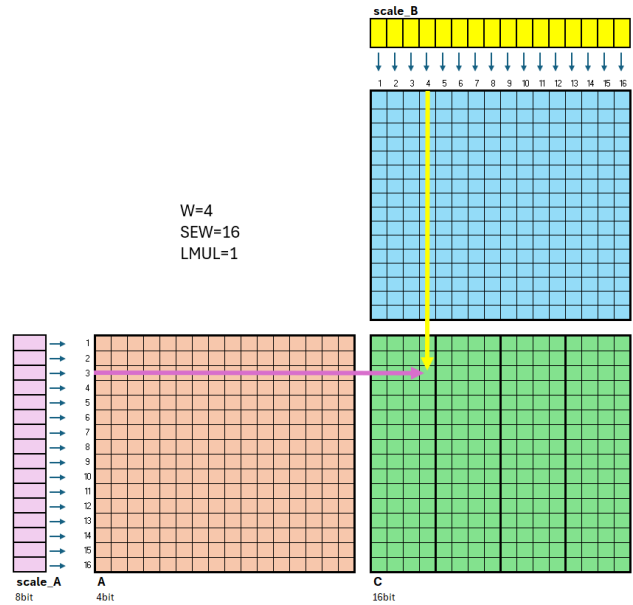


Figure 11. Block scaling multiplication example with $SEW=16$, $\lambda=4$, $W=4$, $LMUL=1$. Block scales are applied along rows of A_{tile} and columns of B_{tile}^T .

The Integrated Matrix extensions support microscaling on widening floating-point multiply-accumulate instructions by encoding $vm=0$ in the instruction and supplying paired block-scale factors through $v0$. The scale format is not implied by $v0.scale$ itself; it is an inherent property of the input data type (e.g., MXFP8 = OFP8 with E8MO scales, MXINT8 = Int8 with E8MO scales). The **bs** field in **vtype** selects the block size (32 or 16 elements).

The assembly syntax for a microscaled operation appends $v0.scale$ as the final operand, analogous to $v0.t$ for masked operations in the base V extension:

```
vfwmacc.vv vd, vs1, vs2, v0.scale
vfqmmacc.vv vd, vs1, vs2, v0.scale
vf8wmacc.vv vd, vs1, vs2, v0.scale
```

When $vm=0$ is not specified (i.e., $vm=1$, the default), no scaling is applied, $v0$ is not read, and the **bs** field is ignored.

36.7.3.1. Semantics

When microscaling is active ($vm=0$), each output element is computed as:

$$C_{i,j} \leftarrow C_{i,j} + \sum_{s=0}^{L-1} \left(\text{scale_A}_{i,s} \times \text{scale_B}_{j,s} \times \left(\sum_{k \in \text{block}(s)} A_{\text{tile}}[i,k] \times B_{\text{tile}}^T[k,j] \right) \right)$$

where $L = \lceil K_{\text{eff}} / \text{block_size} \rceil$ is the number of scale blocks per row/column, and $\text{block}(s)$ covers elements $[s \times \text{block_size}, \min((s+1) \times \text{block_size}, K_{\text{eff}}) - 1]$.

$\text{scale_A}_{i,s}$ and $\text{scale_B}_{j,s}$ are decoded from the per-block scale fields in $v0$ according to the scale format associated with the input data type. If the combined block scale — the product $\text{scale_A}_{i,s} \times \text{scale_B}_{j,s}$ formed in the C accumulator format — is NaN, the corresponding output element $C_{i,j}$ is set to the default NaN regardless of the input element values, and the evaluation of that output element terminates: the scale blocks after the one that produced the NaN scale are not evaluated. For the E8MO scale format this combined-scale NaN occurs exactly when either decoded scale is NaN (the encoding 0xFF); for scale

formats that admit zero and infinity it can also arise from $0 \times \infty$. The exception-flag consequences of this early termination are specified in [Section 36.7.2.3](#).

36.7.3.2. Scale formats

The scale format is determined by the input data type, not by the `v0.scale` mechanism itself. All MX data types currently defined by this specification use the E8MO scale format.

36.7.3.2.1. E8MO

The E8MO format (as defined by the *OCP Microscaling Formats (MX) v1.0 Specification*) is an 8-bit exponent-only format with a bias of 127, representing power-of-two values from 2^{-127} to 2^{127} . The bit pattern 0xFF encodes NaN.

Non-NaN E8MO scale values denote exact powers of two. Applying such a scale to an unbounded or internal floating-point value can be modeled as an exact exponent adjustment before the next required rounding point.

However, the conversion of a scale value to the C accumulator format, formation of the paired block scale, and scaling of a partial sum follow the floating-point operations specified by this section and by the SAIL-like pseudocode. These operations may overflow, underflow, produce NaN, be inexact, or set floating-point exception flags as specified. Therefore, the exactness of the E8MO scale encoding does not imply that every scaled intermediate or final result is exactly representable in the C accumulator format.

When conversion of a non-NaN E8MO scale value to the accumulator FP format overflows, the converted scale value is $+\infty$. For example, the E8MO scale value 2^{127} cannot be represented as a finite IEEE binary16 value.

The subsequent block-scale operations follow IEEE 754 rules: $+\infty$ multiplied by a non-zero finite value yields $\pm\infty$, propagating into the accumulator via `fp_add`; $+\infty$ multiplied by zero yields the default NaN, which sets the output element to NaN via the NaN propagation rule above. Implementations shall handle this case as specified, not as an implementation-defined result.

36.7.3.2.2. Arithmetic considerations and shortened groups

The computation of $C_{i,j}$ follows the grouping, partial-sum, rounding, scaling, and final-accumulation rules of [Section 36.7.2.2](#), with the additional constraint that a group shall not cross a microscaling block boundary.

Microscaling block boundaries are defined over scalar elements along the K axis. For block index `s`, the scalar K interval covered by the block is:

```
blk_k_lo = s * block_size
blk_k_hi = min(blk_k_lo + block_size, K_eff) - 1
```

The computation is also defined as repeated `LMUL=1` steps, in increasing register-index order. For `LMUL=1` step `step`, the scalar K interval is:

```
step_k_lo = step *  $\lambda$  * W
step_k_hi = step_k_lo +  $\lambda$  * W - 1
```

For each output element, the instruction processes the intersection of the current microscaling block and the current **LMUL=1** step:

```
int_k_lo = max(step_k_lo, blk_k_lo)
int_k_hi = min(step_k_hi, blk_k_hi)
```

If $\text{int_k_lo} > \text{int_k_hi}$, that block/step pair contributes no products.

Otherwise, the intersected scalar K interval is converted to sub-dot-product indices relative to the **LMUL=1** step:

```
subdot_lo = (int_k_lo - step_k_lo) ÷ W
subdot_hi = (int_k_hi - step_k_lo) ÷ W
```

The sub-dot-products in this interval are partitioned into consecutive groups of up to **G** sub-dot-products. For a group beginning at sub-dot-product index **g0**, the actual group length is:

```
g_len = min(G, subdot_hi - g0 + 1)
```

Thus:

- $1 \leq \text{g_len} \leq \text{G}$;
- groups do not cross **LMUL=1** step boundaries;
- groups do not cross microscaling block boundaries; and
- the final group in an intersected block/step interval may contain fewer than **G** sub-dot-products.

A group of actual length **g_len** contains $\text{g_len} \times \text{W}$ scalar products. The group dot-product is reduced into a partial sum **S** according to **psm**. The partial sum **S** is treated according to **rnd**.

For **psm=0**, the exact-reduction rule applies to the actual group length **g_len**.

For **psm=1**, the implementation-specific architectural disclosure shall define the reduction result and exception-flag behavior for every actual group length **g_len** that can occur for the disclosed (**SEW**, **W**, **lambda**) entry. In particular, the disclosed SAIL fragment shall not assume that every microscaled group has length exactly **G**.

The partial sum **S** is then scaled by $\text{scale_A}_{i,s} \times \text{scale_B}_{j,s}$, using the scale value associated with microscaling block **s**. Scaling, rounding, final accumulation into **C**, and accrued exception flags follow [Section 36.7.2.3](#) and the operation pseudocode.

If **rnd=frm** or **rnd=rto**, **S** has first been rounded to the **C** accumulator format, and scaling is performed by the accumulator-format floating-point multiply specified by the operation pseudocode. This multiplication may overflow, underflow, be inexact, produce NaN, or set floating-point exception flags.

If **rnd=xct**, scaling is applied to the internal partial sum before final accumulation. The scale factor itself is exact, but the final accumulation into **C** is rounded according to **frm** and may raise floating-point exception flags.

Finally, the scaled partial sum **S** is accumulated into the running value of $C_{i,j}$ by computing

$$C_{i,j} \leftarrow \text{round}_{\text{frm}}(C_{i,j} + S).$$



For unscaled floating-point multiply-accumulate instructions, the only group boundary inside an **LMUL=1** step is the grouping factor **G**, and groups always have length **G**.

For microscaled instructions, scale-block boundaries are additional group boundaries. These boundaries may cut across the **LMUL=1** step structure, so the final group in an intersected block/step interval may be shorter than **G**.

36.7.3.3. Scale layout in **v0**

The register **v0** holds paired scale values at an effective element width of 16 bits. Each 16-bit element contains one (scale_A, scale_B) pair, regardless of the scale format:

- Lower byte [7:0]: **scale_A** — the block scale for a row of A_{tile} .
- Upper byte [15:8]: **scale_B** — the block scale for a column of B_{tile}^T .

The interpretation of each 8-bit field is determined by the scale format associated with the input data type (see [Section 36.7.3.2](#)).

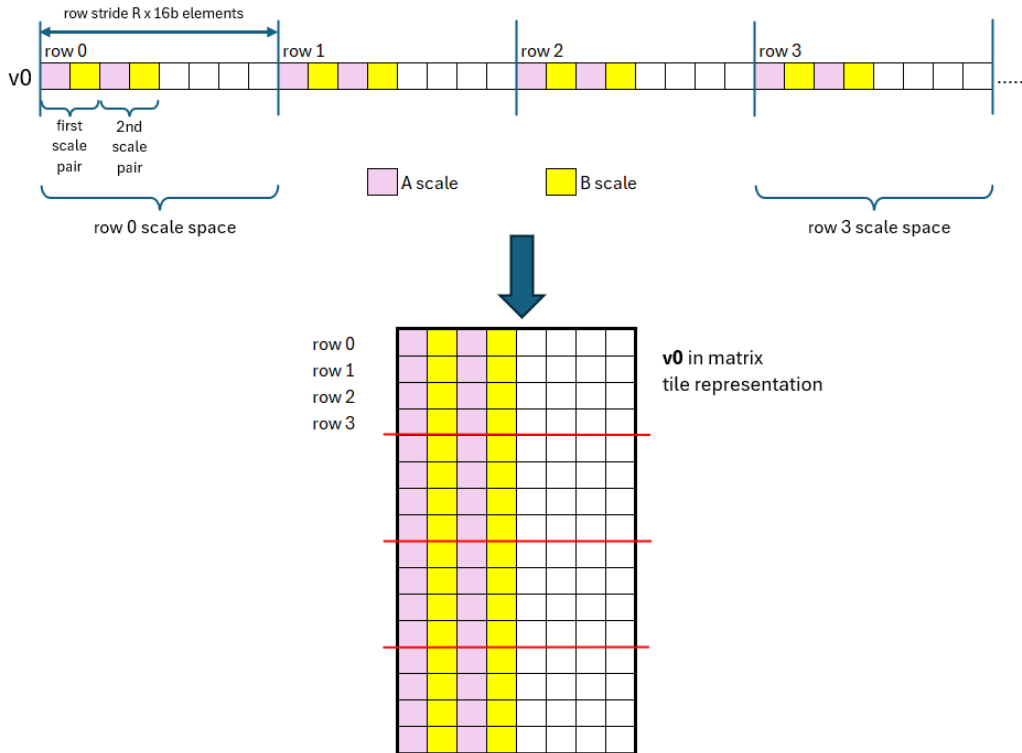


Figure 12. Block scales are folded into **v0** in pairs, separated by the row stride $R = \lambda \times \text{SEW} / \text{pw}$. Each block-scale pair is used for **bs** consecutive tile elements in the corresponding row of A_{tile} and column of B_{tile}^T . In matrix-tile representation, **v0** has the paired scales arranged in the correct scale rows. In a multi-lane setup, the scales are distributed across the lanes appropriately.

Let $\text{sw} = \text{scale_width_of}(\text{scale_fmt})$ be the width in bits of a single scale value and $\text{pw} = 2 \times \text{sw}$ the width of a (scale_A, scale_B) pair. For E8M0: $\text{sw} = 8$, $\text{pw} = 16$.

The row stride in **v0** is $R = \lambda \times \text{SEW} \div \text{pw}$ in units of pw -bit elements (see [Figure 12](#)).

For each tile index m , $S = \lceil K_{\text{eff}} / \text{block_size} \rceil$ elements are active. Element index $p = m \times R + s$ addresses the scale pair for block s .

For **scale_A**, index m identifies row m of A_{tile} . For **scale_B**, index m identifies column m of B_{tile}^T . Specifically:

- **scale_A** $_{m,s}$ is the lower sw bits of the pw -bit element at index $(m \times R + s)$.
- **scale_B** $_{m,s}$ is the upper sw bits of the pw -bit element at index $(m \times R + s)$.

The total number of pw -bit elements spanned per tile dimension is $M \times R$, where $M = N = \text{VLEN} \div (\text{SEW} \times \lambda)$. Because $M \times R = (\text{VLEN} \div (\text{SEW} \times \lambda)) \times (\lambda \times \text{SEW} \div pw) = \text{VLEN} \div pw$, the scale data fills exactly one register regardless of configuration.

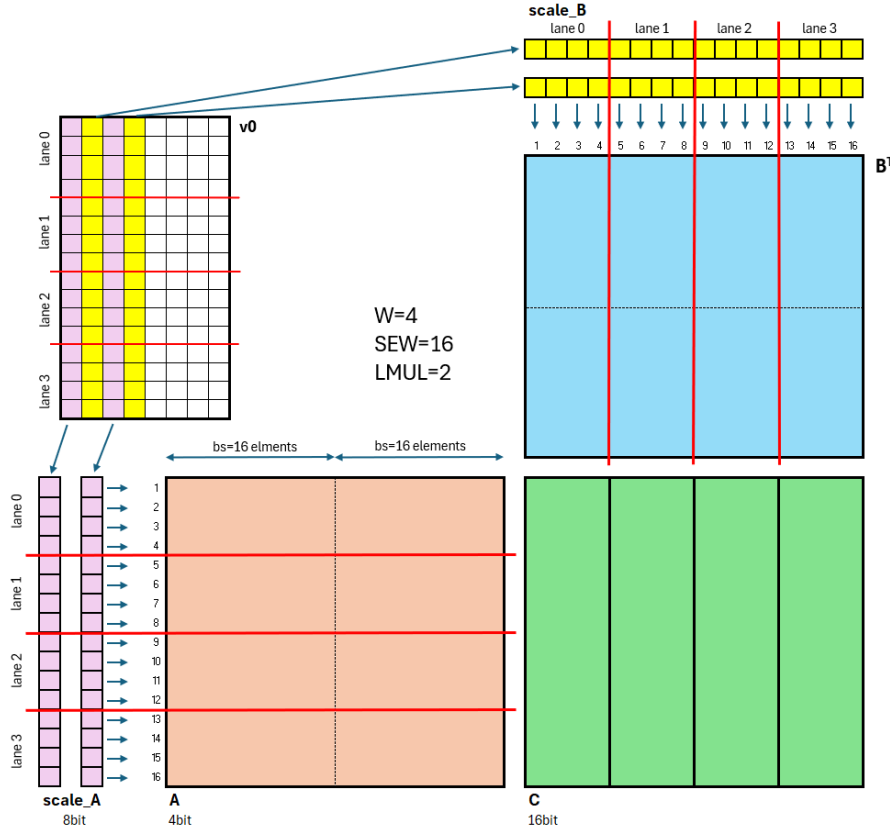


Figure 13. Example for the use of microscales in a 4-lane setup for two blocks per row of A_{tile} and per column of B_{tile}^T . The red lines symbolize the split of the vector register group across the lanes. The R -strided scales distribution in **v0** leads to the correct lane-wise block scales layout.

Capacity proof:

$$M \times R = (\text{VLEN} \div (\text{SEW} \times \lambda)) \times (\lambda \times \text{SEW} \div pw) = \text{VLEN} \div pw.$$

Each of the $\text{VLEN} \div pw$ positions is a pw -bit element, so the total is exactly VLEN bits — the scale data fills one register in every legal configuration.

$R \geq S$ proof (tile stride is never smaller than the active count):

- **block_size = 32:** The encoding tables (Table 87, Section 36.13.3) mark all combinations with $W \times \text{LMUL} > 2 \times \text{SEW}$ as *reserved*, so every legal encoding satisfies $W \times \text{LMUL} \leq 2 \times \text{SEW}$. Therefore $K_{\text{eff}} = \lambda \times W \times \text{LMUL} \leq 2 \times \lambda \times \text{SEW}$, giving $S = \lceil K_{\text{eff}} / 32 \rceil \leq \lceil 2 \times \lambda \times \text{SEW} / 32 \rceil \leq \lambda \times \text{SEW} / pw = R$ (for $pw \leq 16$).
- **block_size = 16:** The $\text{BS}=16$ legality check ($W \times \text{LMUL} \leq \text{SEW}$, stated in each instruction's Exceptions section) ensures $K_{\text{eff}} = \lambda \times W \times \text{LMUL} \leq \lambda \times \text{SEW}$, giving $S = \lceil K_{\text{eff}} / 16 \rceil \leq \lceil \lambda \times \text{SEW} / 16 \rceil \leq \lambda \times \text{SEW} / pw = R$ (for $pw \leq 16$).

When $S < R$, positions $s \in [S, R)$ within each row are padding and their contents are ignored by the

instruction.

Table 81. Scale layout parameters ($block_size = 32$, $VLEN = 256$)

SEW	λ	W	LMUL	M	S	R	16-bit extent ($M \times R$)
8	2	1	1	16	1	1	16
8	2	1	8	16	1	1	16
8	2	2	1	16	1	1	16
8	2	2	8	16	1	1	16
16	1	1	1	16	1	1	16
16	1	1	2	16	1	1	16
16	1	2	1	16	1	1	16
16	1	4	1	16	1	1	16
16	1	4	8	16	1	1	$16^{[2]}$
16	2	4	1	8	1	2	16
16	2	4	8	8	2	2	$16^{[2]}$
32	1	1	1	8	1	2	16
32	1	2	1	8	1	2	16
32	2	1	1	4	1	4	16
64	1	1	1	4	1	4	16

Within each row stride, elements at positions $s \in [S, R)$ are padding and need not be initialised. When $S = R$, the layout is fully packed with no padding.

36.7.3.4. Block size selection

The **bs** field in **vtype** (see [Section 36.4.5](#)) selects the microscaling block size. When **bs=0**, the block size is 32 elements (standard OCP MX); when **bs=1**, the block size is 16 elements.

Block size 16 doubles the number of scales per row/column (S is doubled), which doubles the $M \times S$ capacity requirement. To guarantee that the paired scales fit within **v0**, the following constraint is imposed when **bs=1** and **vm=0**:

$$W \times LMUL \leq SEW$$

If this constraint is violated, the instruction raises an illegal-instruction exception.

Table 82. Maximum LMUL for $block_size = 16$

Input format	Instruction	W	SEW	Max LMUL
FP4 $\rightarrow$ FP16/BF16	vfqmmacc	4	16	4
FP4 $\rightarrow$ OFP8	vfwmacc	2	8	4
Int4 $\rightarrow$ FP16/BF16	vfqimmacc	4	16	4
All others	—	—	—	8 (no restriction)

36.7.3.5. Applicability

Microscaling via `v0.scale` is supported on the widening floating-point multiply-accumulate instructions (`vfwmacc.vv`, `vfqmmacc.vv`, `vf8wmacc.vv`) and on the integer-input FP-accumulate instructions `vwimmacc.vv` (MXINT8→FP16/BF16), `vfqimmacc.vv` (MXINT4→FP16/BF16, MXINT8→FP32), and `vf8wimmacc.vv` (MXINT4→FP32, MXINT8→FP64). For `vmmacc.vv`, `vm=0` remains reserved and raises an illegal-instruction exception.

MXINT8 (E8MO-scaled signed 8-bit integer) is defined by the *OCP Microscaling Formats (MX) v1.0 Specification*. MXINT4 (E8MO-scaled signed 4-bit integer) is defined analogously by this specification: each block of 4-bit signed two's-complement elements shares a single E8MO exponent-only scale factor, following the same block-scaling semantics as MXINT8 but with 4-bit elements.

Microscaling requires $SEW \times \lambda \geq pw$ (the scale-pair width: $2 \times sw$) to ensure that the paired scale elements fit within `v0`. For E8MO ($sw = 8$, $pw = 16$) this reduces to $SEW \times \lambda \geq 16$. Configurations where $SEW \times \lambda < pw$ with `vm=0` raise an illegal-instruction exception.

Microscaling is particularly useful with narrow input formats (OFP4, OFP8) where the limited dynamic range of the per-element format is compensated by the block-level E8MO exponent, enabling high-throughput inference on microscaling-quantised models.

36.7.3.6. Loading or constructing paired scales

This section is informative (non-normative).

The tile-strided scale layout in `v0` uses $M \times R$ 16-bit elements (always $VLEN \div 16$ elements, filling exactly one register). Within each of the M row strides of length R , the first S positions are active and positions $[S, R)$ are padding. The following options are listed from simplest to most complex.

Option 1 — Pre-paired tile-strided load via `vmtl.v` (single instruction):

When the application stores pre-paired (scale_A, scale_B) 16-bit elements in a 2D scale matrix (M rows, stride $LD \geq R$ between rows), a single `vmtl.v` loads the current tile's scales directly into `v0`:



```
# a0 = base of pre-paired scale sub-array for current tile
# a2 = M × R (= VLEN / 16)
# a3 = LD (leading dimension in 16-bit elements, LD ≥ R)
vsetvli t0, a2, e16, m1, ta, ma
vmtl.v  v0, (a0), a3          # tile-load pre-paired scales
```

With $EEW = 16$ and $LMUL = 1$, $linesize = \lambda = R$ (when $SEW = 16$). Each `vmtl.v` line corresponds to one scale row, and LD allows rows to be non-contiguous in the outer scale matrix. When $LD = R$, this reduces to a unit-stride load.

This form requires $SEW = 16$ (i.e., $R = \lambda$). For $SEW > 16$, $R > \lambda$ and a single `vmtl.v` cannot span a full scale row; use Option 2 instead.

Option 2 — Pre-paired flat load (`vle16.v`):

If the pre-paired tile-strided data is already contiguous in memory ($M \times R$ 16-bit elements), a unit-stride load suffices:

```

# a0 = pointer to contiguous pre-paired tile-strided scale
array
# a2 = M × R (= VLEN / 16)
vsetvli t0, a2, e16, m1, ta, ma
vle16.v v0, (a0)

```

This works for all SEW values since $M \times R = \text{VLEN} \div \text{pw}$ always (where pw is the scale-pair width).

Options 3–5 — Constructing from separate **scale_A** / **scale_B** arrays:

The following sequences construct the paired layout from separate **scale_A** and **scale_B** byte arrays, producing $M \times S$ contiguous pairs. This produces the correct tile-strided layout when $S = R$ (fully packed). When $S < R$, the output must be placed at stride R, e.g., by storing the $M \times S$ pairs into a pre-zeroed $M \times R$ buffer before loading with Option 2.

Option 3 — Base V (widen + shift + OR):

```

# a0 = &scale_A, a1 = &scale_B (M × S bytes each), a2 = M × S
vsetvli t0, a2, e8, mf2, ta, ma
vle8.v v1, (a0) # load scale_A
vle8.v v2, (a1) # load scale_B
vsetvli t0, a2, e16, m1, ta, ma
vzext.vf2 v0, v1 # zero-extend scale_A to 16-bit
vzext.vf2 v3, v2 # zero-extend scale_B to 16-bit
vsll.vi v3, v3, 8 # shift scale_B into upper byte
vor.vv v0, v0, v3 # combine into paired layout

```

Option 4 — Zvbb (widening shift + OR):

```

# a0 = &scale_A, a1 = &scale_B (M × S bytes each), a2 = M × S
vsetvli t0, a2, e8, mf2, ta, ma
vle8.v v1, (a0) # load scale_A
vle8.v v2, (a1) # load scale_B
vwsll.vi v0, v2, 8 # widen scale_B to 16-bit,
shift to upper byte
vsetvli t0, a2, e16, m1, ta, ma
vzext.vf2 v3, v1 # zero-extend scale_A to 16-bit
vor.vv v0, v0, v3 # combine into paired layout

```

Option 5 — Zvzip (interleave):

```

# a0 = &scale_A, a1 = &scale_B (M × S bytes each), a2 = M × S
vsetvli t0, a2, e8, mf2, ta, ma
vle8.v v1, (a0) # load scale_A
vle8.v v2, (a1) # load scale_B
vzip.vv v0, v1, v2 # interleave bytes, EMUL = 2 ×

```

```
mf2 = m1
```

The `vzip.vv` instruction interleaves at $SEW=8$: the byte sequence `[A[0], B[0], A[1], B[1], ...]` reinterpreted as 16-bit little-endian elements yields `element[i] = B[i]<<8 | A[i]`, which is exactly the required paired layout.

Option 6 — Separate scales via `vmtl.v` + `vzip.vv` ($SEW = 16$):

When $SEW = 16$ and scales reside in separate 2D arrays, `vmtl.v` can load each scale array with the correct tile stride, then `vzip.vv` pairs them:

```
# a0 = &scale_A, a1 = &scale_B (M rows, LD bytes between
rows)
# a2 = M × R (= VLEN / 16), a3 = LD (leading dimension in
elements)
vsetvli t0, a2, e8, m1, ta, ma
vmtl.v v1, (a0), a3          # tile-load scale_A
vmtl.v v2, (a1), a3          # tile-load scale_B
vsetvli t0, a2, e8, mf2, ta, ma
vzip.vv v0, v1, v2           # pair into 16-bit layout
```

Table 83. Scale-packing instruction count comparison

Option	Data instructions	<code>vsetvli</code> calls	Total
1 (<code>vmtl.v</code> pre-paired)	1	1	2
2 (<code>vle16.v</code> pre-paired)	1	1	2
3 (Base V)	5	2	7
4 (<code>Zvbb</code>)	4	2	6
5 (<code>Zvzip</code>)	3	1	4
6 (<code>vmtl.v</code> + <code>vzip.vv</code>)	3	2	5

36.7.3.7. Microscaling extensions

Microscaling extensions that add block-scaled variants of the narrow input formats (OFP4, OFP8, Int4, Int8) are listed separately in [Table 86](#). All currently defined microscaling extensions use the E8M0 scale format.

36.8. Zvvmtls: Extension for loading and storing vector register groups interpreted as 2D matrix tiles

The `Zvvmtls` extension provides instructions that transfer 2D matrix tiles between memory and the vector register file without requiring a separate repacking step. Matrix data in memory is typically stored as a large 2D array in row-major or column-major order with a leading dimension that may exceed the tile width. This allows a tile to address a sub-matrix within a larger allocation. The tile load and store instructions bridge the gap between this strided 2D memory layout and the flat vector register representation used by the multiply-accumulate instructions.

36.8.1. Memory layout

Tile instructions interpret memory as a 2D array of SEW-bit elements. **rs1** holds the base address of the tile; **rs2** holds the *leading dimension* LD, i.e., the stride between the starts of consecutive segments, measured in elements. For a row-major matrix, LD equals the number of columns in the full matrix; for a column-major matrix, LD equals the number of rows. LD may be larger than the tile dimension, enabling access to a sub-tile of a larger matrix without copying.

The byte address of the element at offset **d** along the stride direction and offset **k** within a segment is:

$$\text{address}(d, k) = \text{rs1} + (d \times \text{LD} + k) \times (\text{SEW} \div 8)$$

36.8.2. Tile geometry for loads and stores

The tile dimensions are fully determined by the current **vtype** settings (SEW, VL, LMUL, and λ), consistent with the operand geometry of the multiply-accumulate instructions. The tile load and store instructions make use of the following parameters from the CSRs:

- SEW — element width
- VL — active vector length (must be a multiple of $\lambda \times \text{LMUL}$, the line size)
- LMUL — vector length multiplier
- λ — selected lambda, read from **lambda[2:0]** in **vtype**

Tile load/store instructions operate on SEW-bit *storage elements*. They do not interpret the logical element types packed within a storage element, and they do not use the widening factor **W**.

When an order-preserving tile load or store is used to prepare an A or B input tile for direct use by a widening multiply-accumulate instruction, SEW shall equal the C accumulator element width. Each SEW-bit storage element then contains **W** packed logical input elements of width SEW / W , in the packing order defined by this specification.

The transposing instructions **vmttl.v** and **vmtts.v** transpose SEW-bit storage elements. They do not unpack, transpose, or repack logical elements contained within an SEW-bit storage element.

Consequently, a transposing tile instruction performs a direct logical matrix transpose only when the logical element width equals SEW. For a widening input tile, where the logical element width is less than SEW, transposition of storage elements is not generally equivalent to transposition of the logical input matrix.

Figure 14 illustrates order-preserving and transposing transfers at the storage-element level for LMUL=1. Both forms transfer the same number of SEW-bit storage elements and produce the required register-group ordering when the memory layout matches the selected instruction. However, their memory-address traversal differs: **vmtl.v** accesses consecutive storage elements within each segment, whereas **vmttl.v** accesses storage elements in transposed order using LD.

When the logical element width equals SEW, these transfers may be interpreted directly as row-major and column-major logical-element loads. When logical elements are packed within SEW-bit storage elements, that interpretation applies only to the storage elements, not to the individual packed logical elements.

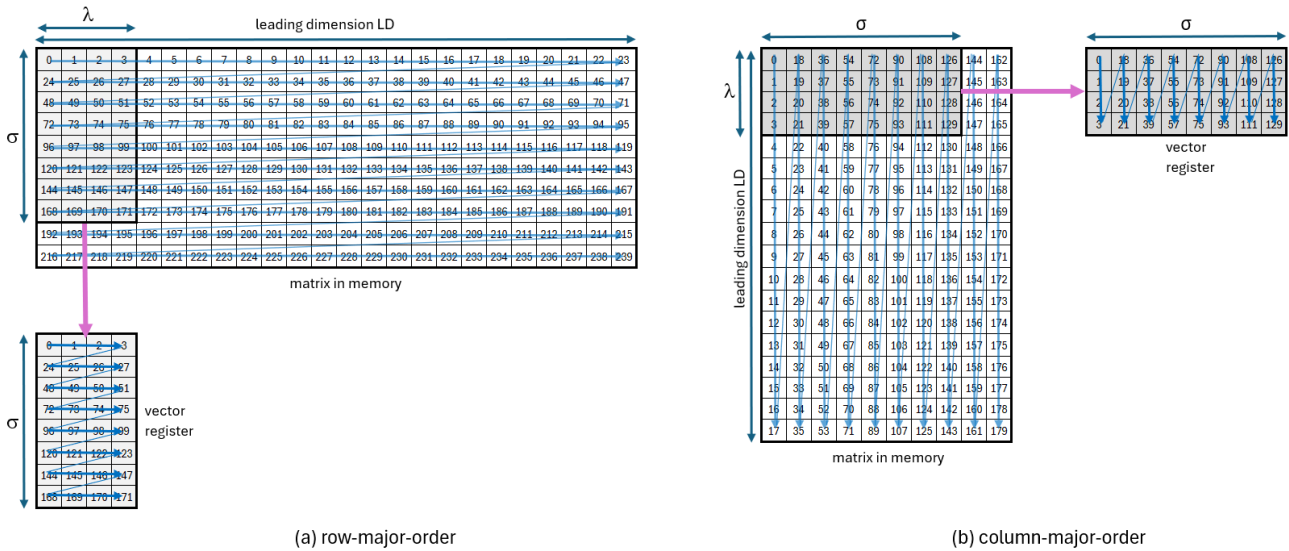


Figure 14. Loading a matrix tile from memory for $LMUL=1$. The matrix is laid out linearly in memory, the leading dimension LD specifies its row size (a) or column size (b). Element indices represent the offset of the elements in memory. Blue arrows indicate the data ordering in memory and in the vector register group.

If $(rs2) = 0$, the leading dimension LD defaults to the *natural dimension* of the transfer, which differs between the order-preserving and the transposing variants:

- For the order-preserving load/store (**vmtl.v**, **vmts.v**), $LD = \lambda \times LMUL$ — the in-memory layout, with elements contiguous to each other, matches the layout of the register group being loaded/stored.
- For the transposing load/store (**vmttl.v**, **vmtts.v**), $LD = VLEN \div (SEW \times \lambda)$ ($= \sigma$, the tile edge) — the leading dimension of a matrix tile that exactly fills one vector register group when transposed.

(Each instruction restates its own default; the two values coincide only when $VLEN \div SEW = \lambda^2 \times LMUL$.)



It is expected that implementations will optimize the case of **rs2** = **x0** to load/store a vector register group with contiguous VL elements in memory.

Efficient implementation of tile loads with **rs2** = **x0** are essential to high-performance linear algebra kernels.

When using a vector length lower than the maximum for the data width, the data is loaded so that it fills complete rows of an A tile or complete columns of B_{tile}^T . Each active column of B_{tile}^T contributes to the corresponding active column of C. The impact is sketched in Figure 15.

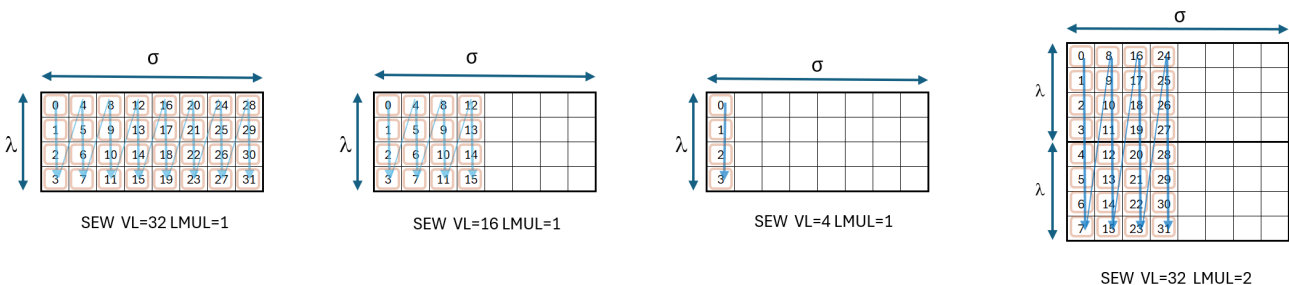


Figure 15. Examples of element distribution for full (left) and reduced vector length matrix tile load / store operations.

36.8.3. Instructions

All tile load/store instructions support two optional operands that follow **rs2** in the assembly syntax:

- An *immediate lambda* $L\lambda$ (one of $L1, L2, L4, L8, L16, L32, L64$), which overrides `lambda[2:0]` from `vtype` for the duration of that instruction. When omitted, the dynamic lambda from `vtype` is used.
- A *mask* specifier `v0.t`, which enables element-level masking using `v0` as the mask register. When omitted, all elements are unmasked.

When both are present, the immediate lambda precedes the mask: `vmtts.v v4, (a0), a1, L64, v0.t`.

Tile load and store instructions follow the standard vector element-status semantics, except that a tile load always leaves inactive and tail destination elements undisturbed: the tail-agnostic (`vtype.vta`) and mask-agnostic (`vtype.vma`) fill policies do not apply to tile loads. This matches the operational pseudocode, which writes only active body elements, and the undisturbed-without-read tile philosophy described in [Section 36.9.3](#).

- *Active* elements (body, mask enabled): loads fetch from memory and write `vd`; stores read `vs3` and write to memory. Active elements may raise memory exceptions.
- *Inactive* elements (body, mask disabled): loads never raise exceptions and leave the destination register undisturbed. Stores do not write to memory and do not raise exceptions.
- *Tail* elements (index $\geq VL$): loads never raise exceptions and leave the destination register undisturbed. Stores do not write to memory and do not raise exceptions.
- *Prestart* elements (index $< vstart$): neither the destination register nor memory is updated; no exceptions are raised.

36.8.3.1. Common legality and restart rules

The following rules apply to all tile load/store instructions:

- `vtype.vlmul` shall encode an integer LMUL value in $\{1, 2, 4, 8\}$. Fractional and reserved LMUL encodings raise an illegal-instruction exception.
- The effective lambda is taken from the nonzero instruction lambda field, when present. Otherwise, it is taken from `vtype.lambda[2:0]`.
- If both the instruction lambda field and `vtype.lambda[2:0]` are zero, no effective lambda is selected and the instruction raises an illegal-instruction exception.
- A nonzero instruction lambda field requests its exact encoded lambda. The immediate value is not WARL-clamped and does not modify `vtype.lambda[2:0]`. If that exact lambda is not supported for the current (`VLEN, SEW`) configuration, the instruction raises an illegal-instruction exception.
- `VL` shall be a multiple of `effective_lambda * LMUL`.
- The tile vector operand shall be aligned to its LMUL-sized register group, and the group shall not extend beyond `v31`.
- For a masked tile load, the destination register group shall not overlap the mask register `v0`.

A nonzero `vstart` value is legal for tile load/store instructions. Execution resumes at element `vstart`. Prestart elements do not access memory or vector-register state. On successful completion, `vstart` is set to zero. If an active element raises a memory exception, `vstart` identifies the faulting element so that the instruction can be restarted.

Mnemonic	Description
<code>vmtl.v vd, (rs1), rs2 [, Lλ] [, vm]</code>	Order-preserving tile load

Mnemonic	Description
vmts.v vs3, (rs1), rs2 [, Lλ] [, vm]	Order-preserving tile store

36.8.3.2. vmtl.v — Order-Preserving Tile Load

```
vmtl.v vd, (rs1), rs2 [, Lλ] [, vm]
```

Loads a 2D matrix tile from memory into the vector register group starting at **vd**. **rs1** holds the matrix base address; **rs2** holds the leading dimension **LD** in elements.

Let $linesize = \lambda \times LMUL$. For each element index **i** in the body [**vstart**, **VL**) where the mask is enabled:

$$VD[i] = M[rs1 + (SEW \div 8) \times ((i / linesize) \times LD + (i \% linesize))]$$

This instruction is the correct choice when A is stored in row-major order or when B is stored in column-major order: in both cases the memory layout consists of *linesize*-element contiguous segments with a stride of **LD** elements between them.

36.8.3.3. vmts.v — Order-Preserving Tile Store

```
vmts.v vs3, (rs1), rs2 [, Lλ] [, vm]
```

Stores the 2D matrix tile held in the vector register group starting at **vs3** to memory. **rs1** holds the matrix base address; **rs2** holds the leading dimension **LD** in elements.

Let $linesize = \lambda \times LMUL$. For each element index **i** in the body [**vstart**, **VL**) where the mask is enabled:

$$M[rs1 + (SEW \div 8) \times ((i / linesize) \times LD + (i \% linesize))] = VS[i]$$


Order preserving tile load/store with $LMUL > 1$ offers optimization opportunities. While vmtl/vmts are very similar to vector constant-stride segment operations, the segment sizes are potentially larger. Matching cache line size with $\lambda \times LMUL \times SEW$ allows for full cacheline transfers.

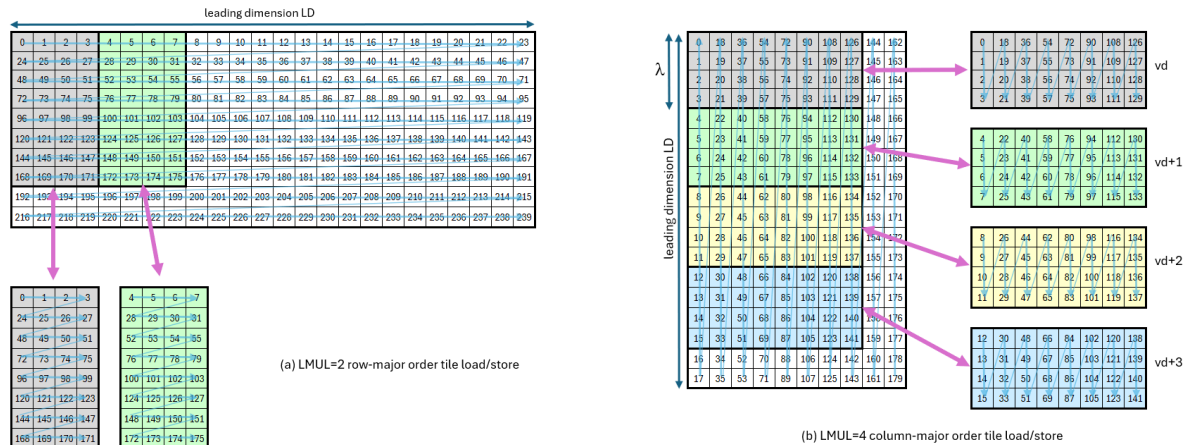


Figure 16. Order preserving tile load/store with LMUL > 1 for row-major (a) and column-major ordering in memory.

36.9. Zvwmmtls: Extension for loading and storing vector register groups interpreted as transposed 2D matrix tiles

The **Zvwmmtls** extension provides instructions that transpose two-dimensional arrays of **SEW**-bit storage elements between memory and the vector register file.

The instructions use the memory-layout, tile-geometry, masking, restart, and legality rules established for **Zvwmmtls**. They transpose storage elements; they do not unpack, transpose, or repack logical elements contained within a storage element.

When each logical matrix element occupies one **SEW**-bit storage element, the storage-element transpose is also a logical-element transpose. When multiple logical elements are packed within each storage element, it is not a transpose of the individual logical elements.

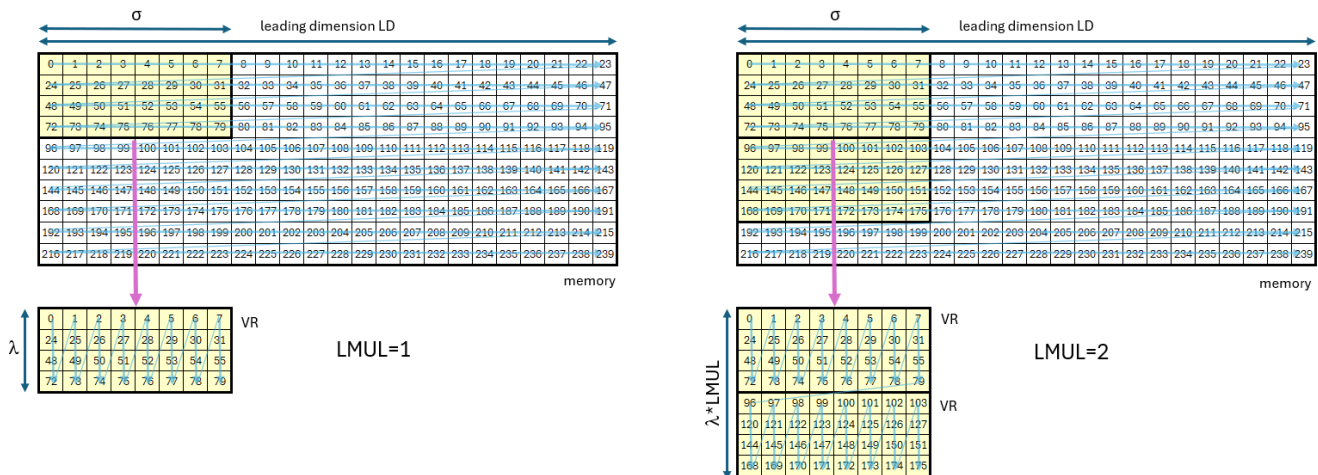


Figure 17. Transposing load examples with LMUL = 1 (left) and LMUL = 2 (right). Indices number elements of width **SEW** from the row-major order matrix in memory. Light-blue arrows indicate the memory / vector register access order.

36.9.1. Instructions

All tile load/store instructions support two optional operands that follow **rs2** in the assembly syntax:

- An immediate **lambda** **Lλ** (one of **L1**, **L2**, **L4**, **L8**, **L16**, **L32**, **L64**), which overrides **lambda[2:0]** from **vtype** for the duration of that instruction. When omitted, the dynamic lambda from **vtype** is used.
- A mask specifier **v0.t**, which enables element-level masking using **v0** as the mask register. When

omitted, all elements are unmasked.

When both are present, the immediate lambda precedes the mask: `vmtts.v v4, (a0), a1, L64, v0.t`.

Tile load and store instructions follow the standard vector element-status semantics, except that a tile load always leaves inactive and tail destination elements undisturbed: the tail-agnostic (`vtype.vta`) and mask-agnostic (`vtype.vma`) fill policies do not apply to tile loads. This matches the operational pseudocode, which writes only active body elements, and the undisturbed-without-read tile philosophy described in [Section 36.9.3](#).

- *Active* elements (body, mask enabled): loads fetch from memory and write `vd`; stores read `vs3` and write to memory. Active elements may raise memory exceptions.
- *Inactive* elements (body, mask disabled): loads never raise exceptions and leave the destination register undisturbed. Stores do not write to memory and do not raise exceptions.
- *Tail* elements (index $\geq$ VL): loads never raise exceptions and leave the destination register undisturbed. Stores do not write to memory and do not raise exceptions.
- *Prestart* elements (index $<$ `vstart`): neither the destination register nor memory is updated; no exceptions are raised.

The common legality and restart rules in [Section 36.8.3.1](#) also apply to `vmttl.v` and `vmtts.v`.

Mnemonic	Description
<code>vmttl.v vd, (rs1), rs2 [, Lλ] [, vm]</code>	Transposing tile load
<code>vmtts.v vs3, (rs1), rs2 [, Lλ] [, vm]</code>	Transposing tile store

36.9.1.1. `vmttl.v` — Transposing Tile Load

```
vmttl.v vd, (rs1), rs2 [, Lλ] [, vm]
```

Loads a two-dimensional array of SEW-bit storage elements from memory and transposes those storage elements into the vector register-group layout.

When each logical matrix element occupies one SEW-bit storage element, this operation is also a logical matrix transpose. When multiple logical elements are packed within a storage element, the instruction does not transpose the packed logical elements within it.

`rs1` holds the matrix base address; `rs2` holds the leading dimension LD, measured in SEW-bit storage elements.

Let $linesize = \lambda \times LMUL$. For each element index `i` in the body [`vstart`, VL) where the mask is enabled:

```
VD[i] = M[rs1 + (SEW ÷ 8) × ((i % linesize) × LD + (i / linesize))]
```

36.9.1.2. **vmtts.v** — Transposing Tile Store

```
vmtts.v vs3, (rs1), rs2 [, LA] [, vm]
```

Stores a two-dimensional array of **SEW**-bit storage elements from a vector register group to memory, applying the inverse storage-element transposition of **vmttl.v**.

The instruction does not unpack, transpose, or repack logical elements contained within an **SEW**-bit storage element.

rs1 holds the matrix base address; **rs2** holds the leading dimension **LD**, measured in **SEW**-bit storage elements.

Let $linesize = \lambda \times LMUL$. For each element index **i** in the body [**vstart**, **VL**) where the mask is enabled:

$$M[rs1 + (SEW \div 8) \times ((i \% linesize) \times LD + (i / linesize))] = VS[i]$$

vmttl.v and **vmtts.v** transpose **SEW**-bit storage elements. They do not transpose logical elements packed within those storage elements.

A direct logical-element transpose is therefore obtained only when the logical element width equals **SEW** and is at least 8 bits.



For widening input tiles, where $W > 1$ and the logical input width is SEW / W , these instructions do not directly transform an ordinary matrix of narrow logical elements into the packed register layout required by the multiply-accumulate instruction.

Sub-byte logical elements, including **OFP4** and **Int4**, cannot be transposed individually by these instructions. Software can prepare such data by pretransposing and packing it into the required **SEW**-bit storage-element layout, or by using separate vector rearrangement operations.

For example, consider **FP16** inputs accumulated into **FP32**, with **SEW=32** and **W=2**. A row-major **B** matrix has shape **K_{eff}** by **N**, while the architectural **B** tile has shape **N** by **K_{eff}** and satisfies:

$$B_tile[j,k] = B_matrix[k,j]$$


To prepare the **B** tile, software may transpose the matrix panel and pack the values **B_tile[j,2q]** and **B_tile[j,2q+1]** into one 32-bit storage element. The resulting packed tile can then be loaded with **vmttl.v** using **SEW=32**.

Using **vmttl.v** with **SEW=32** on the original row-major **FP16** matrix would transpose pairs of packed **FP16** values as 32-bit storage elements; it would not perform the required element-by-element **FP16** transpose.

The same rule applies for **W=4** and **W=8**, including **OFP4** and **Int4** input tiles.

36.9.2. Tile geometry for multiply-accumulate

The tile dimensions are fully determined by the current vector configuration:

M_tile	$= VLEN / (SEW \times \lambda)$	(rows of A and C per computation)
K_eff	$= \lambda \times W \times LMUL$	(shared K-dimension step)
N_tile	$= VL / (\lambda \times LMUL)$	(active columns of B_tile^T and C; set via <code>vsetvl</code>)
N_tile_max	$= M_tile$	(maximum columns when $VL = VL_max$)
$EMUL_C$	$= VLEN / (SEW \times \lambda^2)$	(C accumulator register group size)

M_tile is fixed by the hardware ($VLEN$) and the chosen SEW and λ ; it cannot be changed by VL . N_tile is controlled by the programmer through VL ; use `vsetvl` to select the largest N_tile that fits the remaining columns. VL (set via `vset{i}vl{i}` at `vtype.SEW` equal to the C element width) must be a multiple of $\lambda \times LMUL$, so only full columns can be selected.

The C accumulator tile has a fixed physical shape:

$$M_tile \times N_tile_max$$

where:

$$N_tile_max = M_tile$$

Thus the physical row-major position of C element $C_{i,j}$ within the C accumulator tile is:

$$i \times N_tile_max + j$$

The active computation updates only the sub-tile:

$$\begin{aligned} 0 \leq i < M_tile \\ 0 \leq j < N_tile \end{aligned}$$

The remaining physical C columns, where $N_tile \leq j < N_tile_max$, are C tile tail elements.

Because the multiply-accumulate instructions always read all M_tile rows of A independent of VL , loading an A tile requires:

$$VL = M_tile \times \lambda \times LMUL$$

which is the natural maximum VL for the chosen $LMUL$.

The B_tile load and computation use the narrower active-column VL :

$$VL = \lambda \times LMUL \times N_tile$$

The compute **VL** selects the number of active columns of B_{tile}^T and, equivalently, the number of active columns of **C**. It does not change the physical row stride of the **C** accumulator tile, which remains **N_tile_max**.

36.9.3. C tile tail policy

When **N_tile** < **N_tile_max**, the elements $C_{i,j}$ for $0 \leq i < M_tile$ and **N_tile** ≤ **j** < **N_tile_max** are *C tile tail elements*. Multiply-accumulate instructions never read or write **C** tile tail elements.

C tile tail elements are defined by the two-dimensional **C** tile geometry, not by the ordinary one-dimensional vector condition **element_index** ≥ **VL**. In particular, active **C** elements are those whose column index satisfies **j** < **N_tile**, even though their physical register-group indices are computed using the fixed row stride **N_tile_max**.

- If **vta** = 0 (tail-undisturbed): **C** tile tail elements retain their pre-instruction values. This is achieved by write-skip - the instruction simply does not access those element positions. Implementations are therefore not required to read the tail columns of the **C** register group.
- If **vta** = 1 (tail-agnostic): **C** tile tail elements may be written with any value, including all-ones, or left unchanged. Implementations are likewise not required to read the tail elements before deciding what to write.

This differs from the tail policy for ordinary vector instructions, where tail-undisturbed is typically implemented via read-merge: the implementation reads the old destination register contents, updates active positions, and writes the merged result back.



*For IME multiply-accumulate instructions, undisturbed behaviour can be implemented without reading **C** tile tail elements: the instruction leaves tail element positions untouched. This permits hardware implementations, including outer-product engines and register-renaming machines, to process only the **N_tile** active columns of the **C** tile without referencing the inactive **C** tile columns.*

36.9.4. Memory layout and instruction selection

Matrix / memory order	Load instruction	Store instruction
A, row-major	vmtl.v (order-preserving)	—
A, column-major	vmttl.v (transposing)	vmmts.v
B, column-major	vmtl.v (order-preserving)	—
B, row-major	vmttl.v (transposing)	—
C, row-major	vmtl.v (if loaded from memory)	vmmts.v
C, column-major	vmttl.v (transposing)	vmmts.v



*The **vmttl.v** and **vmmts.v** entries in this table denote a direct logical-element transpose only when each logical matrix element occupies one **SEW**-bit storage element.*

*For widening **A** or **B** inputs, **vmttl.v** and **vmmts.v** transpose packed **SEW**-bit storage elements, not the individual logical elements within them. Software preparing a widening input tile from*



an ordinary row-major or column-major matrix of narrow logical elements must first arrange the data into the required packed tile order, either in memory or with separate vector rearrangement operations.

Performance note: The tile load/store instructions allow software to operate directly on matrices stored in common row-major or column-major layouts. This is useful for simple kernels, edge cases, and avoiding unnecessary data movement when peak throughput is not required.

High-performance GEMM implementations are still expected to use dense packed panels when packing improves locality, alignment, prefetching, or reuse, as is common in BLAS- and BLIS-style libraries. The IME load/store instructions define architectural tile movement, but they do not preclude software from using packed matrix layouts for performance.

36.9.5. Loading and storing C accumulator tiles

The generic tile load/store instructions retain their ordinary vector element-status semantics: **VL** defines the one-dimensional body of the tile load/store instruction, masks suppress individual body elements, inactive stores do not write memory, and inactive or tail elements do not raise memory exceptions.

When loading or storing a C accumulator tile, software shall use the fixed physical C tile layout:

$$M_tile \times N_tile_max$$

where $N_tile_max = M_tile$.

For $EMUL_C \leq 8$, a full physical C tile can be loaded or stored with a single tile load/store instruction by configuring the tile load/store with:

```
SEW      = C element width
LMUL     = EMUL_C
VL_C_full = M_tile × N_tile_max
```

With this configuration:

$$linesize = \lambda \times EMUL_C = N_tile_max$$

so each tile load/store line corresponds to one physical row of the C accumulator tile.

For a row-major C matrix in memory, use **vm_{tl}.v** to load C and **vm_{ts}.v** to store C. For a column-major C matrix in memory, use **vm_{ttl}.v** to load C and **vm_{mts}.v** to store C.

For partial C column blocks where $N_tile < N_tile_max$, software shall not use $VL = M_tile \times N_tile$ to load or store C, because the active C columns are not a contiguous one-dimensional segment of the fixed physical C tile.

Instead, software should use $VL_C_full = M_tile \times N_tile_max$ and a mask that enables only the active C columns. For physical C tile element index **p** in row-major order:

```
row = p / N_tile_max
col = p % N_tile_max
```

the C load/store mask is:

```
mask[p] = (col < N_tile)
```

Masked-off C tile columns do not perform memory accesses on stores, and do not raise memory exceptions on loads or stores. Therefore, the same masked C tile load/store sequence can be used for full-width and partial-width C column blocks.



The compute $VL = \lambda \times LMUL \times N_tile$ used by a multiply-accumulate instruction is not the correct VL for loading or storing a fixed physical C accumulator tile. The compute VL selects active output columns for the multiply-accumulate instruction; C load/store uses the physical C tile layout and, for partial column blocks, a column mask.

When $EMUL_C = 16$, $LMUL=16$ is not a legal $vtype$ setting, so there is no single-instruction C tile load/store form for the full C accumulator tile. Software can use the IME $m16$ pair/unpair operations to decompose the C accumulator into two $m8$ halves, then load or store each half with ordinary tile load/store instructions using $LMUL=8$.

Let:

```
C_half_cols = 8 × λ
```

The low $m8$ half contains columns:

```
0 ≤ j < C_half_cols
```



and the high $m8$ half contains columns:

```
C_half_cols ≤ j < N_tile_max
```

For partial column blocks, the low-half and high-half stores use masks constructed with the same column-mask rule, but with C_half_cols as the line size for each half. The high-half store is skipped when $N_tile \leq C_half_cols$.

A masked half-store reads its mask from $v0$. When the $m16$ accumulator is placed at $v0-v15$, $v0$ is part of the accumulator group and cannot also serve as the mask, so a masked (partial-column) half-store is not expressible for that placement. Software that needs masked half-stores shall place the $m16$ accumulator at $v16-v31$. (Full-tile half-stores, which use no mask, are expressible at either placement.)

36.9.6. Example: single-precision floating-point GEMM

The following pseudocode implements $C \leftarrow C + A \times B$ for single-precision floating-point data, tiled over all

three matrix dimensions. The configuration uses SEW=32, LMUL=1, and $\lambda=4$, giving $K_{\text{eff}} = 4$ and $M_{\text{tile}} = \text{VLEN}/128$. Matrices A and C are stored row-major; Matrix B is stored column-major, allowing its columns to be loaded directly into the column-major register layout of B_{tile}^T .

```
// C (M×N, row-major, stride ldc) += A_matrix (M×K, row-major, stride lda)
//                                     × B_matrix (K×N, column-major, stride ldb)
//
// Pre-condition: vtype can be configured for SEW=32, LMUL=1, lambda=4.
// Pre-condition: K is a multiple of K_eff (= lambda × LMUL = 4).
// For simplicity, this example assumes M is a multiple of M_tile.

const int lambda      = 4;
const int LMUL_AB     = 1;
const int K_eff       = lambda * LMUL_AB;
const int M_tile      = VLEN / (32 * lambda);
const int N_tile_max  = M_tile;
const int EMUL_C      = M_tile / lambda;

// The A tile is loaded at the C accumulator SEW, so its VL is counted
// in SEW-wide storage elements and is independent of the widening
// factor W: VL_A = M_tile * lambda * LMUL_AB (here = M_tile * K_eff,
// because this example is non-widening). A widening variant reuses
// this same VL_A unchanged.
const int VL_A        = M_tile * lambda * LMUL_AB;
const int VL_C_full   = M_tile * N_tile_max;

for (j = 0; j < N; j += N_tile) {
    // Choose N_tile: how many B/C columns to handle this iteration.
    // vsetvl rounds down to a multiple of K_eff automatically.
    configure_vtype(SEW=32, LMUL=1, lambda=4);
    vl_compute = vsetvl_e32m1(K_eff * (N - j));
    N_tile     = vl_compute / K_eff;

    // Construct a C-column mask in v0 for the fixed physical C tile.
    // For physical C tile element p:
    //     row = p / N_tile_max
    //     col = p % N_tile_max
    // mask[p] = (col < N_tile)
    make_c_column_mask(v0, N_tile, N_tile_max, VL_C_full);

    for (i = 0; i < M; i += M_tile) {

        // Load the existing C accumulator tile.
        // C is physically M_tile × N_tile_max, so use the full physical C VL.
        // The mask suppresses inactive columns when N_tile < N_tile_max.
        configure_vtype(SEW=32, LMUL=EMUL_C, lambda=4);
        vsetvl(VL_C_full);
        vmtl.v v8, &C[i*ldc + j], ldc, v0.t;
```

```

// Inner K loop: accumulate A_tile × B_tile^T into the C tile.
for (k = 0; k < K; k += K_eff) {

    // Load A[i:i+M_tile][k:k+K_eff]: M_tile rows, K_eff cols, row-
major.
    // Must use VL_A = M_tile × K_eff so all rows are written to v2.
    configure_vtype(SEW=32, LMUL=1, lambda=4);
    vsetvl(VL_A);
    vmtl.v v2, &A[i*lda + k], lda;    // LD = lda (A row stride)

    // Load a K_eff × N_tile panel of the B_matrix.
    //
    // B_matrix is column-major, so the order-preserving tile load
copies
    // its columns directly into the column-major register layout of
B_tile^T:
    //
    //      B_tile^T[k_local, j_local]
    //      = B_matrix[k + k_local, j + j_local]
    //
    // The compute VL selects the active columns of B_tile^T and C.
    configure_vtype(SEW=32, LMUL=1, lambda=4);
    vsetvl(vl_compute);
    vmtl.v      v4, &B_matrix[j*ldb + k], ldb;
    vfmmacc.vv v8, v2, v4;    // C += A_tile × B_tile^T
}

    // Store the C accumulator tile back to memory.
    // Use the full physical C VL and the same active-column mask.
    configure_vtype(SEW=32, LMUL=EMUL_C, lambda=4);
    vsetvl(VL_C_full);
    vmts.v v8, &C[i*ldc + j], ldc, v0.t;
}
}

```

Key observations:

- The A tile must be loaded with $VL = M_tile \times \lambda \times LMUL$ (the maximum VL for the chosen A/B LMUL) because `vfmmacc.vv` always reads all M_tile rows of A regardless of the VL set for the computation. A separate vector-configuration step is therefore required before each `vmtl.v` for A.
- The order-preserving `B_matrix` load constructs B_{tile}^T directly. The load and the multiply-accumulate instruction share $VL = \lambda \times LMUL \times N_tile$, which selects the active columns of B_{tile}^T and C.
- The C accumulator tile has fixed physical shape $M_tile \times N_tile_max$, where $N_tile_max = M_tile$. Loading or storing C therefore uses the full physical C tile $VL = M_tile \times N_tile_max$ with $LMUL = EMUL_C$, not the compute VL.
- For partial column blocks where $N_tile < N_tile_max$, C load/store uses a mask that enables only physical C tile elements whose column index is less than N_tile . Masked-off C elements do not access memory.

- The A and B tile loads must be performed with the target **SEW**, which is the element width of the C accumulator tile.
- For integer GEMM, substitute **vmacc.vv** for **vfmacc.vv** and set **altfmt_A** and **altfmt_B** in **vtype** to select the desired operand signedness. The load instructions are unchanged.
- For widening variants (W=2, W=4, or W=8), use **vfwmmacc.vv**, **vfqmmacc.vv**, or **vf8wmmacc.vv** (FP), or **vmmacc.vv**, **vqmmacc.vv**, or **v8wmmacc.vv** (integer). The narrower A and B element widths are encoded in the multiply-accumulate instruction. At the assembly level, A and B are still loaded with tile load instructions using **SEW** equal to the C accumulator width. In the C intrinsic interface, use the storage-width-qualified **_as_e{S}** tile load/store forms when the logical A/B element type is narrower than the storage **SEW**.

36.10. Software considerations

36.10.1. C-Language Intrinsics

The IME C-language intrinsics provide portable access to the IME instructions without requiring hand-written assembly. The naming convention and type system extend the RISC-V V Intrinsics API (RISC-V International, *RISC-V V Intrinsics API Specification*):

- Function names take the form `_riscv{mnemonic}_{type-suffix}`.
- The *type-suffix* encodes the **accumulator C** type for multiply-accumulate intrinsics, and the destination/source register group type for tile load/store intrinsics.

For register-group multipliers **m1**, **m2**, **m4**, and **m8**, the type suffixes use the same spelling as the RISC-V V intrinsics: `i8m1` $\rightarrow$ `vint8m1_t`, `f32m2` $\rightarrow$ `vfloat32m2_t`, `u16m4` $\rightarrow$ `vu16m4_t`, etc.

IME additionally defines **m16** type suffixes for accumulator register groups with **EMUL_C = 16**. An **m16** suffix in an IME multiply-accumulate intrinsic denotes an IME accumulator register group occupying sixteen consecutive vector registers. It does not denote **vtype.LMUL = 16**.

- For multiply-accumulate intrinsics, the register-group multiplier encoded in the accumulator type suffix is $\text{EMUL_C} = \text{VLEN} \div (\text{SEW} \times \lambda^2)$, which is determined by **VLEN**, **SEW**, and the selected **lambda**, and is independent of **LMUL**.
- A `_lm{N}` qualifier ($N \in \{1, 2, 4, 8\}$) is appended to the intrinsic name to specify the **LMUL** used by the A and B input operands. The `_lm1` qualifier may be omitted; **LMUL=1** is the default when no qualifier is present. The **vs1** and **vs2** argument types directly reflect **LMUL**: `vx{EEW}m{LMUL}_t`.
- For tile load/store intrinsics, the register-group multiplier in the type suffix denotes the **vtype.LMUL** used by the load/store instruction. Since **LMUL=16** is not a legal **vtype** setting, no single-instruction **m16** tile load/store intrinsic is defined. Software may use the IME pair/unpair intrinsics described in [Section 36.10.1.2](#) when it needs to compose or decompose an **m16** accumulator value from **m8** halves.
- Every instruction-mapping tile load/store or multiply-accumulate intrinsic accepts a final **size_t vl** argument equal to the vector length used for that instruction. Non-instruction intrinsics, such as geometry-query intrinsics and IME pair/unpair pseudo-intrinsics, do not take a **vl** argument.
- Instruction-mapping tile load/store and multiply-accumulate intrinsics are implicitly barrier-like with respect to the vector register file: the compiler must not reorder them across other vector operations that touch the same register state. Non-instruction intrinsics have the ordering properties specified in their own descriptions.
- When the input element type is non-ISO (BFloat16, OFP8, OFP4, or Int4), the input-type name and register-group multiplier are appended as a suffix (`_{type}m{LMUL}`) to the intrinsic name, even for the default **altfmt=0** encoding. When the input type is a standard IEEE floating-point format (FP16, FP32, FP64) or a standard-width integer (Int8–Int64), no input-type suffix is needed for the default encoding.
- When **altfmt_A** $\neq$ **altfmt_B** (mixed input formats), both input-type suffixes appear separated by an underscore, with the A suffix first: `_riscv{mnemonic}_vv_{accum}_{inputA}_{inputB}`. When **altfmt_A** = **altfmt_B**, a single input-type suffix suffices.
- The canonical suffix ordering for multiply-accumulate long-form names is:

`{type-suffix}[_{inputA}[_{inputB}]][_su|_us][_lm{N}]`

where each bracketed component is omitted when not applicable.

- The canonical suffix ordering for tile load/store long-form names is:

```
{type-suffix}[_as_e{S}][_L{N}][_m]
```

The optional `_as_e{S}` qualifier is defined in [Section 36.10.1.6.1](#). The `_L{N}` immediate-lambda qualifier precedes `_m`, and `_m` always appears last.

36.10.1.1. IME-only m16 accumulator types

IME defines additional **m16** accumulator types for use by IME multiply-accumulate intrinsics when `EMUL_C = 16`.

An IME **m16** type denotes a sixteen-register accumulator group. It is not an ordinary RVV vector intrinsic type, and it does not imply that `LMUL=16` is a legal `vtype` setting. The `LMUL` used by the A and B input operands remains specified by the `_lm{N}` qualifier or, in overloaded forms, by the `vs1` and `vs2` argument types.

For each accumulator element type supported by IME, the C API defines an **m16** type when the corresponding **m1**, **m2**, **m4**, and **m8** accumulator types are defined. Examples include:

```
vint8m16_t
vint16m16_t
vint32m16_t
vint64m16_t

vfloat8e4m3m16_t
vfloat8e5m2m16_t
vfloat16m16_t
vbf16m16_t
vfloat32m16_t
vfloat64m16_t
```

An IME **m16** type is valid as:

- the accumulator operand and result type of an IME multiply-accumulate intrinsic when `EMUL_C = 16`;
- the operand or result type of the IME pair/unpair intrinsics described in [Section 36.10.1.2](#);
- a compiler-managed temporary, assignment, or copy object when supported by the implementation.

An IME **m16** type is not valid as the operand type of ordinary RVV arithmetic intrinsics. The IME C API does not define ordinary `vsetvl_e{SEW}m16`, `vsetvlmax_e{SEW}m16`, `vle{SEW}_v_*m16`, or `vse{SEW}_v_*m16` intrinsics.



Register placement of an m16 accumulator. A sixteen-register group must be sixteen-register-aligned, so an **m16** accumulator occupies either `v0–v15` or `v16–v31` — half of the vector register file, in one of only two placements. The A and B input operands (each at most an eight-register **m8** group) must therefore reside in the opposite half. A kernel that needs an **m16** accumulator and **m8** inputs simultaneously consumes the entire register file: sixteen registers for C and up to sixteen for A, B, and any other live vector state.

*Because the mask register is **v0**, a masked operation is not expressible when the **m16** accumulator is placed at **v0–v15**; such a kernel must place the accumulator at **v16–v31**. Masked IME multiply-accumulate is not defined in any case (see the multiply-accumulate descriptions), so this affects only masked tile load/store of the **m8** halves around an **m16** accumulator.*

36.10.1.2. Pairing and unpairing **m16** accumulator values

IME defines pair/unpair pseudo-intrinsics that allow software to compose an IME **m16** accumulator value from two **m8** values, and to decompose an IME **m16** accumulator value into two **m8** values.

These intrinsics are compiler builtins or pseudo-intrinsics; they do not correspond to new ISA instructions. Their cost and side-effect model is given below.

```
/* Representative long-form examples. */

vint32m16_t
__riscv_ime_vpair_i32m16(vint32m8_t lo, vint32m8_t hi);

vint32m8_t
__riscv_ime_vunpairlo_i32m16(vint32m16_t src);

vint32m8_t
__riscv_ime_vunpairhi_i32m16(vint32m16_t src);

vfloat32m16_t
__riscv_ime_vpair_f32m16(vfloat32m8_t lo, vfloat32m8_t hi);

vfloat32m8_t
__riscv_ime_vunpairlo_f32m16(vfloat32m16_t src);

vfloat32m8_t
__riscv_ime_vunpairhi_f32m16(vfloat32m16_t src);
```

A pair/unpair triple is defined for every accumulator element type that has an **m16** accumulator type (see [Section 36.10.1.1](#)), including the non-ISO accumulator formats. The long-form name embeds the same type token used by the **m16** type. For example:

```
/* OFP8 (E4M3) accumulator. */
vfloat8e4m3m16_t __riscv_ime_vpair_f8e4m3m16(vfloat8e4m3m8_t lo,
vfloat8e4m3m8_t hi);
vfloat8e4m3m8_t __riscv_ime_vunpairlo_f8e4m3m16(vfloat8e4m3m16_t src);
vfloat8e4m3m8_t __riscv_ime_vunpairhi_f8e4m3m16(vfloat8e4m3m16_t src);

/* OFP8 (E5M2) accumulator. */
vfloat8e5m2m16_t __riscv_ime_vpair_f8e5m2m16(vfloat8e5m2m8_t lo,
vfloat8e5m2m8_t hi);
```

```

/* BFloat16 accumulator. */
vbf16m16_t __riscv_ime_vpair_bf16m16(vbf16m8_t lo, vbf16m8_t hi);
vbf16m8_t __riscv_ime_vunpairlo_bf16m16(vbf16m16_t src);
vbf16m8_t __riscv_ime_vunpairhi_bf16m16(vbf16m16_t src);

/* IEEE binary16, and the integer accumulators, follow the same pattern:
   __riscv_ime_vpair_f16m16, _i8m16, _i16m16, _i32m16, _i64m16,
   _u8m16 ... and their _vunpairlo / _vunpairhi counterparts. */

```

The overloaded forms are:

```

T_m16 __riscv_ime_vpair(T_m8 lo, T_m8 hi);
T_m8 __riscv_ime_vunpairlo(T_m16 src);
T_m8 __riscv_ime_vunpairhi(T_m16 src);

```

`_riscv_ime_vpair*m16(lo, hi)` constructs an `m16` accumulator value whose low half is the eight-register group `lo` and whose high half is the eight-register group `hi`.

`_riscv_ime_vunpairlo*m16(src)` returns the low eight-register half of `src`.

`_riscv_ime_vunpairhi*m16(src)` returns the high eight-register half of `src`.

The pair and unpair operations differ in their cost and aliasing model:

- `riscv_ime_vunpairlo` and `riscv_ime_vunpairhi` are *register-preserving*. The returned `m8` value is the low (respectively high) eight-register subgroup of the `m16` source. They perform no data movement and emit no instructions, and the result *aliases* the source register group: while the `m16` source is still live, modifying a returned half modifies the source. In the usual unpair-then-store pattern the source is consumed immediately, so the aliasing is benign.
- `_riscv_ime_vpair` is *_value-copying*. It constructs a new, sixteen-register-aligned `m16` group whose low and high halves are `lo` and `hi`. Where the inputs are not already the aligned halves of a sixteen-register group, the implementation *may emit register moves* to place them. The result is independent of the input objects.

Neither operation reads or writes memory, changes `vl`, `vtype`, `vstart`, or `fflags`, and neither performs element reordering, rounding, saturation, sign extension, zero extension, or floating-point format conversion.

36.10.1.3. Overloaded short forms (GCC and Clang)

In addition to the canonical long-form names described above, each intrinsic is available under the shorter name `riscv_{mnemonic}` with compiler-managed overload resolution: GCC resolves calls through its `resolve_overloaded_builtin` mechanism; Clang uses `attribute__overloadable`; C++ compilers use standard function overloading. In these short forms the compiler selects the correct instruction variant solely from the types of `vd`, `vs1`, and `vs2`; no `_lm{N}` qualifier, no input-type suffix, and no `_su` / `_us` sign suffix is needed in user code.

The types of the arguments fully determine every variant:

- `vd` type → accumulator SEW and `EMUL_C`.

An overloaded multiply-accumulate intrinsic may use an IME **m16** accumulator type for **vd** when the selected tile geometry gives **EMUL_C = 16**. The **vs1** and **vs2** argument types continue to determine the A and B input **LMUL**, and therefore use only **m1**, **m2**, **m4**, or **m8**.

- **vs1** / **vs2** types → input EEW and **LMUL**; for integer operations, signed vs. unsigned; for floating-point operations, the FP format (e.g., FP16 vs. BF16 for **altfmt**).

```
/* Long forms – always portable, compiler-independent: */
vint32m2_t __riscv_vmmacc_vv_i32m2 (vint32m2_t vd, vint32m1_t vs1,
vint32m1_t vs2, size_t vl);
vint32m2_t __riscv_vmmacc_vv_i32m2_lm2(vint32m2_t vd, vint32m2_t vs1,
vint32m2_t vs2, size_t vl);

/* Equivalent short forms – overload resolution picks the right variant: */
vint32m2_t __riscv_vmmacc_vv(vint32m2_t vd, vint32m1_t vs1, vint32m1_t vs2,
size_t vl) __attribute__((overloadable));
vint32m2_t __riscv_vmmacc_vv(vint32m2_t vd, vint32m2_t vs1, vint32m2_t vs2,
size_t vl) __attribute__((overloadable));

/* FP format (altfmt) resolved by vs1/vs2 type – no suffix needed: */
vfloat32m2_t __riscv_vfwmacc_vv(vfloat32m2_t vd, vfloat16m1_t vs1,
vfloat16m1_t vs2, size_t vl) __attribute__((overloadable));
vfloat32m2_t __riscv_vfwmacc_vv(vfloat32m2_t vd, vbf16m1_t vs1,
vbf16m1_t vs2, size_t vl) __attribute__((overloadable));

/* Integer sign combination resolved by vs1/vs2 types – no _su/_us suffix: */
vint32m2_t __riscv_vwmacc_vv(vint32m2_t vd, vint16m1_t vs1, vuint16m1_t vs2,
size_t vl) __attribute__((overloadable));
vint32m2_t __riscv_vwmacc_vv(vint32m2_t vd, vuint16m1_t vs1, vint16m1_t vs2,
size_t vl) __attribute__((overloadable));

/* EMUL_C=16 accumulator; A/B use LMUL=1. */
vfloat32m16_t __riscv_vfmmacc_vv(vfloat32m16_t vd, vfloat32m1_t vs1,
vfloat32m1_t vs2, size_t vl) __attribute__((overloadable));
```



The mechanism used for overload resolution in C is compiler-specific (resolve_overloaded_builtin in GCC, attribute(overloadable) in Clang). The resulting user-visible API is identical across compilers; only the internal implementation differs.

36.10.1.4. Geometry-query intrinsics

Two non-instruction intrinsics expose the implementation's IME geometry to software:

```
size_t __riscv_ime_vlen (void); /* VLEN in bits */
size_t __riscv_ime_lambda (void); /* current selected lambda */
```

__riscv_ime_vlen() returns the architectural VLEN (in bits). When VLEN is known to the compiler — for example via **-mrvv-vector-bits=zvl** on GCC or Clang — the call folds to a load-immediate; otherwise the

compiler emits `csrr vlenb` followed by a left-shift by 3.

`__riscv_ime_lambda()` is a pure query of the currently selected lambda value. It reads `vtype.lambda[2:0]`, decodes the field, and returns the resulting lambda value without modifying `vtype`, `vl`, or any vector register.

In a legal, configured `vtype` the returned value is always a nonzero lambda that is valid for the current (`VLEN`, `SEW`) configuration, because the supported set is non-empty. A return value of zero indicates only an as-yet-unconfigured `vtype` (no configuration instruction has yet selected a lambda).

Because an implementation may support multiple lambda values for the same `VLEN` and `SEW`, the result cannot in general be derived from `VLEN` alone. A compiler may fold the query to a constant only when it can prove the current value of `vtype.lambda`.

Otherwise, the compiler shall emit a read of `vtype` or an equivalent implementation-defined sequence that returns the current selected lambda.

Software that needs to enumerate all supported lambda values may use `__riscv_vsetlambda` to request each nonzero candidate and inspect the returned value. Because those requests may change the selected lambda, software should restore its desired value after enumeration.

36.10.1.5. Establishing lambda from C: `__riscv_vsetlambda`

```
size_t __riscv_vsetlambda(size_t requested_lambda);
```

`__riscv_vsetlambda` is the C-level primitive for submitting a lambda write request and returning the lambda value established by that request. It preserves all other `vtype` fields and preserves `vl`.

If `requested_lambda` is a positive power of two in {1, 2, 4, 8, 16, 32, 64}, the intrinsic requests the corresponding nonzero lambda value.

If the requested value is supported for the current (`VLEN`, `SEW`) configuration, that value is established.

If it is not supported, the implementation establishes the largest supported nonzero lambda value that is less than or equal to the request.

If no supported value is less than or equal to the request, the implementation establishes the smallest supported nonzero lambda value.

For a configuration in the IME-legal domain the supported set is non-empty, so the established value is a supported nonzero value. For a configuration outside that domain no nonzero lambda is permissible and the established (and returned) value is zero.

If `requested_lambda` is zero, the intrinsic performs the architectural preserve-or-initialize write:

- If the currently selected nonzero lambda value is valid for the current (`VLEN`, `SEW`) configuration, it is retained.
- Otherwise, if the current (`VLEN`, `SEW`) configuration is in the IME-legal domain, the implementation selects the largest supported nonzero lambda value for that configuration.
- Otherwise, the current configuration is outside the IME-legal domain; no nonzero lambda is selected and the returned value is zero.

The intrinsic returns the selected value after processing the request.

`riscv_vsetlambda(0)` is therefore not a pure read operation. Software that only needs to inspect the selected `lambda` without modifying architectural state should use `riscv_ime_lambda()`.

The set of supported `lambda` values is a function of (`VLEN`, `SEW`) and is non-empty within the IME-legal domain. A change to `SEW` may cause `lambda` to be retained or replaced by another supported nonzero value. A change to `LMUL` alone shall not change `lambda`.

Portable code that depends on a specific `lambda` value should re-establish or re-read `lambda` after any `vsetvli` or `vsetivli` that may change `SEW`, and after any `vsetvl` whose `rs2` value is not known to request the desired `lambda`.

Compiler-modelling guidance (non-normative).

A compiler should model the `lambda` query and write intrinsics as follows:

Operation	DEFs	USEs	Notes
<code>__riscv_ime_lambda()</code>	<i>none</i>	<code>vtype.lambda</code>	Pure read of the currently selected <code>lambda</code> . The compiler may move or combine such reads subject to ordinary read-dependence rules.
<code>__riscv_vsetlambda(0)</code>	<code>lambda</code> (may-DEF)	<code>lambda</code> , <code>SEW</code>	Preserve-or-initialize write. The current value may be retained, may be replaced by a supported nonzero value in the IME-legal domain, or may remain zero outside that domain.
<code>__riscv_vsetlambda</code> with a compile-time nonzero argument	<code>lambda</code>	<code>SEW</code>	Nonzero <code>lambda</code> request followed by WARL canonicalization.
<code>__riscv_vsetlambda</code> with a runtime argument	<code>lambda</code> (may-DEF)	<code>lambda</code> , <code>SEW</code>	The compiler cannot in general determine whether the argument is zero. Conservative ordering should treat the operation as a <code>lambda</code> definition.



A `vsetvli` or `vsetivli` that may change `SEW` should be modeled as a may-DEF of `lambda`, because the selected value may need to be canonicalized for the new `SEW`.

A vector-configuration operation that changes only `LMUL` does not define or modify `lambda`.

The `vsetvl` instruction is a full `vtype` write and should be modeled as a hard DEF of all `vtype` fields. If the `lambda` bits supplied in `rs2` may be zero, the operation also uses the old `lambda` value because a zero request can preserve the old value.

Repeated `lambda` operations may be eliminated when their dataflow permits. For example, a second `riscv_vsetlambda(0)` is redundant when it immediately follows another `riscv_vsetlambda(0)` and no intervening operation can affect `SEW` or `lambda`.

Typical usage is:

```
/* Purely inspect the selected lambda. */
size_t current = __riscv_ime_lambda();
```

```

/* Preserve a valid lambda, or initialize one if repair is
needed. */
size_t selected = __riscv_vsetlambda(0);

/* Save and restore after restoring the surrounding vector
configuration. */
size_t saved = __riscv_ime_lambda();
external_function();
restore_vector_configuration();
__riscv_vsetlambda(saved);

```

36.10.1.6. Tile load and store (Zvmtls) and transposed tile load and store (Zvmttls)

The load intrinsics accept a base pointer, a leading-dimension `ld` in elements, and a vector-length `vl`, returning the loaded vector type. The store intrinsics accept a base pointer, `ld`, the source vector, and `vl`, and return `void`. For tile loads and stores the type-suffix encodes LMUL—the register-group multiplier of the tile operand. The table below lists representative `m1` prototypes; replace `m1` with `m2`, `m4`, or `m8` as needed, and `i` with `u` for unsigned integer variants where such variants are defined.

```

/* vmtl.v – order-preserving tile load */
vint8m1_t    __riscv_vmtl_v_i8m1  (const int8_t    *base, size_t ld, size_t vl);
vint16m1_t   __riscv_vmtl_v_i16m1 (const int16_t   *base, size_t ld, size_t vl);
vint32m1_t   __riscv_vmtl_v_i32m1 (const int32_t   *base, size_t ld, size_t vl);
vint64m1_t   __riscv_vmtl_v_i64m1 (const int64_t   *base, size_t ld, size_t vl);
vfloat16m1_t __riscv_vmtl_v_f16m1 (const _Float16 *base, size_t ld, size_t vl);
vfloat32m1_t __riscv_vmtl_v_f32m1 (const float     *base, size_t ld, size_t vl);
vfloat64m1_t __riscv_vmtl_v_f64m1 (const double    *base, size_t ld, size_t vl);

/* vmts.v – order-preserving tile store */
void __riscv_vmts_v_i8m1  (int8_t    *base, size_t ld, vint8m1_t    vs3, size_t vl);
void __riscv_vmts_v_i16m1 (int16_t   *base, size_t ld, vint16m1_t   vs3, size_t vl);
void __riscv_vmts_v_i32m1 (int32_t   *base, size_t ld, vint32m1_t   vs3, size_t vl);
void __riscv_vmts_v_i64m1 (int64_t   *base, size_t ld, vint64m1_t   vs3, size_t vl);
void __riscv_vmts_v_f16m1 (_Float16 *base, size_t ld, vfloat16m1_t vs3, size_t vl);
void __riscv_vmts_v_f32m1 (float     *base, size_t ld, vfloat32m1_t vs3, size_t vl);
void __riscv_vmts_v_f64m1 (double    *base, size_t ld, vfloat64m1_t vs3, size_t vl);

/* vmttl.v – transposing tile load */
vint8m1_t    __riscv_vmttl_v_i8m1  (const int8_t    *base, size_t ld, size_t vl);
vint16m1_t   __riscv_vmttl_v_i16m1 (const int16_t   *base, size_t ld, size_t vl);

```

```

vint32m1_t  __riscv_vmttl_v_i32m1 (const int32_t  *base, size_t ld, size_t
vl);
vint64m1_t  __riscv_vmttl_v_i64m1 (const int64_t  *base, size_t ld, size_t
vl);
vfloat16m1_t __riscv_vmttl_v_f16m1 (const _Float16 *base, size_t ld, size_t
vl);
vfloat32m1_t __riscv_vmttl_v_f32m1 (const float    *base, size_t ld, size_t
vl);
vfloat64m1_t __riscv_vmttl_v_f64m1 (const double   *base, size_t ld, size_t
vl);

/* vmtts.v – transposing tile store */
void __riscv_vmtts_v_i8m1  (int8_t   *base, size_t ld, vint8m1_t   vs3, size_t
vl);
void __riscv_vmtts_v_i16m1 (int16_t  *base, size_t ld, vint16m1_t  vs3, size_t
vl);
void __riscv_vmtts_v_i32m1 (int32_t  *base, size_t ld, vint32m1_t  vs3, size_t
vl);
void __riscv_vmtts_v_i64m1 (int64_t  *base, size_t ld, vint64m1_t  vs3, size_t
vl);
void __riscv_vmtts_v_f16m1 (_Float16 *base, size_t ld, vfloat16m1_t vs3, size_t
vl);
void __riscv_vmtts_v_f32m1 (float     *base, size_t ld, vfloat32m1_t vs3, size_t
vl);
void __riscv_vmtts_v_f64m1 (double    *base, size_t ld, vfloat64m1_t vs3, size_t
vl);

```

36.10.1.6.1. Alternate-format and storage-width-qualified tile load/store intrinsics

Tile load/store instructions transfer memory in units of the current instruction **SEW**. The C intrinsic type describes the logical interpretation of the bits in the vector register group; it does not imply numeric conversion and, for packed narrow types, it does not by itself determine the storage element width used by the instruction.

For logical element types whose natural storage width is the same as the instruction **SEW**, the unqualified tile load/store intrinsic name is used. Examples include **Int8** at **SEW=8**, **OFP8** at **SEW=8**, **BF16** at **SEW=16**, **FP16** at **SEW=16**, **FP32** at **SEW=32**, and **FP64** at **SEW=64**.

For four-bit logical element types, including **OFP4** and **Int4**, the natural tile-load storage width is 8 bits. Each 8-bit storage element contains two logical 4-bit elements in the packing order defined in [Section 36.5.1.2](#). Therefore, the unqualified **OFP4** and **Int4** tile load/store intrinsics are aliases for the corresponding **_as_e8** forms. They do not imply that **SEW=4** is a supported vector configuration.

Table 84. Natural storage width for unqualified alternate-format tile load/store intrinsics

Logical type family	Logical element width	Natural storage width	Notes
OFP8	8	8	One logical element per storage element.
BF16	16	16	One logical element per storage element.
OFP4	4	8	Two logical elements packed per 8-bit storage element.

Logical type family	Logical element width	Natural storage width	Notes
Int4 / UInt4	4	8	Two logical elements packed per 8-bit storage element.

The unqualified natural-storage alternate-format intrinsics are defined for order-preserving tile loads and stores. Representative examples are:

```

/* OFP8 natural-storage order-preserving load/store: SEW=8. */
vfloat8e4m3m1_t __riscv_vmtl_v_f8e4m3m1(const uint8_t *base, size_t ld, size_t vl);

vfloat8e5m2m1_t __riscv_vmtl_v_f8e5m2m1(const uint8_t *base, size_t ld, size_t vl);

void __riscv_vmts_v_f8e4m3m1(uint8_t *base, size_t ld, vfloat8e4m3m1_t vs3, size_t vl);

void __riscv_vmts_v_f8e5m2m1(uint8_t *base, size_t ld, vfloat8e5m2m1_t vs3, size_t vl);

/* BF16 natural-storage order-preserving load/store: SEW=16. */
vbf16_t __riscv_vmtl_v_bf16m1(const __bf16 *base, size_t ld, size_t vl);

void __riscv_vmts_v_bf16m1(__bf16 *base, size_t ld, vbf16_t vs3, size_t vl);

/* OFP4 and Int4 natural-storage order-preserving forms use SEW=8 storage. */
vfloat4e2m1m1_t __riscv_vmtl_v_f4e2m1m1(const uint8_t *base, size_t ld, size_t vl);

vint4m1_t __riscv_vmtl_v_i4m1(const uint8_t *base, size_t ld, size_t vl);

vuint4m1_t __riscv_vmtl_v_u4m1(const uint8_t *base, size_t ld, size_t vl);

void __riscv_vmts_v_f4e2m1m1(uint8_t *base, size_t ld, vfloat4e2m1m1_t vs3, size_t vl);

void __riscv_vmts_v_i4m1(uint8_t *base, size_t ld, vint4m1_t vs3, size_t vl);

void __riscv_vmts_v_u4m1(uint8_t *base, size_t ld, vuint4m1_t vs3, size_t vl);

```

In addition to the unqualified natural-storage forms, IME defines storage-width-qualified tile load/store intrinsics using the suffix `_as_e{S}`, where `S` is the storage element width in bits.

The logical type suffix describes the logical element type of the value returned by the load or supplied to the store. The `_as_e{S}` suffix describes the `SEW` used by the tile load/store instruction and the width of one

memory storage element.

The storage width **S** shall be a supported **SEW** value for the implementation and shall be greater than or equal to the logical element width encoded by the type suffix. For 4-bit logical element types, **S** shall be at least 8. The number of logical elements packed into each storage element is $S / \text{logical_EEW}$.

The **ld** and **vl** arguments of an **_as_e{S}** intrinsic are counted in storage elements of width **S**, not in logical narrow elements. No numeric conversion is performed. The operation is a bit-preserving load or store of packed storage elements.

For storage-width-qualified tile load/store intrinsics, the base pointer type is determined by the storage width **S**, not by the logical element type. The storage pointer types are:

Table 85. Storage pointer types for storage-width-qualified tile load/store intrinsics

Suffix	Load base type	Store base type
_as_e8	<code>const uint8_t *</code>	<code>uint8_t *</code>
_as_e16	<code>const uint16_t *</code>	<code>uint16_t *</code>
_as_e32	<code>const uint32_t *</code>	<code>uint32_t *</code>
_as_e64	<code>const uint64_t *</code>	<code>uint64_t *</code>

The logical type suffix continues to determine the vector type returned by a load or supplied to a store. No numeric conversion is performed.

When **S** is the natural storage width of the logical type, the **_as_e{S}** form and the unqualified form are equivalent. For example, `riscv_vmtl_v_f8e4m3m1_as_e8` is equivalent `toriscv_vmtl_v_f8e4m3m1`, and `riscv_vmtl_v_i4m1_as_e8` is equivalent `toriscv_vmtl_v_i4m1`.

Representative storage-width-qualified examples are:

```
/* FP16 logical elements packed in 32-bit storage positions. */
vfloat16m1_t __riscv_vmtl_v_f16m1_as_e32(const uint32_t *base, size_t ld,
size_t vl);

void __riscv_vmts_v_f16m1_as_e32(uint32_t *base, size_t ld, vfloat16m1_t vs3,
size_t vl);

/* OFP8 logical elements packed in 32-bit storage positions. */
vfloat8e4m3m1_t __riscv_vmtl_v_f8e4m3m1_as_e32(const uint32_t *base, size_t ld,
size_t vl);

void __riscv_vmts_v_f8e4m3m1_as_e32(uint32_t *base, size_t ld, vfloat8e4m3m1_t
vs3, size_t vl);

/* OFP4 logical elements packed in 32-bit storage positions. */
vfloat4e2m1m1_t __riscv_vmtl_v_f4e2m1m1_as_e32(const uint32_t *base, size_t ld,
size_t vl);

void __riscv_vmts_v_f4e2m1m1_as_e32(uint32_t *base, size_t ld, vfloat4e2m1m1_t
```

```

vs3, size_t vl);

/* Int8 logical elements packed in 32-bit storage positions. */
vint8m1_t __riscv_vmtl_v_i8m1_as_e32(const uint32_t *base, size_t ld, size_t
vl);

void __riscv_vmts_v_i8m1_as_e32(uint32_t *base, size_t ld, vint8m1_t vs3,
size_t vl);

/* Int4 logical elements packed in 32-bit storage positions. */
vint4m1_t __riscv_vmtl_v_i4m1_as_e32(const uint32_t *base, size_t ld, size_t
vl);

void __riscv_vmts_v_i4m1_as_e32(uint32_t *base, size_t ld, vint4m1_t vs3,
size_t vl);

```

Storage-width-qualified intrinsics are defined for `vmtl.v` and `vmts.v`. They are the C intrinsic mechanism for loading or storing packed narrow logical input tiles using the storage SEW required by a widening multiply-accumulate instruction.

Transposing tile load/store intrinsics are defined for natural-storage forms whose logical element width equals the storage element width and is at least 8 bits. In these cases, the transpose of storage elements is also the transpose of logical elements.

Representative alternate-format transposing examples are:

```

/* OFP8 natural-storage transposing load/store: SEW=8. */
vfloat8e4m3m1_t __riscv_vmttl_v_f8e4m3m1(const uint8_t *base, size_t ld, size_t
vl);

void __riscv_vmtts_v_f8e4m3m1(uint8_t *base, size_t ld, vfloat8e4m3m1_t vs3,
size_t vl);

/* BF16 natural-storage transposing load/store: SEW=16. */
vbf16m1_t __riscv_vmttl_v_bf16m1(const __bf16 *base, size_t ld, size_t vl);

void __riscv_vmtts_v_bf16m1(__bf16 *base, size_t ld, vbf16m1_t vs3, size_t
vl);

```

The C intrinsic API does not define transposing forms for packed-narrow logical elements. For example, no OFP4 or Int4 transposing intrinsic is defined.

The underlying ISA instructions remain defined as transposes of SEW-bit storage elements. A logical-narrow transposing intrinsic is not provided because such a name could incorrectly imply that the instruction transposes the individual logical elements packed within each storage element.

A future extension may define storage-transposing `_as_e{S}` forms whose transpose applies to S-bit storage

elements rather than to logical narrow elements.

36.10.1.6.2. Immediate-lambda and mask qualifiers

When an immediate lambda override is required, a `_L{N}` variant encodes the lambda value ($N \in \{1, 2, 4, 8, 16, 32, 64\}$) directly in the intrinsic name. Because the lambda value maps to a 3-bit instruction-immediate field, the compiler can statically verify legality; no runtime argument is needed.

```
/* vmtl.v with lambda=4: */
vint32m1_t __riscv_vmtl_v_i32m1_L4 (const int32_t *base, size_t ld, size_t
vl);
/* vmts.v with lambda=4: */
void __riscv_vmts_v_i32m1_L4 (int32_t *base, size_t ld, vint32m1_t
vs3, size_t vl);
/* vmttl.v with lambda=4: */
vint32m1_t __riscv_vmttl_v_i32m1_L4 (const int32_t *base, size_t ld, size_t
vl);
/* vmtts.v with lambda=4: */
void __riscv_vmtts_v_i32m1_L4 (int32_t *base, size_t ld, vint32m1_t
vs3, size_t vl);
```

Tile load and store intrinsics support an optional mask through the `_m` suffix. The masked load takes a leading `vbool{N}_t mask` argument; the masked store takes a leading `vbool{N}_t mask` followed by the usual arguments. The mask bit width matches the data type's element width (e.g., `vbool8_t` for `i8m1`, `vbool32_t` for `i32m1`, `vbool16_t` for `bf16m1`), following the standard V-extension convention.

```
/* vmtl.v masked: */
vint8m1_t __riscv_vmtl_v_i8m1_m (vbool8_t mask, const int8_t *base, size_t
ld, size_t vl);
vint32m1_t __riscv_vmtl_v_i32m1_m (vbool32_t mask, const int32_t *base, size_t
ld, size_t vl);
/* vmts.v masked: */
void __riscv_vmts_v_i8m1_m (vbool8_t mask, int8_t *base, size_t ld,
vint8m1_t vs3, size_t vl);
void __riscv_vmts_v_i32m1_m (vbool32_t mask, int32_t *base, size_t ld,
vint32m1_t vs3, size_t vl);
/* vmttl.v masked: */
vint32m1_t __riscv_vmttl_v_i32m1_m (vbool32_t mask, const int32_t *base, size_t
ld, size_t vl);
/* vmtts.v masked: */
void __riscv_vmtts_v_i32m1_m (vbool32_t mask, int32_t *base, size_t ld,
vint32m1_t vs3, size_t vl);
```

The `_L{N}` (immediate-lambda) and `_m` (masked) qualifiers are orthogonal and may be combined. `_L{N}` precedes `_m` per the canonical suffix order:

```
/* vmtl.v with lambda=4 and a mask: */
vint8m1_t __riscv_vmtl_v_i8m1_L4_m (vbool8_t mask, const int8_t *base, size_t
```

```
ld, size_t vl);
/* vmts.v with lambda=4 and a mask: */
void __riscv_vmts_v_i8m1_L4_m(vbool8_t mask, int8_t *base, size_t ld,
vint8m1_t vs3, size_t vl);
```

For storage-width-qualified tile load/store intrinsics, the mask type is determined by the storage width *S* and LMUL, not by the logical narrow element type. For example:

```
vfloat16m1_t __riscv_vmtl_v_f16m1_as_e32_m(vbool32_t mask, const uint32_t *
base, size_t ld, size_t vl);

vfloat16m1_t __riscv_vmtl_v_f16m1_as_e32_L4_m(vbool32_t mask, const uint32_t
*base, size_t ld, size_t vl);
```

Example: loading FP16 input tiles for an FP16-by-FP16 to FP32 widening multiply-accumulate can keep SEW=32 for both the tile load and the multiply-accumulate instruction.

```
/* vtype has SEW=32, LMUL=1, and the desired lambda. */
/* Each 32-bit storage element contains two packed FP16 values. */

vfloat16m1_t a = __riscv_vmtl_v_f16m1_as_e32(A_packed, lda_in_e32, vl_in_e32);

vfloat16m1_t b = __riscv_vmtl_v_f16m1_as_e32(B_packed, ldb_in_e32, vl_in_e32);

c = __riscv_vfwmmacc_vv_f32m16_f16m1(c, a, b, vl_in_e32);
```

36.10.1.7. Integer matrix multiply-accumulate (Zvmm)

The accumulator register group *C* uses $EMUL_C = VLEN \div (SEW \times \lambda^2)$ registers. *EMUL_C* is determined by the implementation's *VLEN* and is independent of *LMUL*. The type-suffix in the intrinsic name always encodes the accumulator *C* type (including its *EMUL_C*); *A* and *B* use *vs1* and *vs2* argument types that reflect *LMUL* directly. The tables below list representative prototypes for *VLEN*=256 and $\lambda=2$ (giving $EMUL_C = VLEN \div (SEW \times 4)$); all valid (*EMUL_C*, *LMUL*) combinations are available with the appropriate *_lm{N}* qualifier.

For *vmmacc.vv* (no widening, *W*=1), accumulator and both input tiles use SEW:

```
/* SEW=8, VLEN=256, λ=2: EMUL_C=8. LMUL=1 (default): */
vint8m8_t __riscv_vmmacc_vv_i8m8 (vint8m8_t vd, vint8m1_t vs1, vint8m1_t
vs2, size_t vl);
/* SEW=16, VLEN=256, λ=2: EMUL_C=4. LMUL=1 (default): */
vint16m4_t __riscv_vmmacc_vv_i16m4 (vint16m4_t vd, vint16m1_t vs1, vint16m1_t
vs2, size_t vl);
/* SEW=32, VLEN=256, λ=2: EMUL_C=2. LMUL=1 (default): */
vint32m2_t __riscv_vmmacc_vv_i32m2 (vint32m2_t vd, vint32m1_t vs1, vint32m1_t
vs2, size_t vl);
/* Same EMUL_C=2, LMUL=2 (_lm2 qualifier): */
```

```

vint32m2_t __riscv_vmmacc_vv_i32m2_lm2(vint32m2_t vd, vint32m2_t vs1,
vint32m2_t vs2, size_t vl);
/* SEW=64, VLEN=256, λ=2: EMUL_C=1. LMUL=1 (default, EMUL_C=LMUL): */
vint64m1_t __riscv_vmmacc_vv_i64m1 (vint64m1_t vd, vint64m1_t vs1, vint64m1_t
vs2, size_t vl);
/* Unsigned variants: __riscv_vmmacc_vv_u{sew}m{EMUL_C}[_lm{N}] , accepting
vuint types. */

```

For `vmmacc.vv` ($W=2$), A and B use SEW/2 elements; the accumulator uses SEW. The type-suffix on `vd` determines the accumulator © SEW and `EMUL_C`:

```

/* SEW=16 accum / EEW=8 inputs, VLEN=256, λ=2: EMUL_C=4. LMUL=1 (default): */
vint16m4_t __riscv_vmmacc_vv_i16m4 (vint16m4_t vd, vint8m1_t vs1, vint8m1_t
vs2, size_t vl);
/* SEW=32 accum / EEW=16 inputs, VLEN=256, λ=2: EMUL_C=2. LMUL=1 (default): */
vint32m2_t __riscv_vmmacc_vv_i32m2 (vint32m2_t vd, vint16m1_t vs1,
vint16m1_t vs2, size_t vl);
/* SEW=64 accum / EEW=32 inputs, VLEN=256, λ=2: EMUL_C=1. LMUL=1 (default): */
vint64m1_t __riscv_vmmacc_vv_i64m1 (vint64m1_t vd, vint32m1_t vs1,
vint32m1_t vs2, size_t vl);

```

For `vqmmacc.vv` ($W=4$), A and B use SEW/4 elements:

```

/* SEW=32 accum / EEW=8 inputs, VLEN=256, λ=2: EMUL_C=2. LMUL=1 (default): */
vint32m2_t __riscv_vqmmacc_vv_i32m2 (vint32m2_t vd, vint8m1_t vs1, vint8m1_t
vs2, size_t vl);
/* SEW=64 accum / EEW=16 inputs, VLEN=256, λ=2: EMUL_C=1. LMUL=1 (default): */
vint64m1_t __riscv_vqmmacc_vv_i64m1 (vint64m1_t vd, vint16m1_t vs1, vint16m1_t
vs2, size_t vl);

```

The `vint4m{N}_t` and `vuint4m{N}_t` vector types are implementation-defined extensions for 4-bit integer elements, analogous to the OFP4 vector types for 4-bit floating-point.

For `vmmacc.vv` ($W=2$) with SEW=8, the inputs are 4-bit integers (`Zvvi4i8mm`):

```

/* Zvvi4i8mm: Int4→Int8, SEW=8, VLEN=256, λ=2: EMUL_C=8. LMUL=1 (default): */
vint8m8_t __riscv_vmmacc_vv_i8m8_i4m1 (vint8m8_t vd,
vint4m1_t vs1, vint4m1_t vs2, size_t
vl);

```

For `vqmmacc.vv` ($W=4$) with SEW=16, the inputs are 4-bit integers (`Zvvi4i16mm`):

```

/* Zvvi4i16mm: Int4→Int16, SEW=16, VLEN=256, λ=2: EMUL_C=4. LMUL=1 (default):
*/
vint16m4_t __riscv_vqmmacc_vv_i16m4_i4m1(vint16m4_t vd,
vint4m1_t vs1, vint4m1_t vs2,

```

```
size_t vl);
```

The following extensions require an effective widening ratio of 8 ($W=8$) and are served by the `v8wmmacc.vv` (integer) and `vf8wmmacc.vv` (floating-point) instructions:

- Integer: `Zvvi4i32mm` ($\text{Int4} \rightarrow \text{Int32}$), `Zvvi8i64mm` ($\text{Int8} \rightarrow \text{Int64}$)
- Base FP: `Zvvofp4fp32mm` ($\text{OFP4} \rightarrow \text{FP32}$), `Zvvofp8fp64mm` ($\text{OFP8} \rightarrow \text{FP64}$)
- Microscaling: `Zvvxofp4fp32mm` / `Zvvxnofp4fp32mm` ($\text{MXFP4} \rightarrow \text{FP32}$), `Zvvxofp8fp64mm` / `Zvvxnofp8fp64mm` ($\text{MXFP8} \rightarrow \text{FP64}$), `Zvvxi8fp64mm` / `Zvvxni8fp64mm` ($\text{MXINT8} \rightarrow \text{FP64}$)

The `altfmt_A` and `altfmt_B` fields in `vtype` select the signedness of A and B independently. When mixed-sign multiplication is needed (signed A $\times$ unsigned B, or vice versa), use the `_su` (signed $\times$ unsigned) or `_us` (unsigned $\times$ signed) suffix on the intrinsic name:

```
/* vwmacc.vv signed A  $\times$  unsigned B; SEW=32, VLEN=256,  $\lambda=2$ : EMUL_C=2, LMUL=1 */
vint32m2_t __riscv_vwmacc_vv_i32m2_su(vint32m2_t vd, vint16m1_t vs1,
                                       vuint16m1_t vs2, size_t vl);

/* vwmacc.vv unsigned A  $\times$  signed B; SEW=32, VLEN=256,  $\lambda=2$ : EMUL_C=2, LMUL=1 */
vint32m2_t __riscv_vwmacc_vv_i32m2_us(vint32m2_t vd, vuint16m1_t vs1,
                                       vint16m1_t vs2, size_t vl);
```

The `_su` / `_us` mixed-sign suffix and the `_lm{N}` LMUL suffix combine in the canonical order `_su|_us` followed by `_lm{N}`:

```
/* vmmacc.vv signed A  $\times$  unsigned B with LMUL=2 (EMUL_C=2, LMUL=2): */
vint32m2_t __riscv_vmmacc_vv_i32m2_su_lm2(vint32m2_t vd, vint32m2_t vs1,
                                           vuint32m2_t vs2, size_t vl);

/* vwmacc.vv signed A  $\times$  unsigned B with LMUL=4 (EMUL_C=2, LMUL=4): */
vint32m2_t __riscv_vwmacc_vv_i32m2_su_lm4(vint32m2_t vd, vint16m4_t vs1,
                                           vuint16m4_t vs2, size_t vl);

/* vqmmacc.vv unsigned A  $\times$  signed B with LMUL=8 (EMUL_C=2, LMUL=8): */
vint32m2_t __riscv_vqmmacc_vv_i32m2_us_lm8(vint32m2_t vd, vuint8m8_t vs1,
                                           vint8m8_t vs2, size_t vl);
```

When `EMUL_C = 16`, the accumulator type suffix uses the IME-only `m16` accumulator type. The `_lm{N}` qualifier, when present, still specifies the LMUL of the A and B input operands.

```
/* EMUL_C=16, LMUL=1. */
vint32m16_t __riscv_vmmacc_vv_i32m16(vint32m16_t vd, vint32m1_t vs1,
                                      vint32m1_t vs2, size_t vl);

/* EMUL_C=16, LMUL=2 for A and B. */
vint32m16_t __riscv_vmmacc_vv_i32m16_lm2(vint32m16_t vd, vint32m2_t vs1,
                                           vint32m2_t vs2, size_t vl);
```


36.10.1.8. Floating-point matrix multiply-accumulate (Zvfmfm)

The OFP8 vector types `vfloat8e4m3mX_t` (E4M3) and `vfloat8e5m2mX_t` (E5M2), and the OFP4 vector type `vfloat4e2m1mX_t` (E2M1), are implementation-defined extensions following the V intrinsics naming convention for non-ISO types, analogous to `vbfloat16mX_t` for BFloat16. The corresponding scalar types (`_Float8E4M3`, `_Float8E5M2`, `_Float4E2M1`) are not part of ISO C; once standardised, headers will alias these to the ISO names.

For `vfmfmacc.vv` (no widening, $W=1$), accumulator and input tiles share SEW:

```
/* OFP8 E4M3, VLEN=256,  $\lambda=2$ : EMUL_C=8. LMUL=1 (default): */
vfloat8e4m3m8_t __riscv_vfmfmacc_vv_f8e4m3m8(vfloat8e4m3m8_t vd,
                                              vfloat8e4m3m1_t vs1,
                                              vfloat8e4m3m1_t vs2, size_t vl);
/* OFP8 E5M2, VLEN=256,  $\lambda=2$ : EMUL_C=8. LMUL=1 (default): */
vfloat8e5m2m8_t __riscv_vfmfmacc_vv_f8e5m2m8(vfloat8e5m2m8_t vd,
                                              vfloat8e5m2m1_t vs1,
                                              vfloat8e5m2m1_t vs2, size_t vl);
/* FP16, VLEN=256,  $\lambda=2$ : EMUL_C=4. LMUL=1 (default): */
vfloat16m4_t __riscv_vfmfmacc_vv_f16m4(vfloat16m4_t vd,
                                       vfloat16m1_t vs1, vfloat16m1_t vs2,
                                       size_t vl);
/* BF16, VLEN=256,  $\lambda=2$ : EMUL_C=4. LMUL=1 (default): */
vbfloat16m4_t __riscv_vfmfmacc_vv_bf16m4(vbfloat16m4_t vd,
                                         vbfloat16m1_t vs1, vbfloat16m1_t vs2,
                                         size_t vl);
/* FP32, VLEN=256,  $\lambda=2$ : EMUL_C=2. LMUL=1 (default): */
vfloat32m2_t __riscv_vfmfmacc_vv_f32m2(vfloat32m2_t vd,
                                       vfloat32m1_t vs1, vfloat32m1_t vs2,
                                       size_t vl);
/* FP64, VLEN=256,  $\lambda=2$ : EMUL_C=1. LMUL=1 (default, EMUL_C=LMUL): */
vfloat64m1_t __riscv_vfmfmacc_vv_f64m1(vfloat64m1_t vd,
                                       vfloat64m1_t vs1, vfloat64m1_t vs2,
                                       size_t vl);
```

For `vfwmmacc.vv` ($W=2$), A/B use SEW/2:

```
/* OFP8 E4M3 accum / OFP4 E2M1 inputs, VLEN=256,  $\lambda=2$ : EMUL_C=8. LMUL=1
(default): */
vfloat8e4m3m8_t __riscv_vfwmmacc_vv_f8e4m3m8_f4e2m1m1(
    vfloat8e4m3m8_t vd, vfloat4e2m1m1_t vs1, vfloat4e2m1m1_t vs2, size_t vl);
/* FP16 accum / OFP8 E4M3 inputs, VLEN=256,  $\lambda=2$ : EMUL_C=4. LMUL=1 (default):
*/
vfloat16m4_t __riscv_vfwmmacc_vv_f16m4_f8e4m3m1(vfloat16m4_t vd,
                                                vfloat8e4m3m1_t vs1,
                                                vfloat8e4m3m1_t vs2, size_t vl);
/* BF16 accum / OFP8 E4M3 inputs, VLEN=256,  $\lambda=2$ : EMUL_C=4. LMUL=1 (default):
*/
vbfloat16m4_t __riscv_vfwmmacc_vv_bf16m4_f8e4m3m1(vbfloat16m4_t vd,
```

```

vfloat8e4m3m1_t vs1,
vfloat8e4m3m1_t vs2, size_t vl);
/* FP32 accum / FP16 inputs, VLEN=256, λ=2: EMUL_C=2. LMUL=1 (default): */
vfloat32m2_t __riscv_vfwmmacc_vv_f32m2(vfloat32m2_t vd,
vfloat16m1_t vs1, vfloat16m1_t vs2,
size_t vl);
/* FP64 accum / FP32 inputs, VLEN=256, λ=2: EMUL_C=1. LMUL=1 (default): */
vfloat64m1_t __riscv_vfwmmacc_vv_f64m1(vfloat64m1_t vd,
vfloat32m1_t vs1, vfloat32m1_t vs2,
size_t vl);

```

For `vfqmmacc.vv` (W=4), A/B use SEW/4:

```

/* FP16 accum / OFP4 E2M1 inputs, VLEN=256, λ=2: EMUL_C=4. LMUL=1 (default): */
vfloat16m4_t __riscv_vfqmmacc_vv_f16m4_f4e2m1m1(vfloat16m4_t vd,
vfloat4e2m1m1_t vs1,
vfloat4e2m1m1_t vs2, size_t vl);
/* BF16 accum / OFP4 E2M1 inputs, VLEN=256, λ=2: EMUL_C=4. LMUL=1 (default): */
vbffloat16m4_t __riscv_vfqmmacc_vv_bf16m4_f4e2m1m1(vbffloat16m4_t vd,
vfloat4e2m1m1_t vs1,
vfloat4e2m1m1_t vs2, size_t vl);
/* FP32 accum / OFP8 E4M3 inputs, VLEN=256, λ=2: EMUL_C=2. LMUL=1 (default): */
vfloat32m2_t __riscv_vfqmmacc_vv_f32m2_f8e4m3m1(vfloat32m2_t vd,
vfloat8e4m3m1_t vs1,
vfloat8e4m3m1_t vs2, size_t vl);
/* FP64 accum / BF16 inputs, VLEN=256, λ=2: EMUL_C=1. LMUL=1 (default): */
vfloat64m1_t __riscv_vfqmmacc_vv_f64m1_bf16m1(vfloat64m1_t vd,
vbffloat16m1_t vs1,
vbffloat16m1_t vs2, size_t vl);
/* FP64 accum / FP16 inputs, VLEN=256, λ=2: EMUL_C=1. LMUL=1 (default): */
vfloat64m1_t __riscv_vfqmmacc_vv_f64m1(vfloat64m1_t vd,
vfloat16m1_t vs1, vfloat16m1_t vs2,
size_t vl);

```

The `altfmt_A` and `altfmt_B` fields in `vtype` select the floating-point format of the A and B input tiles. When a specific alternative format is required, the `vs1/vs2` input element-type name is appended to the intrinsic name in place of the default input type; no separate `_alt` suffix is used. The following examples show `altfmt_A=altfmt_B=1` variants for selected combinations:

```

/* vfwmmacc.vv FP32 accum / BF16 inputs (altfmt_A=B=1 for 16-bit inputs): */
vfloat32m2_t __riscv_vfwmmacc_vv_f32m2_bf16m1(vfloat32m2_t vd,
vbffloat16m1_t vs1, vbffloat16m1_t
vs2, size_t vl);
/* vfwmmacc.vv FP16 accum / OFP8 E5M2 inputs (altfmt_A=B=1 for 8-bit inputs): */

```

```

vfloat16m4_t __riscv_vfwmmacc_vv_f16m4_f8e5m2m1(vfloat16m4_t vd,
                                                  vfloat8e5m2m1_t vs1,
vfloat8e5m2m1_t vs2, size_t vl);
/* vfwmmacc.vv FP32 accum / OFP8 E5M2 inputs (altfmt_A=B=1 for 8-bit inputs):
*/
vfloat32m2_t __riscv_vfwmmacc_vv_f32m2_f8e5m2m1(vfloat32m2_t vd,
                                                  vfloat8e5m2m1_t vs1,
vfloat8e5m2m1_t vs2, size_t vl);

```

When `altfmt_A` $\neq$ `altfmt_B`, the two input type suffixes differ:

```

/* Mixed OFP8: E4M3  $\times$  E5M2  $\rightarrow$  FP16 via vfwmmacc.vv (altfmt_A=0, altfmt_B=1): */
vfloat16m4_t __riscv_vfwmmacc_vv_f16m4_f8e4m3m1_f8e5m2m1(vfloat16m4_t vd,
                                                           vfloat8e4m3m1_t
vs1, vfloat8e5m2m1_t vs2, size_t vl);
/* Mixed 16-bit: FP16  $\times$  BF16  $\rightarrow$  FP32 via vfwmmacc.vv (altfmt_A=0, altfmt_B=1):
*/
vfloat32m2_t __riscv_vfwmmacc_vv_f32m2_f16m1_bf16m1(vfloat32m2_t vd,
                                                       vfloat16m1_t vs1,
vbf16m1_t vs2, size_t vl);
/* Mixed OFP8 with BF16 accumulator (altfmt selects BF16 over FP16
   for the C type, plus altfmt_A=0, altfmt_B=1 for the inputs).
   The accumulator type appears once, followed by both input types: */
vbf16m4_t __riscv_vfwmmacc_vv_bf16m4_f8e4m3m1_f8e5m2m1(vbf16m4_t vd,
                                                         vfloat8e4m3m1_t
vs1, vfloat8e5m2m1_t vs2, size_t vl);
/* Mixed OFP8  $\rightarrow$  OFP8 (non-widening vfwmmacc.vv). Both input formats and
   the accumulator format are independent; up to three type tokens: */
vfloat8e4m3m1_t __riscv_vfwmmacc_vv_f8e4m3m1_f8e4m3m1_f8e5m2m1(vfloat8e4m3m1_t
vd,

vfloat8e4m3m1_t vs1, vfloat8e5m2m1_t vs2, size_t vl);
/* Overloaded short forms – compiler resolves from vs1/vs2 types: */
vfloat16m4_t __riscv_vfwmmacc_vv(vfloat16m4_t vd,
                                vfloat8e4m3m1_t vs1, vfloat8e5m2m1_t vs2,
size_t vl) __attribute__((overloadable));
vfloat32m2_t __riscv_vfwmmacc_vv(vfloat32m2_t vd,
                                vfloat16m1_t vs1, vbf16m1_t vs2, size_t
vl) __attribute__((overloadable));
vbf16m4_t __riscv_vfwmmacc_vv(vbf16m4_t vd,
                              vfloat8e4m3m1_t vs1, vfloat8e5m2m1_t vs2,
size_t vl) __attribute__((overloadable));

```

The three-token long-form name encodes the accumulator type first (as the canonical type-suffix), followed by the A and B input-type suffixes in order. When the accumulator is the default for the given instruction and SEW (e.g., FP16 for `vfwmmacc.vv` with 8-bit inputs), the two-token form is used; the three-token form arises only when the accumulator type itself is a non-default selection (e.g., BF16 from `vfwmmacc.vv`).

The integer-input floating-point-accumulate MX instructions (`vwfimmacc.vv`, `vfqimmacc.vv`, and `vf8wimmacc.vv`) exist only in microscaled form. Their C intrinsics therefore always include the paired scale operand `v0` and a `_bs{16|32}` block-size suffix, as described in [Section 36.10.1.9](#). MXINT input operands are always signed; unsigned MXINT intrinsic variants are not defined.

The same rule for `EMUL_C = 16` applies to floating-point accumulator types.

```
/* Non-widening FP32, EMUL_C=16, LMUL=1. */
vfloat32m16_t __riscv_vfmmacc_vv_f32m16(vfloat32m16_t vd, vfloat32m1_t vs1,
vfloat32m1_t vs2, size_t vl);

/* FP16 inputs, FP32 accumulator, EMUL_C=16, LMUL=1. */
vfloat32m16_t __riscv_vfwmacc_vv_f32m16_f16m1(vfloat32m16_t vd, vfloat16m1_t
vs1, vfloat16m1_t vs2, size_t vl);
```

36.10.1.9. Microscaled multiply-accumulate intrinsics

Microscaled (MX) multiply-accumulate variants take an extra `vuint16m1_t v0` argument carrying the paired block-scale values (see [Section 36.7.3](#)). Two qualifiers in the intrinsic name distinguish them from the unscaled forms:

- `_scaled` — applied to the floating-point widening intrinsics (`vwfmmacc`, `vfqmmacc`, `vf8wmmacc`). These instructions exist in both unscaled and scaled forms; the qualifier disambiguates.
- `_bs{N}` ($N \in \{16, 32\}$) — appended to the type-suffix to select the block size. This applies to all microscaled intrinsics, including the integer-input ones (`vwfimmacc`, `vfqimmacc`, `vf8wimmacc`), which exist only in the microscaled form and therefore do not carry `_scaled`.

The full canonical suffix order for microscaled multiply-accumulate intrinsics extends the one given in [C-language intrinsics](#):

```
{type-suffix}[_{inputA}[_{inputB}]][_su|_us][_lm{N}]_bs{N}
```

```
/* Floating-point microscaled (FP × FP × MX scale): _scaled qualifier */

/* FP16 × FP16 → FP32, block size 32 */
vfloat32m2_t __riscv_vfwmacc_scaled_vv_f32m2_f16m1_bs32(
    vfloat32m2_t vd, vfloat16m1_t vs1, vfloat16m1_t vs2,
    vuint16m1_t v0, size_t vl);
/* FP16 × FP16 → FP32, block size 16 */
vfloat32m2_t __riscv_vfwmacc_scaled_vv_f32m2_f16m1_bs16(
    vfloat32m2_t vd, vfloat16m1_t vs1, vfloat16m1_t vs2,
    vuint16m1_t v0, size_t vl);
/* OFP8 E4M3 × OFP8 E4M3 → FP32, block size 32 */
vfloat32m2_t __riscv_vfqmmacc_scaled_vv_f32m2_f8e4m3m1_bs32(
    vfloat32m2_t vd, vfloat8e4m3m1_t vs1, vfloat8e4m3m1_t vs2,
    vuint16m1_t v0, size_t vl);
/* OFP4 E2M1 × OFP4 E2M1 → FP32, block size 32 */
vfloat32m2_t __riscv_vf8wmmacc_scaled_vv_f32m2_f4e2m1m1_bs32(
```

```

vfloat32m2_t vd, vfloat4e2m1m1_t vs1, vfloat4e2m1m1_t vs2,
vuint16m1_t v0, size_t vl);

/* Integer-input microscaled (Int × Int × MX scale → FP):
   no _scaled qualifier – these instructions only exist in MX form. */

/* MXINT8 → FP16, block size 32 */
vfloat16m1_t __riscv_vfwimmacc_vv_f16m1_i8m1_bs32(
    vfloat16m1_t vd, vint8m1_t vs1, vint8m1_t vs2,
    vuint16m1_t v0, size_t vl);
/* MXINT8 → FP32, block size 32 */
vfloat32m2_t __riscv_vfqimmacc_vv_f32m2_i8m1_bs32(
    vfloat32m2_t vd, vint8m1_t vs1, vint8m1_t vs2,
    vuint16m1_t v0, size_t vl);
/* MXINT8 → FP64, block size 32 */
vfloat64m4_t __riscv_vf8wimmacc_vv_f64m4_i8m1_bs32(
    vfloat64m4_t vd, vint8m1_t vs1, vint8m1_t vs2,
    vuint16m1_t v0, size_t vl);

```

The scale-format is implied by the input data type per the encoding maps in [Section 36.7.3](#); software does not select the scale format separately.

36.10.1.10. VLEN-portable code

Because $EMUL_C = VLEN / (SEW \times \lambda^2)$, the accumulator register-group multiplier varies across implementations with different values of VLEN and λ , even for a fixed SEW tied to an output type. The type suffix in a long-form multiply-accumulate intrinsic name directly encodes EMUL_C, whereas in an overloaded name that EMUL_C is implicit. Either way, source code with IME intrinsics is tied to a specific combination of input/output types and value of EMUL_C.



IME m16 types are intended primarily as compiler intrinsic operand types. Passing or returning IME m16 values across externally visible function boundaries requires psABI support. Until such support is specified, portable software should avoid using IME m16 types in externally visible function interfaces, except through implementation-defined compiler conventions.

Although it is possible to write more general assembly code, it is common industry practice to favor coding with compiler intrinsics. The recommended approach for writing portable code with IME intrinsics is to package multiple code paths in the same executable, each optimized for a specific value of EMUL_C. Runtime selection of the appropriate code path is then performed based on the result of `vsetv1` and computations of $EMUL_C = VLEN / (SEW \times \lambda^2)$.

36.10.2. Encoding-map legality overview

The detailed encoding maps appear in [Section 36.13](#). They are normative and determine whether a particular instruction, SEW, W, `altfmt`, `altfmt_A`, `altfmt_B`, `vm`, and `bs` combination is legal.

In the instruction descriptions below:

- ordinary integer `vm=1` encodings are governed by [Table 88](#);

- ordinary floating-point-input `vm=1` encodings are governed by [Table 87](#);
- floating-point-input microscaled `vm=0` encodings are governed by the MX block-size columns of [Table 87](#); and
- integer-input microscaled encodings are governed by [Section 36.13.3](#).

An encoding-map cell marked *reserved*, a cell naming no extension, or a cell naming an extension that is not implemented causes an illegal-instruction exception.

36.11. Instructions (in alphabetical order)

The semantics of each instruction is expressed in a SAIL-like syntax, using the following helper functions that are shared across all instructions in this chapter:

```
val is_pow2 : int -> bool

// Scale an internal partial sum by a block-scale value in the C
// accumulator format. The scale operand is the product (scale_A x
// scale_B) formed by read_block_scales, and is therefore a general
// fmt_C value, not necessarily a single power of two; for a single
// non-NaN E8M0 scale the operation reduces to an exponent adjustment,
// but the pair product may be inexact (or overflow to infinity) where
// the scale-pair magnitude is not representable in fmt_C. The final
// conversion or accumulation into the C accumulator format remains
// responsible for rounding and exception flags.
val fp_internal_scale : (bits, fp_fmt, fp_internal) -> fp_internal

// Decode a 3-bit lambda encoding.
// Returns None() for 0b000. The caller interprets None() according
// to the context in which the encoding appears.
function decode_vlambda(lam_enc : bits(3)) -> option(int) =
  if lam_enc == 0b000 then None()
  else Some(2 ^ (unsigned(lam_enc) - 1))

// Return true only when the exact lambda value is supported for the
// current VLEN and the supplied SEW.
//
// Lambda support does not depend on LMUL.
val is_ime_lambda_supported : (int, int) -> bool

// Decode vtype.vlmul for an IME instruction.
// IME permits only integer LMUL values 1, 2, 4, and 8.
function decode_ime_lmul() -> int = {
  let LMUL_pow : int = get_lmul_pow();

  if LMUL_pow not in {0, 1, 2, 3} then
    return Illegal_Instruction();

  2 ^ LMUL_pow
}
```

```

// Select the effective lambda for a tile load/store instruction.
//
// A nonzero instruction lambda field overrides vtype.lambda.
// A zero instruction lambda field selects vtype.lambda.
// If neither field supplies a nonzero value, the instruction is illegal.
//
// A nonzero instruction immediate requests its exact value. It is not
// WARL-clamped and it does not modify vtype.lambda.
function decode_tile_lambda(lam_enc : bits(3), SEW : int) -> int = {
  let eff_lambda : int =
    match decode_vlambda(lam_enc) {
      Some(lam) => lam,
      None()    => match decode_vlambda(vtype[lambda]) {
        Some(lam) => lam,
        None()    => return Illegal_Instruction()
      }
    };

  if not (is_ime_lambda_supported(SEW, eff_lambda)) then
    return Illegal_Instruction();

  eff_lambda
}

// Check alignment and architectural-register-file bounds for an
// LMUL-sized vector register group.
function check_ime_reg_group(base : regidx, LMUL : int) -> unit = {
  let base_idx : int = unsigned(base);

  if base_idx % LMUL != 0 then
    return Illegal_Instruction();

  if base_idx + LMUL > 32 then
    return Illegal_Instruction()
}

// Check register-group alignment and bounds for a multiply-accumulate.
// The A and B input operands (vs1, vs2) form LMUL-register groups; the C
// accumulator (vd) forms an EMUL_C-register group. Raises
// illegal-instruction if any group is misaligned or extends past v31.
function check_gemm_reg_groups(g : gemm_geom,
                               vs1 : regidx, vs2 : regidx, vd : regidx) -> unit
= {
  check_ime_reg_group(vs1, g.LMUL);
  check_ime_reg_group(vs2, g.LMUL);
  check_ime_reg_group(vd, g.EMUL_C)
}

```



```

// Common decoded geometry for tile load/store instructions.
struct tile_ls_geom = {
    SEW_pow      : int,
    EEW          : int,
    LMUL         : int,
    lambda       : int,
    linesize     : int,
    elems_per_reg : int,
    num_elem     : int
}

function decode_tile_ls_geometry(lam_enc : bits(3)) -> tile_ls_geom = {
    let SEW_pow      : int = get_sew_pow();
    let EEW          : int = 2 ^ SEW_pow;
    let LMUL         : int = decode_ime_lmul();
    let lambda       : int = decode_tile_lambda(lam_enc, EEW);
    let linesize     : int = lambda * LMUL;
    let elems_per_reg : int = vlen / EEW;
    let num_elem     : int = unsigned(vl);

    if num_elem % linesize != 0 then
        return Illegal_Instruction();

    struct {
        SEW_pow, EEW, LMUL, lambda, linesize, elems_per_reg, num_elem
    }
}

// read_single_element(EEW : int, flat_idx : int, vs : regidx) -> bits(EEW)
//   Read one EEW-wide element from the vector register file at the flat
//   register-group element index flat_idx (as produced by tile_reg_idx /
//   mat*_idx). The flat index addresses element (flat_idx mod epr) of
//   register (vs + flat_idx / epr), where epr = VLEN / EEW is the number
//   of EEW-wide elements per vector register. The access is at width EEW
//   irrespective of vtype.SEW; no numeric conversion is performed.

// write_single_element(EEW : int, flat_idx : int, vd : regidx, value :
// bits(EEW)) -> unit
//   Write one EEW-wide element to the vector register file at the flat
//   register-group element index flat_idx, using the same flat-index ->
//   (register, in-register element) mapping as read_single_element.

// Map sequential tile element index i to the flat register-group element
// index.
// linesize = lambda * LMUL; elements within one tile row span LMUL registers.
function tile_reg_idx(i : int, LMUL : int, lambda : int, elems_per_reg : int)
-> int = {

```

```

let linesize      : int = lambda * LMUL;
let line          : int = i / linesize;
let elem_in_line  : int = i % linesize;
let regoff        : int = elem_in_line / lambda;
let elementoff    : int = line * lambda + elem_in_line % lambda;
    regoff * elems_per_reg + elementoff
}

// A[i, k] in vs1: A is stored row-major (rows are the outer dimension).
function mat_A_idx(i : int, k : int, K_eff : int,
                  LMUL : int, lambda : int, epr : int) -> int =
    tile_reg_idx(i * K_eff + k, LMUL, lambda, epr)

// B_tile^T[k, j] in vs2: B_tile^T has shape K_eff x N and is
// stored column-major. Column j contributes to output column j.
// Equivalently, this element is B_tile[j, k] in the row-major
// N x K_eff view of the same register contents.
function mat_B_idx(k : int, j : int, K_eff : int,
                  LMUL : int, lambda : int, epr : int) -> int =
    tile_reg_idx(j * K_eff + k, LMUL, lambda, epr)

// C[i, j] in vd: C is stored in a fixed physical row-major tile.
// The physical row stride is N_max (= M), not the active column count N.
// EMUL_C = VLEN / (SEW * lambda^2) is the number of registers in the C group.
function mat_C_idx(i : int, j : int, N_max : int,
                  EMUL_C : int, lambda : int, epr : int) -> int =
    tile_reg_idx(i * N_max + j, EMUL_C, lambda, epr)

// fp_format_of(EEW : int, alt : bit) -> fp_fmt
//   Decode (element width in bits, altfmt flag) to the concrete FP format as
//   specified
//   by the altfmt encoding tables in the vtype section (e.g., EEW=8, alt=0 →
//   E4M3 (OFP8)).

// fp_add(acc : bits(EEW), prod : bits(EEW),
//        EEW : int,      fmt : fp_fmt,
//        rm  : rounding_mode) -> bits(EEW)
//   Add prod to acc, both in format fmt at width EEW, rounded per rm. Updates
//   fflags.

// fp_mul(a : bits(EEW), b : bits(EEW),
//        EEW : int,      fmt : fp_fmt,
//        rm  : rounding_mode) -> bits(EEW)
//   Multiply a by b, both in format fmt at width EEW, rounded per rm. Updates
//   fflags.

// fp_one(EEW : int, fmt : fp_fmt) -> bits(EEW)
//   Return the bit pattern for +1.0 in the given format.

// fp_is_NaN(x : bits(EEW), EEW : int, fmt : fp_fmt) -> bool

```

```

// Return true if x is a NaN in the given format.

fp_defaultNaN(EEW : int, fmt : fp_fmt) -> bits(EEW)
// Return the default (canonical) NaN bit pattern for the given format.
// Every shared helper that materializes a NaN result in a concrete
// floating-point format returns this bit pattern.

// fp_zero(EEW : int, fmt : fp_fmt) -> bits(EEW)
// Return the positive zero bit pattern for the given format.

// scale_width_of(sfmt : scale_format) -> int
// Return the width in bits of a single scale value for the given format.
// E8M0 → 8. Future formats may return 16.
function scale_width_of(sfmt : scale_format) -> int =
  match sfmt {
    E8M0 => 8
  }

decode_scale(scale : bits(sw), scale_fmt : scale_format,
             EEW : int, fmt : fp_fmt) -> bits(EEW)
// Decode a block-scale value of width sw = scale_width_of(scale_fmt)
// in the given scale format to the corresponding value in the target
// FP format. The scale format is determined by the input data type.
// This helper updates fflags for exceptions raised by conversion to
// the target FP format.
// For E8M0: converts an 8-bit exponent-only value (bias 127) to a
// power-of-two; the bit pattern 0xFF maps to NaN.

int_to_fp(x : int, EEW : int, fmt : fp_fmt, rm : rounding_mode) -> bits(EEW)
// Convert a mathematical integer x to the nearest representable value in
// format fmt at width EEW, rounded per rm. Updates fflags.

// — Shared GEMM helpers —————

// Decode tile geometry from vtype for widening factor W.
// Returns a record with all derived tile dimensions.
struct gemm_geom = {
  EEW_C : int, // accumulator element width (= SEW)
  EEW_A : int, // input element width (= SEW / W)
  W : int, // widening / packing factor
  LMUL : int, // register-group multiplier
  lambda : int, // tile-layout parameter from vtype
  K_eff : int, // effective K dimension (= lambda * W * LMUL)
  M : int, // rows of A / C tile
  N : int, // active columns of B_tile^T and C (set via VL)
  N_max : int, // physical columns of the C tile (= M)
  EMUL_C : int, // number of registers in C group
  epr_A : int, // elements per register at EEW_A
  epr_C : int, // elements per register at EEW_C
}

```

```

function decode_gemm_geometry(W : int) -> gemm_geom = {
  let SEW_pow  : int = get_sew_pow();
  let EEW_C    : int = 2 ^ SEW_pow;
  let EEW_A    : int = EEW_C / W;
  let LMUL     : int = decode_ime_lmulo();
  let epr_A    : int = vlen / EEW_A;
  let epr_C    : int = vlen / EEW_C;

  let lambda : int =
    match decode_vlambda(vtype[lambda]) {
      Some(lam) => lam,
      None()    => return Illegal_Instruction()
    };
  if not (is_ime_lambda_supported(EEW_C, lambda)) then
    return Illegal_Instruction();

  let K_eff : int = lambda * W * LMUL;

  // VL divisibility: VL must be a multiple of lambda * LMUL.
  // N_tile is the number of active columns of B_tile^T and C.
  // Assumes VL is set via vsetvli at vtype.SEW = C element width.
  let vl_div : int = lambda * LMUL;
  let N      : int = unsigned(vl) / vl_div;
  let M      : int = (LMUL * vlen / EEW_A) / K_eff;
  let N_max  : int = M;
  let EMUL_C : int = M / lambda;

  if unsigned(vl) % vl_div != 0 then return Illegal_Instruction();
  if N > N_max then return Illegal_Instruction();
  if EMUL_C not in {1,2,4,8,16} then return Illegal_Instruction();

  struct { EEW_C, EEW_A, W, LMUL, lambda, K_eff, M, N, N_max, EMUL_C, epr_A,
epr_C }
}

// Partial-sum formation mode for floating-point group reduction.
enum psm_mode = { PSM_EXACT, PSM_DISCLOSED }

// Abstract internal floating-point representation used for the group
// partial sum S before any rounding to the C accumulator format.
type fp_internal

// Exact internal product of two FP operands.
// No rounding to the C accumulator format is performed here.
// Finite products are represented exactly in fp_internal.
// Special values are handled according to the operand formats.
// This helper updates fflags for exceptional special-value cases,
// such as signaling NaN operands or zero multiplied by infinity.
val fp_mul_exact : forall ('n, 'm).

```

```

(bits('n), fp_fmt, bits('m), fp_fmt) -> fp_internal

// Zero value of the abstract internal FP representation.
val fp_internal_zero : unit -> fp_internal

// Exact internal addition of two abstract internal FP values.
// Finite sums are represented exactly in fp_internal.
// Special values are handled according to the participating FP values.
// This helper updates fflags for exceptional special-value cases,
// such as adding infinities of opposite signs.
val fp_internal_add : (fp_internal, fp_internal) -> fp_internal

// Non-exact bulk reduction (psm=1). The implementation chooses the
// reduction algorithm and discloses it; common choices include RVBNA,
// other bulk-normalised fixed-point schemes, and Kulisch-style
// accumulators.
//
// The entire group of exact products is presented at once, as a list,
// rather than folded pairwise. A pairwise fold cannot express a bulk
// normalisation -- such as RVBNA, the flagship psm=1 algorithm -- which
// aligns and sums all g_len x W terms together with a single terminal
// normalisation. fmt_A and fmt_B give the input formats, and fmt_C gives the C
// accumulator format. These formats provide the significand and exponent
// widths, and any other format-dependent parameters, that the disclosed
// algorithm may need. g_len and W give the term count (g_len x W).
//
// The reduction is defined
// to be independent of the order in which the products appear in the
// list. The returned value has not yet been rounded to the C accumulator
// format; any terminal rounding (e.g. RVBNA's round-to-odd) is applied by
// the separate rnd step.
//
// The SAIL leaves the operation abstract here: two implementations
// producing different numerical results or different accrued fflags may
// both be conforming, provided each discloses its algorithm, parameters,
// and exception-flag behavior.
val fp_disclosed_reduce_group : (list(fp_internal), fp_fmt, fp_fmt, fp_fmt,
int, int) -> fp_internal

// Group partial sum S for one output element.
// The group lies within one LMUL=1 step, identified by step, and begins
// at sub-dot-product index g0 within that step. Each sub-dot-product
// contributes W scalar products, so the actual group corresponds to a dot
// product of two vectors with g_len x W elements.
//
// For unscaled FP GEMM, callers pass g_len = G. For microscaled FP GEMM,
// callers may pass any g_len with 1 <= g_len <= G, because groups are not
// permitted to cross microscaling block boundaries.
//

```

```

// The returned value S has not yet been rounded to the C accumulator format;
// any such rounding is handled separately according to rnd.
function fp_group_sum(i : int, j : int,
                     step : int, g0 : int, g_len : int,
                     g : gemm_geom,
                     fmt_A : fp_fmt, fmt_B : fp_fmt, fmt_C : fp_fmt,
                     psm : psm_mode,
                     vs1 : regidx, vs2 : regidx) -> fp_internal = {
  let k_base : int = step * g.lambda * g.W;
  let k_lo   : int = k_base + g0 * g.W;
  let k_hi   : int = k_base + (g0 + g_len) * g.W - 1;

  match psm {
    PSM_EXACT => {
      var S : fp_internal = fp_internal_zero();
      foreach (k from k_lo to k_hi) {
        let a_flat : int = mat_A_idx(i, k, g.K_eff, g.LMUL, g.lambda, g.epr_A);
        let b_flat : int = mat_B_idx(k, j, g.K_eff, g.LMUL, g.lambda, g.epr_A);

        let a_bits : bits(g.EEW_A) = read_single_element(g.EEW_A, a_flat, vs1);
        let b_bits : bits(g.EEW_A) = read_single_element(g.EEW_A, b_flat, vs2);

        let prod : fp_internal = fp_mul_exact(a_bits, fmt_A, b_bits, fmt_B);
        S = fp_internal_add(S, prod)
      };
      S
    },

    PSM_DISCLOSED => {
      // Gather the group's exact products, then reduce them in a single
      // bulk operation so that a disclosed bulk-normalisation algorithm
      // (e.g. RVBNA) can normalise all terms together. The reduction is
      // order-independent, so the list order is unspecified.
      var prods : list(fp_internal) = [];
      foreach (k from k_lo to k_hi) {
        let a_flat : int = mat_A_idx(i, k, g.K_eff, g.LMUL, g.lambda, g.epr_A);
        let b_flat : int = mat_B_idx(k, j, g.K_eff, g.LMUL, g.lambda, g.epr_A);

        let a_bits : bits(g.EEW_A) = read_single_element(g.EEW_A, a_flat, vs1);
        let b_bits : bits(g.EEW_A) = read_single_element(g.EEW_A, b_flat, vs2);

        let prod : fp_internal = fp_mul_exact(a_bits, fmt_A, b_bits, fmt_B);
        prods = prod :: prods
      };
      fp_disclosed_reduce_group(prods, fmt_A, fmt_B, fmt_C, g_len, g.W)
    }
  }
}

// Treatment of the group partial sum S before final accumulation into C.

```

```

enum rnd_mode = { RND_FRM, RND_RTO, RND_XCT }

// Abstract implementation-defined parameters for FP group semantics.
// These correspond to the disclosed mapping from (SEW, W, lambda)
// to (G, psm, rnd).
val get_fp_grouping : (int, int, int) -> int
val get_fp_psm      : (int, int, int) -> psm_mode
val get_fp_rnd      : (int, int, int) -> rnd_mode

// Round an internal FP value to the C accumulator format using frm.
// Updates fflags according to the conversion to the C accumulator format.
val fp_round_to_frm : (fp_internal, int, fp_fmt, rounding_mode) -> bits

// Round an internal FP value to the C accumulator format using round-to-odd.
// Updates fflags according to the conversion to the C accumulator format
// under the specified round-to-odd rule.
val fp_round_to_rto : (fp_internal, int, fp_fmt) -> bits

// Add an internal FP value directly to an accumulator-format operand,
// with the only rounding step being the final rounding according to frm.
// Updates fflags according to the final addition and rounding.
val fp_add_internal : (bits, fp_internal, int, fp_fmt, rounding_mode) -> bits

// Apply the implementation-selected treatment of the group partial sum S
// before the final accumulation into C.
//
// Returns Some(bits(...)) when S is rounded to the C accumulator format
// before accumulation, and None() when S is retained in internal form
// (rnd = RND_XCT).
function round_group_sum(S : fp_internal,
                        rnd : rnd_mode,
                        EEW_C : int, fmt_C : fp_fmt,
                        rm : rounding_mode) -> option(bits(EEW_C)) =

  match rnd {
    RND_FRM => Some(fp_round_to_frm(S, EEW_C, fmt_C, rm)),
    RND_RTO => Some(fp_round_to_rto(S, EEW_C, fmt_C)),
    RND_XCT => None()
  }

// Read and unpack paired block scales from v0 for block index s.
// The scale format (e.g., E8M0) is determined by the input data type
// and passed as scale_fmt. The per-scale width (sw) and pair width
// (pw = 2 × sw) are derived from the scale format.
// When vm=1 (no microscaling), returns (1.0, false).
// When vm=0, reads a pw-bit scale-pair element from v0, decodes both
// scale values into the C accumulator format, forms their product with
// fp_mul, and returns (scale_A × scale_B, is_nan). This operation follows
// the accumulator-format floating-point rules and may raise exception flags.
function read_block_scales(vm : bit, i : int, j : int, s : int,
                          R : int, EEW_C : int, fmt_C : fp_fmt,

```

```

        scale_fmt : scale_format,
        rm : rounding_mode) -> (bits(EEW_C), bool) = {
let sw : int = scale_width_of(scale_fmt);
let pw : int = 2 * sw;

let pair_A : bits(pw) =
    if vm == 0
    then read_single_element(pw, i * R + s, zvmreg(0))
    else zeros(pw);
let pair_B : bits(pw) =
    if vm == 0
    then read_single_element(pw, j * R + s, zvmreg(0))
    else zeros(pw);

let sA : bits(EEW_C) =
    if vm == 0
    then decode_scale(pair_A[sw-1..0], scale_fmt, EEW_C, fmt_C)
    else fp_one(EEW_C, fmt_C);
let sB : bits(EEW_C) =
    if vm == 0
    then decode_scale(pair_B[2*sw-1..sw], scale_fmt, EEW_C, fmt_C)
    else fp_one(EEW_C, fmt_C);

let blk_scale : bits(EEW_C) = fp_mul(sA, sB, EEW_C, fmt_C, rm);
(blk_scale, fp_is_NaN(blk_scale, EEW_C, fmt_C))
}

// Exact signed integer inner product over elements [k_lo, k_hi].
// Returns an unbounded mathematical integer (no overflow).
function int_block_dot(i : int, j : int, k_lo : int, k_hi : int,
    g : gemm_geom,
    signed_A : bool, signed_B : bool,
    vs1 : regidx, vs2 : regidx) -> int = {
var int_sum : int = 0;
foreach (k from k_lo to k_hi) {
    let a_flat : int = mat_A_idx(i, k, g.K_eff, g.LMUL, g.lambda, g.epr_A);
    let b_flat : int = mat_B_idx(k, j, g.K_eff, g.LMUL, g.lambda, g.epr_A);
    let a_val : int = if signed_A
        then signed(read_single_element(g.EEW_A, a_flat, vs1))
        else unsigned(read_single_element(g.EEW_A, a_flat,
vs1));
    let b_val : int = if signed_B
        then signed(read_single_element(g.EEW_A, b_flat, vs2))
        else unsigned(read_single_element(g.EEW_A, b_flat,
vs2));
    int_sum = int_sum + a_val * b_val
};
int_sum
}

```



```

// Check microscaling legality constraints.
// Raises Illegal_Instruction if  $SEW \times \lambda < pw$  (pair width) or
// (for BS=16)  $W \times LMUL > SEW$ . The pair width is derived from the
// scale format:  $pw = 2 \times scale\_width\_of(scale\_fmt)$ .
function check_microscaling_legality(W : int, LMUL : int,
                                     EEW_C : int, lambda : int,
                                     scale_fmt : scale_format) -> unit = {
    let pw : int = 2 * scale_width_of(scale_fmt);
    if EEW_C * lambda < pw then return Illegal_Instruction();
    if vtype[bs] == 0b1 & (W * LMUL > EEW_C) then return Illegal_Instruction()
}

// Enforce the Extension column of tbl-int-encoding-map for an
// ordinary integer matrix multiply-accumulate instruction.
//
// The argument order is:
//   W, SEW, altfmt_A, altfmt_B
//
// Raises Illegal_Instruction when the selected row is reserved or
// the extension named by that row is not implemented.
val check_int_encoding_legality :
    (int, int, bit, bit) -> unit

// Enforce the Extension column of tbl-fp-encoding-map for an
// ordinary floating-point-input matrix multiply-accumulate instruction.
//
// The argument order is:
//   W, SEW, altfmt_A, altfmt_B, altfmt
//
// Raises Illegal_Instruction when the selected row is reserved or
// any extension named by that row is not implemented. A row that
// names more than one extension requires all named extensions.
//
// Fields identified as ignored by the table do not participate in
// row selection.
val check_fp_encoding_legality :
    (int, int, bit, bit, bit) -> unit

// Enforce the vm=0 columns of tbl-fp-encoding-map for a
// floating-point-input microscaled instruction.
//
// The argument order is:
//   W, SEW, altfmt_A, altfmt_B, altfmt, bs
//
// Raises Illegal_Instruction when the selected MX cell is reserved,
// contains no extension, or names one or more required extensions that
// are not implemented.
val check_fp_mx_encoding_legality :
    (int, int, bit, bit, bit, bit) -> unit

```

```

// Enforce tbl-intmx-encoding-map for an integer-input microscaled
// floating-point-accumulate instruction.
//
// The argument order is:
//   W, SEW, altfmt_A, altfmt_B, altfmt, bs
//
// Raises Illegal_Instruction when the selected MX cell is reserved
// or names an extension that is not implemented.
val check_int_mx_encoding_legality :
  (int, int, bit, bit, bit, bit) -> unit

// — Top-level GEMM operations —————
//
// Each instruction decodes its parameters and dispatches to one of the
// following four operations.

// Integer GEMM: C (integer) += A (integer) × B (integer), no scaling.
function int_gemm(g : gemm_geom, signed_A : bool, signed_B : bool,
  vs1 : regidx, vs2 : regidx, vd : regidx) -> unit = {
  check_gemm_reg_groups(g, vs1, vs2, vd);
  foreach (j from 0 to (g.N - 1)) {
    foreach (i from 0 to (g.M - 1)) {
      let c_flat : int = mat_C_idx(i, j, g.N_max, g.EMUL_C, g.lambda, g.epr_C);
      let c_bits : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
      var acc : int = signed(c_bits);
      acc = acc + int_block_dot(i, j, 0, g.K_eff - 1, g,
        signed_A, signed_B, vs1, vs2);
      write_single_element(g.EEW_C, c_flat, vd, to_bits_unsafe(g.EEW_C, acc))
    }
  }
}

// FP GEMM: C (FP) += A (FP) × B (FP), no scaling.
function fp_gemm(g : gemm_geom,
  fmt_A : fp_fmt, fmt_B : fp_fmt, fmt_C : fp_fmt,
  rm : rounding_mode,
  vs1 : regidx, vs2 : regidx, vd : regidx) -> unit = {
  check_gemm_reg_groups(g, vs1, vs2, vd);
  let G : int = get_fp_grouping(g.EEW_C, g.W, g.lambda);
  let psm : psm_mode = get_fp_psm(g.EEW_C, g.W, g.lambda);
  let rnd : rnd_mode = get_fp_rnd(g.EEW_C, g.W, g.lambda);

  if G < 1 | G > g.lambda | not(is_pow2(G)) then
    return Illegal_Instruction();

  foreach (j from 0 to (g.N - 1)) {
    foreach (i from 0 to (g.M - 1)) {
      let c_flat : int = mat_C_idx(i, j, g.N_max, g.EMUL_C, g.lambda, g

```

```

.epr_C);
    var acc : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);

    // The semantics for LMUL > 1 are defined as repeated application
    // of the LMUL = 1 computation, in increasing register-index order.
    foreach (step from 0 to (g.LMUL - 1)) {

        // Each LMUL = 1 step contains lambda sub-dot-products.
        foreach (g0 from 0 to (g.lambda - 1) by G) {
            let S : fp_internal =
                fp_group_sum(i, j, step, g0, G,
                            g, fmt_A, fmt_B, fmt_C, psm,
                            vs1, vs2);

            match round_group_sum(S, rnd, g.EEW_C, fmt_C, rm) {
                Some(S_bits) =>
                    acc = fp_add(acc, S_bits, g.EEW_C, fmt_C, rm),
                None() =>
                    acc = fp_add_internal(acc, S, g.EEW_C, fmt_C, rm)
            }
        }
    };

    write_single_element(g.EEW_C, c_flat, vd, acc)
}
}

// FP scaled GEMM: C (FP) += scale_A × scale_B × (A (FP) × B (FP)),
// with block scales from v0.

function fp_scaled_gemm(g : gemm_geom,
                        fmt_A : fp_fmt, fmt_B : fp_fmt, fmt_C : fp_fmt,
                        scale_fmt : scale_format,
                        rm : rounding_mode,
                        vs1 : regidx, vs2 : regidx, vd : regidx) -> unit = {
    check_gemm_reg_groups(g, vs1, vs2, vd);
    let pw : int = 2 * scale_width_of(scale_fmt);
    let block_size : int = if vtype[bs] == 0b1 then 16 else 32;
    let S_blocks : int = (g.K_eff + block_size - 1) / block_size;
    let R : int = g.lambda * g.EEW_C / pw;

    let G : int = get_fp_grouping(g.EEW_C, g.W, g.lambda);
    let psm : psm_mode = get_fp_psm(g.EEW_C, g.W, g.lambda);
    let rnd : rnd_mode = get_fp_rnd(g.EEW_C, g.W, g.lambda);

    if G < 1 | G > g.lambda | not(is_pow2(G)) then
        return Illegal_Instruction();

    foreach (j from 0 to (g.N - 1)) {

```

```

foreach (i from 0 to (g.M - 1)) {
    let c_flat : int          = mat_C_idx(i, j, g.N_max, g.EMUL_C, g.lambda, g
.epr_C);
    var acc : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
    var nan_out : bool = false;

    foreach (s from 0 to (S_blocks - 1)) {
        let (blk_scale, is_nan) =
            read_block_scales(0b0, i, j, s, R, g.EEW_C, fmt_C, scale_fmt, rm);
        // is_nan is keyed on the combined block scale (scale_A x scale_B),
        // not on either decoded scale alone. A NaN combined scale forces
        // this output element to the default NaN and terminates its
        // evaluation: the scale decode/multiply above has already accrued
        // its flags, but this block's products and accumulation, and all
        // later blocks of this element, are not evaluated and raise no
        // further flags. See the fflags rules for forced-NaN elements.
        if is_nan then { nan_out = true; break };

        // Scalar k-range covered by this scale block.
        let blk_k_lo : int = s * block_size;
        let blk_k_hi : int = min(blk_k_lo + block_size, g.K_eff) - 1;

        // Process the block as repeated LMUL=1 steps, in order.
        foreach (step from 0 to (g.LMUL - 1)) {
            let step_k_lo : int = step * g.lambda * g.W;
            let step_k_hi : int = step_k_lo + g.lambda * g.W - 1;

            // Intersect this LMUL=1 step with the current scale block.
            let int_k_lo : int = max(step_k_lo, blk_k_lo);
            let int_k_hi : int = min(step_k_hi, blk_k_hi);

            if int_k_lo <= int_k_hi then {
                // Convert the intersected scalar-k interval back into a
                // sub-dot-product interval relative to this LMUL=1 step.
                let subdot_lo : int = (int_k_lo - step_k_lo) / g.W;
                let subdot_hi : int = (int_k_hi - step_k_lo) / g.W;

                foreach (g0 from subdot_lo to subdot_hi by G) {
                    // Groups are not permitted to cross either an LMUL=1 step
                    // or a microscaling block boundary. Therefore the final group in
                    // intersected range may contain fewer than G sub-dot-products.
                    // The actual group length g_len is part of the architectural
                    // fp_group_sum and to any disclosed psm=1 reduction.
                    let g_len : int = min(G, subdot_hi - g0 + 1);

                    let S : fp_internal =
                        fp_group_sum(i, j, step, g0, g_len,

```

```

        g, fmt_A, fmt_B, fmt_C, psm,
        vs1, vs2);

match round_group_sum(S, rnd, g.EEW_C, fmt_C, rm) {
  Some(S_bits) => {
    let scaled_S : bits(g.EEW_C) =
      fp_mul(blk_scale, S_bits, g.EEW_C, fmt_C, rm);
    acc = fp_add(acc, scaled_S, g.EEW_C, fmt_C, rm)
  },
  None() => {
    let scaled_S : fp_internal =
      fp_internal_scale(blk_scale, fmt_C, S);
    acc = fp_add_internal(acc, scaled_S, g.EEW_C, fmt_C, rm)
  }
}
}
}
}
};

if nan_out then
  write_single_element(g.EEW_C, c_flat, vd, fp_defaultNaN(g.EEW_C,
fmt_C))
else
  write_single_element(g.EEW_C, c_flat, vd, acc)
}
}
}

// Integer-input scaled GEMM: C (FP) += scale_A × scale_B × (A (int) × B
(int)),
// with block scales from v0. Integer products are exact; the sum is converted
// to FP before scaling and accumulation.
function int_scaled_gemm(g : gemm_geom,
                        fmt_C : fp_fmt, scale_fmt : scale_format,
                        rm : rounding_mode,
                        vs1 : regidx, vs2 : regidx, vd : regidx) -> unit = {
  check_gemm_reg_groups(g, vs1, vs2, vd);
  let pw : int = 2 * scale_width_of(scale_fmt);
  let block_size : int = if vtype[bs] == 0b1 then 16 else 32;
  let S : int = (g.K_eff + block_size - 1) / block_size;
  let R : int = g.lambda * g.EEW_C / pw;

  foreach (j from 0 to (g.N - 1)) {
    foreach (i from 0 to (g.M - 1)) {
      let c_flat : int = mat_C_idx(i, j, g.N_max, g.EMUL_C, g.lambda, g
.epr_C);
      var acc : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
      var nan_out : bool = false;

```

```

foreach (s from 0 to (S - 1)) {
    let (blk_scale, is_nan) = read_block_scales(0b0, i, j, s, R,
                                                g.EEW_C, fmt_C, scale_fmt,
rm);
    if is_nan then { nan_out = true; break };

    // The final microscaling block may contain fewer than block_size
scalar elements along the K axis.
    let k_lo : int = s * block_size;
    let k_hi : int = min(k_lo + block_size, g.K_eff) - 1;
    let dot : int = int_block_dot(i, j, k_lo, k_hi, g, true, true, vs1,
vs2);
    let fp_sum : bits(g.EEW_C) = int_to_fp(dot, g.EEW_C, fmt_C, rm);
    acc = fp_add(acc, fp_mul(blk_scale, fp_sum, g.EEW_C, fmt_C, rm),
                g.EEW_C, fmt_C, rm)
};

if nan_out then
    write_single_element(g.EEW_C, c_flat, vd, fp_defaultNaN(g.EEW_C,
fmt_C))
else
    write_single_element(g.EEW_C, c_flat, vd, acc)
}
}
}

```

36.11.1. v8wmmacc.vv

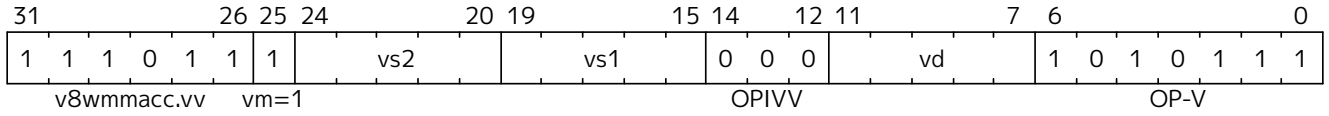
Synopsis

Integer $8 \times$ Widening Matrix Multiply-Accumulate

Mnemonic

v8wmmacc.vv *vd*, *vs1*, *vs2*

Encoding



Description

Interpreting *vs1* as A_{tile} and *vs2* as B_{tile}^T , the instruction computes

$$T = A_{\text{tile}} \times B_{\text{tile}}^T$$

and accumulates T into *vd*: $C \leftarrow C + T$, with a widening factor of 8 applied to the K dimension.

+ *vs1* is interpreted as the $M \times K_{\text{eff}}$ tile A_{tile} in row-major register layout, and *vs2* is interpreted as the $K_{\text{eff}} \times N$ tile B_{tile}^T in column-major register layout, and *vd* as an $M \times M$ physical accumulator tile (row-major register layout), of which only the first N columns are active. *vs1* and *vs2* have elements of width $\text{SEW} \div 8$; *vd* has elements of width SEW.

+ $K_{\text{effective}} = \lambda \times 8 \times \text{LMUL}$. VL selects how many columns of B_{tile}^T , and therefore how many columns of C, are active; VL must be a multiple of $\lambda \times \text{LMUL}$. The remaining columns of the physical C tile are C tile tail elements. The signedness of each input operand is selected by **altfmt_A** and **altfmt_B** in **vtype**: 0 = signed, 1 = unsigned. Accumulation is modular (wraps at 2^{SEW}).

+ Only $\text{SEW} \in \{32, 64\}$ is valid (giving $\text{EEW} = 4$ or 8); $\text{SEW} \in \{8, 16\}$ is *reserved*.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- **vstart** $\neq 0$.
- $\text{SEW} < 32$ ($\text{EEW} = \text{SEW} \div 8$ would be sub-nibble).
- VL is not a multiple of $\lambda \times \text{LMUL}$.
- $\text{EMUL_C} (= \text{VLEN} \div (\text{SEW} \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for EMUL_C.
- When **vm=1**: the selected row in [Table 88](#) is marked *reserved*, or the extension named by that row is not implemented.

Operation

```

if vstart != 0 then return Illegal_Instruction();

if (2 ^ get_sew_pow()) < 32 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(8);
  
```

```
check_int_encoding_legality(8, g.EEW_C, vtype[altfmt_A], vtype[altfmt_B]);

let signed_A : bool = vtype[altfmt_A] == 0b0;
let signed_B : bool = vtype[altfmt_B] == 0b0;

int_gemm(g, signed_A, signed_B, vs1, vs2, vd);
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvmm (Matrix-multiplication instructions (integer))	0.1	Draft

36.11.2. vf8wmmacc.vv

Synopsis

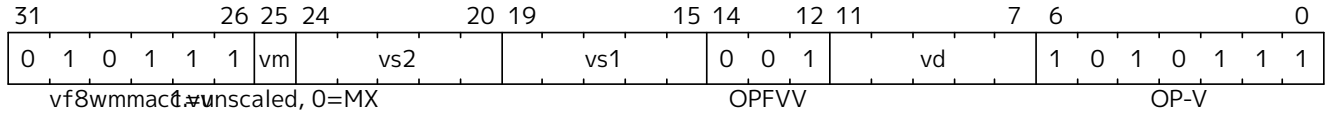
Floating-Point 8× Widening Matrix Multiply-Accumulate

Mnemonic

vf8wmmacc.vv *vd*, *vs1*, *vs2*

vf8wmmacc.vv *vd*, *vs1*, *vs2*, *v0.scale*

Encoding



Description

Interpreting *vs1* as A_{tile} and *vs2* as B_{tile}^T , the instruction computes

$$T = A_{\text{tile}} \times B_{\text{tile}}^T$$

and accumulates T into *vd*: $C \leftarrow C + T$, with a widening factor of 8 applied to the K dimension.

+ *vs1* is interpreted as the $M \times K_{\text{eff}}$ tile A_{tile} in row-major register layout, and *vs2* is interpreted as the $K_{\text{eff}} \times N$ tile B_{tile}^T in column-major register layout, and *vd* as an $M \times M$ physical accumulator tile (row-major register layout), of which only the first N columns are active. *vs1* and *vs2* have elements of width $\text{SEW} \div 8$; *vd* has elements of width SEW.

+ $K_{\text{effective}} = \lambda \times 8 \times \text{LMUL}$. VL selects how many columns of B_{tile}^T , and therefore how many columns of C, are active; VL must be a multiple of $\lambda \times \text{LMUL}$. The remaining columns of the physical C tile are C tile tail elements. The floating-point format for A and B elements is selected independently by **altfmt_A** and **altfmt_B**, interpreted for input element width $\text{SEW} \div 8$; the C format is selected by **altfmt**, interpreted for width SEW. Final accumulation into the C accumulator format uses the dynamic rounding mode from **frm**. Any intermediate grouping, partial-sum formation, and optional rounding of partial sums follow [Section 36.7.2](#). Floating-point exception flags accumulate in **fflags**.

+ When **vm=0** (**v0.scale**), the register **v0** supplies paired block-scale factors for microscaling; the scale format is determined by the input data type (see [Section 36.7.3](#)). The **bs** field in **vtype** selects the block size (32 or 16 elements).

+ Only $\text{SEW} \in \{32, 64\}$ is valid (giving $\text{EEW} = 4$ or 8); $\text{SEW} \in \{8, 16\}$ is *reserved*.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- **vstart** $\neq 0$.
- $\text{SEW} < 32$ ($\text{EEW} = \text{SEW} \div 8$ would be sub-nibble).
- VL is not a multiple of $\lambda \times \text{LMUL}$.
- $\text{EMUL_C} (= \text{VLEN} \div (\text{SEW} \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for EMUL_C.
- When **vm=0**: the selected MX cell in [Table 87](#) is – or *reserved*, or any required extension named by that cell is not implemented.

- When $vm=0$: $SEW \times \lambda < 16$.
- When $vm=0$ and $bs=1$: $W \times LMUL > SEW$ (i.e., $8 \times LMUL > SEW$).
- When $vm=0$: vd , $vs1$, or $vs2$ overlaps $v0$.
- When $vm=1$: the selected row in [Table 87](#) is marked *reserved*, or any extension named by that row is not implemented.

Operation

```

if vstart != 0 then return Illegal_Instruction();

if (2 ^ get_sew_pow()) < 32 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(8);

if vm == 0 then {
  check_microscaling_legality(8, g.LMUL, g.EEW_C, g.lambda, E8M0);
  check_fp_mx_encoding_legality(8, g.EEW_C, vtype[altfmt_A], vtype[altfmt_B],
vtype[altfmt], vtype[bs])
} else {
  check_fp_encoding_legality(8, g.EEW_C, vtype[altfmt_A], vtype[altfmt_B],
vtype[altfmt])
};

let fmt_A : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_A]);
let fmt_B : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_B]);
let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm      : rounding_mode = get_fp_rounding_mode();

if vm == 0 then
  fp_scaled_gemm(g, fmt_A, fmt_B, fmt_C, E8M0, rm, vs1, vs2, vd)
else
  fp_gemm(g, fmt_A, fmt_B, fmt_C, rm, vs1, vs2, vd);
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvvpmm (Matrix-multiplication instructions (floating-point))	0.1	Draft
Zvvxofp4fp32mm, Zvvxofp8fp64mm (Section 36.7.3)	0.1	Draft
Zvvxnofp4fp32mm, Zvvxnofp8fp64mm (Section 36.7.3)	0.1	Draft

36.11.3. vfmacc.vv

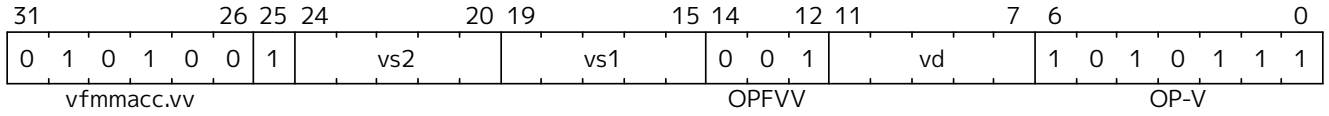
Synopsis

Floating-Point Matrix Multiply-Accumulate

Mnemonic

vfmacc.vv *vd*, *vs1*, *vs2*

Encoding



Description

Interpreting *vs1* as A_{tile} and *vs2* as B_{tile}^T , the instruction computes

$$T = A_{\text{tile}} \times B_{\text{tile}}^T$$

and accumulates T into *vd*: $C \leftarrow C + T$.

+ *vs1* is interpreted as the $M \times K_{\text{eff}}$ tile A_{tile} in row-major register layout, and *vs2* is interpreted as the $K_{\text{eff}} \times N$ tile B_{tile}^T in column-major register layout, and *vd* as an $M \times M$ physical accumulator tile (row-major register layout), of which only the first N columns are active. All three tiles have elements of width SEW.

+ $K_{\text{effective}} = \lambda \times \text{LMUL}$. **VL** selects how many columns of B_{tile}^T , and therefore how many columns of C, are active; VL must be a multiple of $\lambda \times \text{LMUL}$. The remaining columns of the physical C tile are C tile tail elements. The floating-point format for A and B elements is selected independently by **altfmt_A** and **altfmt_B** in **vtype** (see [Section 36.7.1](#)); the C accumulator format is selected by **altfmt**. Final accumulation into the C accumulator format uses the dynamic rounding mode from **frm**. Any intermediate grouping, partial-sum formation, and optional rounding of partial sums follow [Section 36.7.2](#). Floating-point exception flags accumulate in **fflags**.

+ Microscaling (**vm=0**) is not supported for non-widening floating-point multiply-accumulate; **vm=0** is reserved.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- **vstart** $\neq 0$.
- **vm** = 0 (*reserved*).
- VL is not a multiple of $\lambda \times \text{LMUL}$.
- $\text{EMUL_C} (= \text{VLEN} \div (\text{SEW} \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for EMUL_C.
- The selected row in [Table 87](#) is marked *reserved*, or any extension named by that row is not implemented.

Operation

```
if vstart != 0 then return Illegal_Instruction();
```

```
if vm == 0 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(1);
check_fp_encoding_legality(1, g.EEW_C, vtype[altfmt_A], vtype[altfmt_B], vtype
[altfmt]);

let fmt_A : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt_A]);
let fmt_B : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt_B]);
let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm    : rounding_mode = get_fp_rounding_mode();

fp_gemm(g, fmt_A, fmt_B, fmt_C, rm, vs1, vs2, vd);
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvbfmm (Matrix-multiplication instructions (floating-point))	0.1	Draft

36.11.4. vfmacc.vv

Synopsis

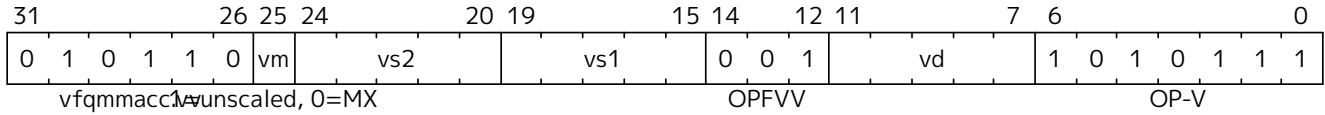
Floating-Point Quad-Widening Matrix Multiply-Accumulate

Mnemonic

vfmacc.vv *vd*, *vs1*, *vs2*

vfmacc.vv *vd*, *vs1*, *vs2*, *v0.scale*

Encoding



Description

Interpreting *vs1* as A_{tile} and *vs2* as B_{tile}^T , the instruction computes

$$T = A_{\text{tile}} \times B_{\text{tile}}^T$$

and accumulates *T* into *vd*: $C \leftarrow C + T$, with a widening factor of 4 applied to the *K* dimension.

+ *vs1* is interpreted as the $M \times K_{\text{eff}}$ tile A_{tile} in row-major register layout, and *vs2* is interpreted as the $K_{\text{eff}} \times N$ tile B_{tile}^T in column-major register layout, and *vd* as an $M \times M$ physical accumulator tile (row-major register layout), of which only the first *N* columns are active. *vs1* and *vs2* have elements of width $\text{SEW} \div 4$; *vd* has elements of width SEW .

+ $K_{\text{effective}} = \lambda \times 4 \times \text{LMUL}$. *VL* selects how many columns of B_{tile}^T , and therefore how many columns of *C*, are active; *VL* must be a multiple of $\lambda \times \text{LMUL}$. The remaining columns of the physical *C* tile are *C* tile tail elements. The floating-point format for *A* and *B* elements is selected independently by *altfmt_A* and *altfmt_B*, interpreted for input element width $\text{SEW} \div 4$; the *C* format is selected by *altfmt*, interpreted for width SEW . Final accumulation into the *C* accumulator format uses the dynamic rounding mode from *frm*. Any intermediate grouping, partial-sum formation, and optional rounding of partial sums follow [Section 36.7.2](#). Floating-point exception flags accumulate in *fflags*.

+ When *vm=0* (*v0.scale*), the register *v0* supplies paired block-scale factors for microscaling; the scale format is determined by the input data type (see [Section 36.7.3](#)). The *bs* field in *vtype* selects the block size (32 or 16 elements). Only $\text{SEW} \in \{16, 32, 64\}$ is valid, giving $\text{EEW} = \text{SEW} \div 4 \in \{4, 8, 16\}$. $\text{SEW} = 8$, which would imply $\text{EEW} = 2$, is *reserved*.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- *vstart* $\neq 0$.
- $\text{SEW} = 8$ ($\text{EEW} = \text{SEW} \div 4 = 2$ is reserved).
- *VL* is not a multiple of $\lambda \times \text{LMUL}$.
- $\text{EMUL_C} (= \text{VLEN} \div (\text{SEW} \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL .
- *vd* is not aligned for EMUL_C .
- When *vm=0*: the selected *MX* cell in [Table 87](#) is – or *reserved*, or any required extension named by that cell is not implemented.

- When $vm=0$: $SEW \times \lambda < 16$.
- When $vm=0$ and $bs=1$: $W \times LMUL > SEW$.
- When $vm=0$: vd , $vs1$, or $vs2$ overlaps $v0$.
- When $vm=1$: the selected row in [Table 87](#) is marked *reserved*, or any extension named by that row is not implemented.

Operation

```

if vstart != 0 then return Illegal_Instruction();

let SEW_check : int = 2 ^ get_sew_pow();
if SEW_check == 8 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(4);

if vm == 0 then {
    check_microscaling_legality(4, g.LMUL, g.EEW_C, g.lambda, E8M0);
    check_fp_mx_encoding_legality(4, g.EEW_C, vtype[altfmt_A], vtype[altfmt_B],
    vtype[altfmt], vtype[bs])
} else {
    check_fp_encoding_legality(4, g.EEW_C, vtype[altfmt_A], vtype[altfmt_B],
    vtype[altfmt])
};

let fmt_A : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_A]);
let fmt_B : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_B]);
let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm    : rounding_mode = get_fp_rounding_mode();

if vm == 0 then
    fp_scaled_gemm(g, fmt_A, fmt_B, fmt_C, E8M0, rm, vs1, vs2, vd)
else
    fp_gemm(g, fmt_A, fmt_B, fmt_C, rm, vs1, vs2, vd);
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvxfmm (Matrix-multiplication instructions (floating-point))	0.1	Draft
Zvvxofp4fp16mm, Zvvxofp4bf16mm, Zvvxofp8fp32mm (Section 36.7.3)	0.1	Draft
Zvvxnofp4fp16mm, Zvvxnofp4bf16mm, Zvvxnofp8fp32mm (Section 36.7.3)	0.1	Draft

- When **vm=0**: *vd*, *vs1*, or *vs2* overlaps *v0*.
- When **vm=1**: the selected row in [Table 87](#) is marked *reserved*, or any extension named by that row is not implemented.

Operation

```

if vstart != 0 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(2);

if vm == 0 then {
    check_microscaling_legality(2, g.LMUL, g.EEW_C, g.lambda, E8M0);
    check_fp_mx_encoding_legality(2, g.EEW_C, vtype[altfmt_A], vtype[altfmt_B],
    vtype[altfmt], vtype[bs])
} else {
    check_fp_encoding_legality(2, g.EEW_C, vtype[altfmt_A], vtype[altfmt_B],
    vtype[altfmt])
};

let fmt_A : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_A]);
let fmt_B : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_B]);
let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm    : rounding_mode = get_fp_rounding_mode();

if vm == 0 then
    fp_scaled_gemm(g, fmt_A, fmt_B, fmt_C, E8M0, rm, vs1, vs2, vd)
else
    fp_gemm(g, fmt_A, fmt_B, fmt_C, rm, vs1, vs2, vd);
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvxfmm (Matrix-multiplication instructions (floating-point))	0.1	Draft
Zvvxofp4ofp8mm, Zvvxofp8fp16mm, Zvvxofp8bf16mm (Section 36.7.3)	0.1	Draft
Zvvxnofp4ofp8mm, Zvvxnofp8fp16mm, Zvvxnofp8bf16mm (Section 36.7.3)	0.1	Draft

36.11.6. vfwimmac.vv

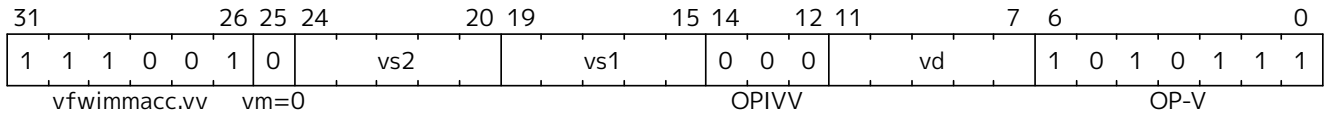
Synopsis

Integer-Input Widening Floating-Point-Accumulate Matrix Multiply-Accumulate with Microscaling

Mnemonic

vfwimmac.vv *vd*, *vs1*, *vs2*, *v0.scale*

Encoding



This instruction shares its opcode with **vmmacc.vv**; **vm=0** selects the integer-input FP-accumulate MX form. **vmmacc.vv** requires **vm=1**.

Description

Interpreting *vs1* as A_{tile} and *vs2* as B_{tile}^T , the instruction computes the exact integer tile product

$$T = A_{\text{tile}} \times B_{\text{tile}}^T$$

block by block, scales each block by the paired scale factors in **v0**, and accumulates the scaled result into *vd*: $C \leftarrow C + \text{scale_A} \times \text{scale_B} \times T$. The widening factor is 2 applied to the K dimension.

+ *vs1* is interpreted as the $M \times K_{\text{eff}}$ tile A_{tile} in row-major register layout, and *vs2* is interpreted as the $K_{\text{eff}} \times N$ tile B_{tile}^T in column-major register layout, and *vd* as an $M \times M$ physical FP accumulator tile (row-major register layout), of which only the first N columns are active. *vs1* and *vs2* have integer elements of width $\text{SEW} \div 2$; *vd* has FP elements of width SEW.

+ $K_{\text{effective}} = \lambda \times 2 \times \text{LMUL}$. **VL** selects how many columns of B_{tile}^T , and therefore how many columns of C, are active; VL must be a multiple of $\lambda \times \text{LMUL}$. The remaining columns of the physical C tile are C tile tail elements. MXINT inputs are always signed; **altfmt_A** and **altfmt_B** must be 0. **altfmt** selects the FP format of the accumulator C.

+ The register **v0** supplies paired block-scale factors using the same tile-strided layout as **vfwimmac.vv** (see [Section 36.7.3](#)). The **bs** field in **vtype** selects the block size (32 or 16 elements).

+ The encodings SEW=8 (EEW=4, Int4 inputs into OFP8 accumulator), SEW=32, and SEW=64 are *reserved*.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- **vstart** $\neq 0$.
- SEW = 8 (reserved encoding).
- SEW = 32 (reserved encoding).
- SEW = 64 (reserved encoding).
- **altfmt_A** = 1 (reserved; MXINT inputs are signed).
- **altfmt_B** = 1 (reserved; MXINT inputs are signed).
- The selected MX cell in [Section 36.13.3](#) is *reserved*, or the extension named by that cell is not implemented.
- VL is not a multiple of $\lambda \times \text{LMUL}$.

- $EMUL_C (= VLEN \div (SEW \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- $vs1$ or $vs2$ is not aligned for LMUL.
- vd is not aligned for $EMUL_C$.
- $SEW \times \lambda < 16$.
- When $bs=1$: $W \times LMUL > SEW$ (i.e., $2 \times LMUL > SEW$).
- vd , $vs1$, or $vs2$ overlaps $v0$.

Operation

```

if vstart != 0 then return Illegal_Instruction();

let EEW_C_check : int = 2 ^ get_sew_pow();
if EEW_C_check == 8 then return Illegal_Instruction();
if EEW_C_check == 32 then return Illegal_Instruction();
if EEW_C_check == 64 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(2);

check_microscaling_legality(2, g.LMUL, g.EEW_C, g.lambda, E8M0);

if vtype[altfmt_A] == 0b1 then return Illegal_Instruction();
if vtype[altfmt_B] == 0b1 then return Illegal_Instruction();

check_int_mx_encoding_legality(
    2, g.EEW_C,
    vtype[altfmt_A], vtype[altfmt_B],
    vtype[altfmt], vtype[bs]);

let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm    : rounding_mode = get_fp_rounding_mode();

int_scaled_gemm(g, fmt_C, E8M0, rm, vs1, vs2, vd);
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvvxi8fp16mm, Zvvxi8bf16mm (Section 36.7.3)	0.1	Draft
Zvvxni8fp16mm, Zvvxni8bf16mm (Section 36.7.3)	0.1	Draft

36.11.7. vf8wimmacc.vv

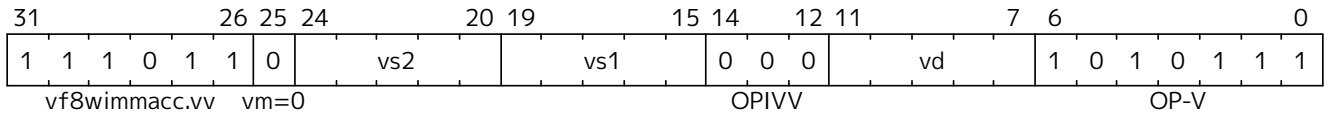
Synopsis

Integer-Input 8× Widening Floating-Point-Accumulate Matrix Multiply-Accumulate with Microscaling

Mnemonic

vf8wimmacc.vv vd, vs1, vs2, v0.scale

Encoding



This instruction shares its opcode with **v8wmmacc.vv**; **vm=0** selects the integer-input FP-accumulate MX form. **v8wmmacc.vv** requires **vm=1**.

Description

Interpreting *vs1* as A_{tile} and *vs2* as B_{tile}^T , the instruction computes the exact integer tile product

$$T = A_{\text{tile}} \times B_{\text{tile}}^T$$

block by block, scales each block by the paired scale factors in **v0**, and accumulates the scaled result into *vd*: $C \leftarrow C + \text{scale_A} \times \text{scale_B} \times T$. The widening factor is 8 applied to the K dimension.

+ *vs1* is interpreted as the $M \times K_{\text{eff}}$ tile A_{tile} in row-major register layout, and *vs2* is interpreted as the $K_{\text{eff}} \times N$ tile B_{tile}^T in column-major register layout, and *vd* as an $M \times M$ physical FP accumulator tile (row-major register layout), of which only the first N columns are active. *vs1* and *vs2* have integer elements of width $\text{SEW} \div 8$; *vd* has FP elements of width SEW.

+ $K_{\text{effective}} = \lambda \times 8 \times \text{LMUL}$. **VL** selects how many columns of B_{tile}^T , and therefore how many columns of C, are active; VL must be a multiple of $\lambda \times \text{LMUL}$. The remaining columns of the physical C tile are C tile tail elements. MXINT inputs are always signed; **altfmt_A** and **altfmt_B** must be 0. **altfmt** selects the FP format of the accumulator C.

+ The register **v0** supplies paired block-scale factors using the same tile-strided layout as **vf8wmmacc.vv** (see [Section 36.7.3](#)). The **bs** field in **vtype** selects the block size (32 or 16 elements).

+ Only $\text{SEW} \in \{32, 64\}$ is valid; $\text{SEW} \in \{8, 16\}$ is reserved.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- **vstart** $\neq 0$.
- $\text{SEW} < 32$ (reserved encoding).
- **altfmt_A** = 1 (reserved; MXINT inputs are signed).
- **altfmt_B** = 1 (reserved; MXINT inputs are signed).
- The selected MX cell in [Section 36.13.3](#) is *reserved*, or the extension named by that cell is not implemented.
- VL is not a multiple of $\lambda \times \text{LMUL}$.
- $\text{EMUL_C} (= \text{VLEN} \div (\text{SEW} \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.

- vd is not aligned for $EMUL_C$.
- $SEW \times \lambda < 16$.
- When $bs=1$: $W \times LMUL > SEW$ (i.e., $8 \times LMUL > SEW$).
- vd , $vs1$, or $vs2$ overlaps $v0$.

Operation

```
if vstart != 0 then return Illegal_Instruction();

if (2 ^ get_sew_pow()) < 32 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(8);

check_microscaling_legality(8, g.LMUL, g.EEW_C, g.lambda, E8M0);

if vtype[altfmt_A] == 0b1 then return Illegal_Instruction();
if vtype[altfmt_B] == 0b1 then return Illegal_Instruction();

check_int_mx_encoding_legality(
    8, g.EEW_C,
    vtype[altfmt_A], vtype[altfmt_B],
    vtype[altfmt], vtype[bs]);

let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm     : rounding_mode = get_fp_rounding_mode();

int_scaled_gemm(g, fmt_C, E8M0, rm, vs1, vs2, vd);
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvvxi4fp32mm, Zvvxi8fp64mm (Section 36.7.3)	0.1	Draft
Zvxxni4fp32mm, Zvxxni8fp64mm (Section 36.7.3)	0.1	Draft

36.11.8. vfqimmac.vv

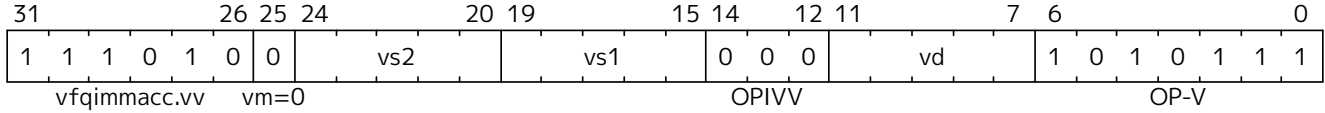
Synopsis

Integer-Input Quad-Widening Floating-Point-Accumulate Matrix Multiply-Accumulate with Microscaling

Mnemonic

vfqimmac.vv vd, vs1, vs2, v0.scale

Encoding



This instruction shares its opcode with `vqmmacc.vv`; `vm=0` selects the integer-input FP-accumulate MX form. `vqmmacc.vv` requires `vm=1`.

Description

Interpreting `vs1` as A_{tile} and `vs2` as B_{tile}^T , the instruction computes the exact integer tile product

$$T = A_{\text{tile}} \times B_{\text{tile}}^T$$

block by block, scales each block by the paired scale factors in `v0`, and accumulates the scaled result into `vd`: $C \leftarrow C + \text{scale_A} \times \text{scale_B} \times T$. The widening factor is 4 applied to the K dimension.

+ `vs1` is interpreted as the $M \times K_{\text{eff}}$ tile A_{tile} in row-major register layout, and `vs2` is interpreted as the $K_{\text{eff}} \times N$ tile B_{tile}^T in column-major register layout, and `vd` as an $M \times M$ physical FP accumulator tile (row-major register layout), of which only the first N columns are active. `vs1` and `vs2` have integer elements of width $\text{SEW} \div 4$; `vd` has FP elements of width SEW.

+ $K_{\text{effective}} = \lambda \times 4 \times \text{LMUL}$. `VL` selects how many columns of B_{tile}^T , and therefore how many columns of C, are active; `VL` must be a multiple of $\lambda \times \text{LMUL}$. The remaining columns of the physical C tile are C tile tail elements. MXINT inputs are always signed; `altfmt_A` and `altfmt_B` must be 0. `altfmt` selects the FP format of the accumulator C.

+ The register `v0` supplies paired block-scale factors using the same tile-strided layout as `vqmmacc.vv` (see [Section 36.7.3](#)). The `bs` field in `vtype` selects the block size (32 or 16 elements).

+ The encodings $\text{SEW}=8$ ($\text{EEW}=2$), `altfmt=1` with $\text{SEW}=32$, and $\text{SEW}=64$ are reserved.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- `vstart` $\neq 0$.
- $\text{SEW} = 8$ (reserved encoding).
- $\text{SEW} = 32$ and `altfmt` = 1 (reserved).
- $\text{SEW} = 64$ (reserved encoding).
- `altfmt_A` = 1 (reserved; MXINT inputs are signed).
- `altfmt_B` = 1 (reserved; MXINT inputs are signed).
- The selected MX cell in [Section 36.13.3](#) is *reserved*, or the extension named by that cell is not implemented.

- VL is not a multiple of $\lambda \times \text{LMUL}$.
- $\text{EMUL_C} (= \text{VLEN} \div (\text{SEW} \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for EMUL_C.
- $\text{SEW} \times \lambda < 16$.
- When **bs=1**: $W \times \text{LMUL} > \text{SEW}$ (i.e., $4 \times \text{LMUL} > \text{SEW}$).
- *vd*, *vs1*, or *vs2* overlaps *v0*.

Operation

```

if vstart != 0 then return Illegal_Instruction();

let EEW_C_check : int = 2 ^ get_sew_pow();
if EEW_C_check == 8 then return Illegal_Instruction();
if EEW_C_check == 32 & vtype[altfmt] == 0b1 then return Illegal_Instruction();
if EEW_C_check == 64 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(4);

check_microscaling_legality(4, g.LMUL, g.EEW_C, g.lambda, E8M0);

if vtype[altfmt_A] == 0b1 then return Illegal_Instruction();
if vtype[altfmt_B] == 0b1 then return Illegal_Instruction();

check_int_mx_encoding_legality(
    4, g.EEW_C,
    vtype[altfmt_A], vtype[altfmt_B],
    vtype[altfmt], vtype[bs]);

let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm    : rounding_mode = get_fp_rounding_mode();

int_scaled_gemm(g, fmt_C, E8M0, rm, vs1, vs2, vd);
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvvxi4fp16mm, Zvvxi4bf16mm, Zvvxi8fp32mm (Section 36.7.3)	0.1	Draft
Zvvxni4fp16mm, Zvvxni4bf16mm, Zvvxni8fp32mm (Section 36.7.3)	0.1	Draft

36.11.9. vmmacc.vv

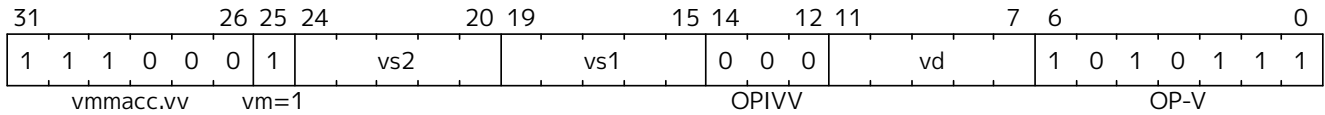
Synopsis

Integer Matrix Multiply-Accumulate

Mnemonic

vmmacc.vv *vd*, *vs1*, *vs2*

Encoding



Description

Interpreting *vs1* as A_{tile} and *vs2* as B_{tile}^T , the instruction computes

$$T = A_{\text{tile}} \times B_{\text{tile}}^T$$

and accumulates T into *vd*: $C \leftarrow C + T$.

+ *vs1* is interpreted as the $M \times K_{\text{eff}}$ tile A_{tile} in row-major register layout, and *vs2* is interpreted as the $K_{\text{eff}} \times N$ tile B_{tile}^T in column-major register layout, and *vd* as an $M \times M$ physical accumulator tile (row-major register layout), of which only the first N columns are active. All three tiles have elements of width SEW.

+ $K_{\text{effective}} = \lambda \times \text{LMUL}$. **VL** selects how many columns of B_{tile}^T , and therefore how many columns of C, are active; VL must be a multiple of $\lambda \times \text{LMUL}$. The remaining columns of the physical C tile are C tile tail elements. The signedness of each input operand is selected by **altfmt_A** and **altfmt_B** in **vtype**: 0 = signed, 1 = unsigned. Accumulation is modular (wraps at 2^{SEW}).

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- **vstart** $\neq 0$.
- **vm** = 0 (*reserved*).
- VL is not a multiple of $\lambda \times \text{LMUL}$.
- $\text{EMUL_C} (= \text{VLEN} \div (\text{SEW} \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for EMUL_C.
- The selected row in [Table 88](#) is marked *reserved*, or the extension named by that row is not implemented.

Operation

```

if vstart != 0 then return Illegal_Instruction();

if vm == 0 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(1);
check_int_encoding_legality(1, g.EEW_C, vtype[altfmt_A], vtype[altfmt_B]);

```

```
let signed_A : bool = vtype[altfmt_A] == 0b0;
let signed_B : bool = vtype[altfmt_B] == 0b0;

int_gemm(g, signed_A, signed_B, vs1, vs2, vd);
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvmm (Matrix-multiplication instructions (integer))	0.1	Draft

36.11.10. vmtl.v

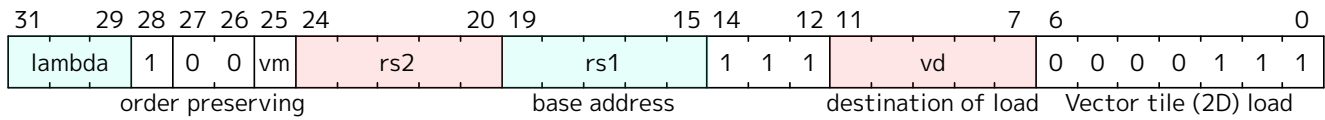
Synopsis

Order-Preserving Matrix Tile Load

Mnemonic

vmtl.v *vd*, (*rs1*), *rs2* [, *Lλ*] [, *vm*]

Encoding



Description

Loads a 2D matrix tile from memory into the vector register group starting at *vd*. *rs1* holds the matrix base address; *rs2* holds the leading dimension **LD** in elements of width SEW. if *rs2* holds zero (0), the leading dimension **LD** is set to $\lambda \times \text{LMUL}$. When *Lλ* is specified, it overrides **lambda[2:0]** from **vtype** for this instruction. Inactive elements (masked off) are left undisturbed in the destination register. Tail elements (index $\geq \text{VL}$) are left undisturbed in the destination register. The tail-agnostic (**vtype.vta**) and mask-agnostic (**vtype.vma**) fill policies do not apply to tile loads. Neither inactive nor tail elements raise memory exceptions.

Let $\text{linesize} = \lambda \times \text{LMUL}$. For each element index *i* in the body [**vstart**, **VL**) where the mask is enabled:

$$\text{VD}[i] = \text{M}[\text{rs1} + (\text{SEW} \div 8) \times ((i / \text{linesize}) \times \text{LD} + (i \% \text{linesize}))]$$

This is the correct load variant when A is stored in row-major order or when B is stored in column-major order: in both cases the memory layout consists of *linesize*-element contiguous segments with a stride of **LD** elements between them. For **LMUL**=1 this simplifies to λ -element segments.

Exceptions

The instruction raises the standard vector memory exceptions for active elements.

Illegal Instruction is raised if any of the following conditions holds:

- **vtype.vlmul** does not encode **LMUL** in {1, 2, 4, 8}.
- No effective lambda is selected.
- The effective lambda is not supported for the current (**VLEN**, **SEW**) configuration. A nonzero instruction lambda field is checked exactly and is not WARL-clamped.
- **VL** is not a multiple of **effective_lambda * LMUL**.
- *vd* is not aligned for **LMUL**, or its register group extends past **v31**.
- **vm**=0 and the destination register group overlaps **v0**.

A nonzero **vstart** value is legal and is not an illegal-instruction condition.

Operation

```
let t : tile_ls_geom =
  decode_tile_ls_geometry(instr.lambda);
```

```

let SEW_pow      : int      = t.SEW_pow;
let EEW          : int      = t.EEW;
let EEW_bytes    : word_width = 2 ^ (SEW_pow - 3);
let LMUL         : int      = t.LMUL;
let eff_lambda   : int      = t.lambda;
let linesize     : int      = t.linesize;
let elems_per_reg : int      = t.elems_per_reg;
let num_elem     : int      = t.num_elem;

check_ime_reg_group(vd, LMUL);

// A masked load destination may not overlap mask register v0.
// Because the destination group is LMUL-aligned, overlap with v0
// occurs exactly when the destination base register is v0.
if vm == 0b0 then {
    if unsigned(vd) == 0 then
        return Illegal_Instruction()
};

var LD : int = unsigned(X(rs2));
let vm_val = read_vmask(num_elem, vm, zvreg);

if LD == 0 then {
    LD = linesize;
};

let start : int = unsigned(vstart);

if start < num_elem then {
    foreach (i from start to (num_elem - 1)) {
        if vm_val[i] == 0b1 then {
            set_vstart(to_bits_unsafe(16, i));

            let flat_idx : int =
                tile_reg_idx(i, LMUL, eff_lambda, elems_per_reg);

            let mem_off : int =
                (i / linesize) * LD + (i % linesize);

            match vmem_read(
                rs1,
                to_bits_unsafe(xlen, mem_off * EEW_bytes),
                EEW_bytes,
                Load(Data),
                false,
                false,
                false
            ) {
                Ok(elem) =>

```

```

        write_single_element(EEW, flat_idx, vd, elem),
    Err(e) =>
        return e,
    }
}
// Inactive body elements perform no memory access.
// Prestart elements are excluded by the loop lower bound.
// Tail elements are excluded by the loop upper bound.
}
};

set_vstart(zeros());
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvvmtls (Load/store instructions for matrix tiles)	0.1	Draft

36.11.11. vmts.v

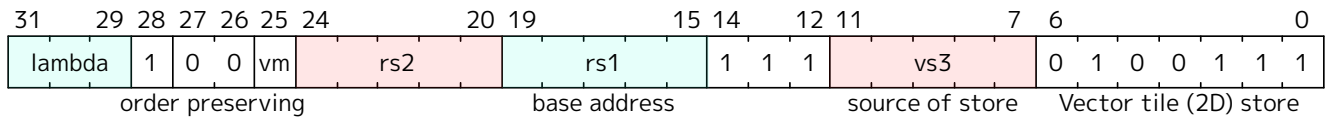
Synopsis

Order-Preserving Matrix Tile Store

Mnemonic

vmts.v vs3, (rs1), rs2 [, Lλ] [, vm]

Encoding



Description

Stores the 2D matrix tile held in the vector register group starting at vs3 to memory. rs1 holds the matrix base address; rs2 holds the leading dimension LD in elements of width SEW. If rs2 holds zero (0), the leading dimension LD is set to $\lambda \times \text{LMUL}$. When Lλ is specified, it overrides lambda[2:0] from vtype for this instruction. Inactive elements (masked off) are not written to memory and do not raise exceptions. Tail elements (index $\geq \text{VL}$) are not written to memory and do not raise exceptions.

Let $\text{linesize} = \lambda \times \text{LMUL}$. For each element index i in the body [vstart, VL) where the mask is enabled:

$$M[\text{rs1} + (\text{SEW} \div 8) \times ((i / \text{linesize}) \times \text{LD} + (i \% \text{linesize}))] = \text{VS}[i]$$

Exceptions

The instruction raises the standard vector memory exceptions for active elements.

Illegal Instruction is raised if any of the following conditions holds:

- vtype.vlmul does not encode LMUL in {1, 2, 4, 8}.
- No effective lambda is selected.
- The effective lambda is not supported for the current (VLEN, SEW) configuration. A nonzero instruction lambda field is checked exactly and is not WARL-clamped.
- VL is not a multiple of effective_lambda * LMUL.
- vs3 is not aligned for LMUL, or its register group extends past v31.

A nonzero vstart value is legal and is not an illegal-instruction condition.

Operation

```
let t : tile_ls_geom =
    decode_tile_ls_geometry(instr.lambda);

let SEW_pow      : int      = t.SEW_pow;
let EEW          : int      = t.EEW;
let EEW_bytes    : word_width = 2 ^ (SEW_pow - 3);
let LMUL         : int      = t.LMUL;
let eff_lambda   : int      = t.lambda;
let linesize     : int      = t.linesize;
```

```

let elems_per_reg : int          = t.elems_per_reg;
let num_elem       : int          = t.num_elem;

check_ime_reg_group(vs3, LMUL);

var LD : int = unsigned(X(rs2));
let vm_val = read_vmask(num_elem, vm, zvreg);

if LD == 0 then {
    LD = linesize;
};

let start : int = unsigned(vstart);

if start < num_elem then {
    foreach (i from start to (num_elem - 1)) {
        if vm_val[i] == 0b1 then {
            set_vstart(to_bits_unsafe(16, i));

            let flat_idx : int =
                tile_reg_idx(i, LMUL, eff_lambda, elems_per_reg);

            let mem_off : int =
                (i / linesize) * LD + (i % linesize);

            let data =
                read_single_element(EEW, flat_idx, vs3);

            match vmem_write(
                rs1,
                to_bits_unsafe(xlen, mem_off * EEW_bytes),
                EEW_bytes,
                data,
                Store(Data),
                false,
                false,
                false
            ) {
                Ok(true) =>
                    (),
                Ok(false) =>
                    internal_error(
                        __FILE__,
                        __LINE__,
                        "store got false from vmem_write"
                    ),
                Err(e) =>
                    return e,
            }
        }
    }
}

```

```
// Inactive body elements perform no memory access.
// Prestart elements are excluded by the loop lower bound.
// Tail elements are excluded by the loop upper bound.
}
};

set_vstart(zeros());
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvmtls (Load/store instructions for matrix tiles)	0.1	Draft

36.11.12. vmttl.v

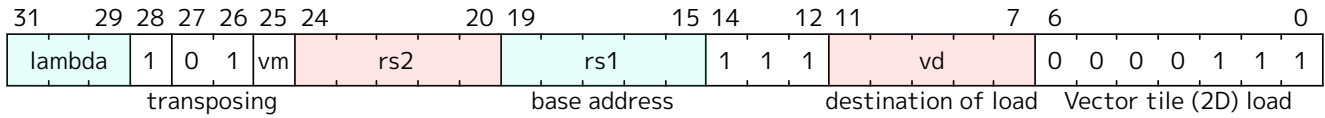
Synopsis

Transposing Matrix Tile Load

Mnemonic

vmttl.v vd, (rs1), rs2 [, Lλ] [, vm]

Encoding



Description

Loads a two-dimensional array of **SEW**-bit storage elements from memory and transposes those storage elements into the vector register-group layout.

When each logical matrix element occupies one **SEW**-bit storage element, this operation is also a logical matrix transpose. In that case it can be used when a B matrix is stored in row-major order, or when an A or C matrix is stored in column-major order.

When multiple logical elements are packed within each **SEW**-bit storage element, the instruction transposes the storage elements but does not transpose the packed logical elements within them.

rs1 holds the matrix base address; *rs2* holds the leading dimension **LD** in elements of width **SEW**. If *rs2* holds zero (0), the leading dimension **LD** is set to $VLEN / (SEW \times \lambda)$, which is the natural maximum width of a B tile and height of an A or C tile. The parameter λ refers to the result matrix tile of the transposing load operation. When *Lλ* is specified, it overrides **lambda[2:0]** from **vtype** for this instruction. Inactive elements (masked off) are left undisturbed in the destination register. Tail elements (index $\geq VL$) are left undisturbed in the destination register. The tail-agnostic (**vtype.vta**) and mask-agnostic (**vtype.vma**) fill policies do not apply to tile loads. Neither inactive nor tail elements raise memory exceptions.

+ Let $linesize = \lambda \times LMUL$. For each element index *i* in the body [**vstart**, **VL**) where the mask is enabled:

+ $VD[i] = M[rs1 + (SEW \div 8) \times i \% linesize] \times LD + (i / linesize)$

Exceptions

The instruction raises the standard vector memory exceptions for active elements.

Illegal Instruction is raised if any of the following conditions holds:

- **vtype.vlmul** does not encode **LMUL** in {1, 2, 4, 8}.
- No effective **lambda** is selected.
- The effective **lambda** is not supported for the current (**VLEN**, **SEW**) configuration. A nonzero instruction **lambda** field is checked exactly and is not WARL-clamped.
- **VL** is not a multiple of **effective_lambda * LMUL**.
- *vd* is not aligned for **LMUL**, or its register group extends past **v31**.
- **vm=0** and the destination register group overlaps **v0**.

A nonzero **vstart** value is legal and is not an illegal-instruction condition.

Operation

```

let t : tile_ls_geom =
    decode_tile_ls_geometry(instr.lambda);

let SEW_pow      : int      = t.SEW_pow;
let EEW          : int      = t.EEW;
let EEW_bytes    : word_width = 2 ^ (SEW_pow - 3);
let LMUL         : int      = t.LMUL;
let eff_lambda   : int      = t.lambda;
let linesize     : int      = t.linesize;
let elems_per_reg : int      = t.elems_per_reg;
let num_elem     : int      = t.num_elem;

check_ime_reg_group(vd, LMUL);

// A masked load destination may not overlap mask register v0.
// Because the destination group is LMUL-aligned, overlap with v0
// occurs exactly when the destination base register is v0.
if vm == 0b0 then {
    if unsigned(vd) == 0 then
        return Illegal_Instruction();
};

var LD : int = unsigned(X(rs2));
let vm_val = read_vmask(num_elem, vm, zvreg);

if LD == 0 then {
    LD = elems_per_reg / eff_lambda;
};

let start : int = unsigned(vstart);

if start < num_elem then {
    foreach (i from start to (num_elem - 1)) {
        if vm_val[i] == 0b1 then {
            set_vstart(to_bits_unsafe(16, i));

            let flat_idx : int =
                tile_reg_idx(i, LMUL, eff_lambda, elems_per_reg);

            let mem_off : int =
                (i % linesize) * LD + (i / linesize);

            match vmem_read(
                rs1,
                to_bits_unsafe(xlen, mem_off * EEW_bytes),
                EEW_bytes,
                Load(Data),
                false,
                false,

```



```
        false
    ) {
        Ok(elem) =>
            write_single_element(EEW, flat_idx, vd, elem),
        Err(e) =>
            return e,
    }
}
// Inactive body elements perform no memory access.
// Prestart elements are excluded by the loop lower bound.
// Tail elements are excluded by the loop upper bound.
}
};

set_vstart(zeros());
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvmmttls (Transposing load/store instructions for matrix tiles)	0.1	Draft

36.11.13. vmtts.v

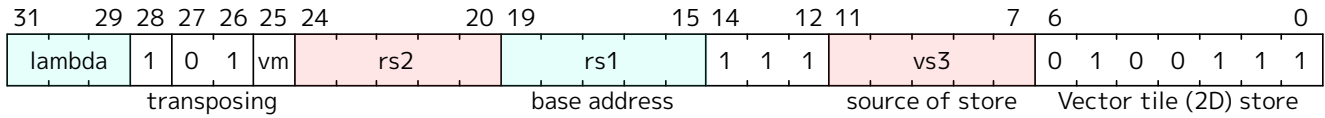
Synopsis

Transposing Matrix Tile Store

Mnemonic

vmtts.v vs3, (rs1), rs2 [, Lλ] [, vm]

Encoding



Description

Stores a two-dimensional array of **SEW**-bit storage elements from a vector register group to memory, applying the inverse storage-element transposition of **vmttl.v**.

The instruction does not unpack, transpose, or repack logical elements contained within an **SEW**-bit storage element.

rs1 holds the matrix base address; *rs2* holds the leading dimension **LD** in elements of width **SEW**. If *rs2* holds zero (0), the leading dimension **LD** is set to $VLEN / (SEW \times \lambda)$. The parameter λ refers to the source matrix tile of the transposing store operation. When **Lλ** is specified, it overrides **lambda[2:0]** from **vtype** for this instruction. Inactive elements (masked off) are not written to memory and do not raise exceptions. Tail elements (index $\geq VL$) are not written to memory and do not raise exceptions.

+ Let $linesize = \lambda \times LMUL$. For each element index **i** in the body [**vstart**, **VL**) where the mask is enabled:

+ $M[rs1 + (SEW \div 8) \times i \% linesize) \times LD + (i / linesize)] = VS[i]$

Exceptions

The instruction raises the standard vector memory exceptions for active elements.

Illegal Instruction is raised if any of the following conditions holds:

- **vtype.vlmul** does not encode **LMUL** in {1, 2, 4, 8}.
- No effective **lambda** is selected.
- The effective **lambda** is not supported for the current (**VLEN**, **SEW**) configuration. A nonzero instruction **lambda** field is checked exactly and is not WARL-clamped.
- **VL** is not a multiple of **effective_lambda * LMUL**.
- **vs3** is not aligned for **LMUL**, or its register group extends past **v31**.

A nonzero **vstart** value is legal and is not an illegal-instruction condition.

Operation

```

let t : tile_ls_geom =
    decode_tile_ls_geometry(instr.lambda);

let SEW_pow      : int      = t.SEW_pow;
let EEW          : int      = t.EEW;

```

```

let EEW_bytes      : word_width = 2 ^ (SEW_pow - 3);
let LMUL           : int        = t.LMUL;
let eff_lambda     : int        = t.lambda;
let linesize       : int        = t.linesize;
let elems_per_reg  : int        = t.elems_per_reg;
let num_elem       : int        = t.num_elem;

check_ime_reg_group(vs3, LMUL);

var LD : int = unsigned(X(rs2));
let vm_val = read_vmask(num_elem, vm, zvreg);

if LD == 0 then {
  LD = elems_per_reg / eff_lambda;
};

let start : int = unsigned(vstart);

if start < num_elem then {
  foreach (i from start to (num_elem - 1)) {
    if vm_val[i] == 0b1 then {
      set_vstart(to_bits_unsafe(16, i));

      let flat_idx : int =
        tile_reg_idx(i, LMUL, eff_lambda, elems_per_reg);

      let mem_off : int =
        (i % linesize) * LD + (i / linesize);

      let data =
        read_single_element(EEW, flat_idx, vs3);

      match vmem_write(
        rs1,
        to_bits_unsafe(xlen, mem_off * EEW_bytes),
        EEW_bytes,
        data,
        Store(Data),
        false,
        false,
        false
      ) {
        Ok(true) =>
          (),
        Ok(false) =>
          internal_error(
            __FILE__,
            __LINE__,
            "store got false from vmem_write"
          ),
      }
    }
  }
}

```

```
        Err(e) =>
            return e,
        }
    }
    // Inactive body elements perform no memory access.
    // Prestart elements are excluded by the loop lower bound.
    // Tail elements are excluded by the loop upper bound.
}
};

set_vstart(zeros());
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvmmttls (Transposing load/store instructions for matrix tiles)	0.1	Draft

36.11.14. vqmmacc.vv

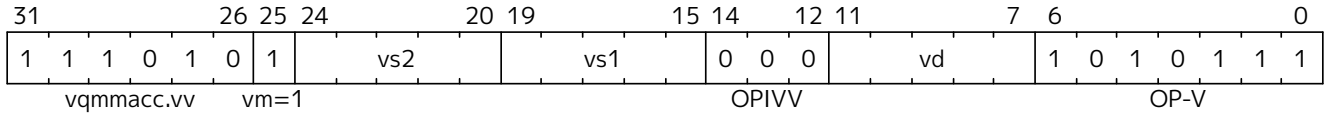
Synopsis

Integer Quad-Widening Matrix Multiply-Accumulate

Mnemonic

vqmmacc.vv vd, vs1, vs2

Encoding



Description

Interpreting *vs1* as A_{tile} and *vs2* as B_{tile}^T , the instruction computes

$$T = A_{\text{tile}} \times B_{\text{tile}}^T$$

and accumulates *T* into *vd*: $C \leftarrow C + T$, with a widening factor of 4 applied to the *K* dimension.

+ *vs1* is interpreted as the $M \times K_{\text{eff}}$ tile A_{tile} in row-major register layout, and *vs2* is interpreted as the $K_{\text{eff}} \times N$ tile B_{tile}^T in column-major register layout, and *vd* as an $M \times M$ physical accumulator tile (row-major register layout), of which only the first *N* columns are active. *vs1* and *vs2* have elements of width $\text{SEW} \div 4$; *vd* has elements of width *SEW*.

+ $K_{\text{effective}} = \lambda \times 4 \times \text{LMUL}$. *VL* selects how many columns of B_{tile}^T , and therefore how many columns of *C*, are active; *VL* must be a multiple of $\lambda \times \text{LMUL}$. The remaining columns of the physical *C* tile are *C* tile tail elements. The signedness of each input operand is selected by *altfmt_A* and *altfmt_B* in *vtype*: 0 = signed, 1 = unsigned. Accumulation is modular (wraps at 2^{SEW}). Only $\text{SEW} \in \{16, 32, 64\}$ is valid, giving $\text{EEW} = \text{SEW} \div 4 \in \{4, 8, 16\}$. $\text{SEW} = 8$, which would imply $\text{EEW} = 2$, is *reserved*.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- *vstart* $\neq 0$.
- $\text{SEW} = 8$ ($\text{EEW} = \text{SEW} \div 4 = 2$ is reserved).
- *VL* is not a multiple of $\lambda \times \text{LMUL}$.
- $\text{EMUL_C} (= \text{VLEN} \div (\text{SEW} \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for *LMUL*.
- *vd* is not aligned for *EMUL_C*.
- When *vm*=1: the selected row in [Table 88](#) is marked *reserved*, or the extension named by that row is not implemented.

Operation

```
if vstart != 0 then return Illegal_Instruction();

let SEW_check : int = 2 ^ get_sew_pow();
if SEW_check == 8 then return Illegal_Instruction();
```

```
let g : gemm_geom = decode_gemm_geometry(4);
check_int_encoding_legality(4, g.EEW_C, vtype[altfmt_A], vtype[altfmt_B]);

let signed_A : bool = vtype[altfmt_A] == 0b0;
let signed_B : bool = vtype[altfmt_B] == 0b0;

int_gemm(g, signed_A, signed_B, vs1, vs2, vd);
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvmm (Matrix-multiplication instructions (integer))	0.1	Draft

36.11.15. vwmacc.vv

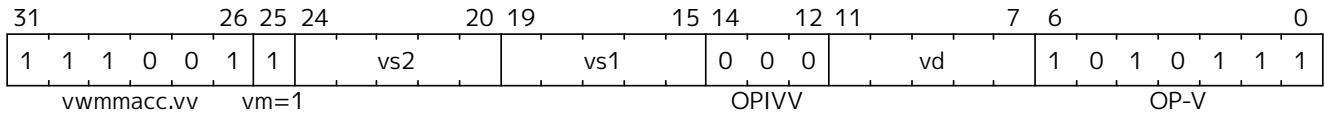
Synopsis

Integer Widening Matrix Multiply-Accumulate

Mnemonic

vwmacc.vv *vd*, *vs1*, *vs2*

Encoding



Description

Interpreting *vs1* as A_{tile} and *vs2* as B_{tile}^T , the instruction computes

$$T = A_{\text{tile}} \times B_{\text{tile}}^T$$

and accumulates T into *vd*: $C \leftarrow C + T$, with a widening factor of 2 applied to the K dimension.

+ *vs1* is interpreted as the $M \times K_{\text{eff}}$ tile A_{tile} in row-major register layout, and *vs2* is interpreted as the $K_{\text{eff}} \times N$ tile B_{tile}^T in column-major register layout, and *vd* as an $M \times M$ physical accumulator tile (row-major register layout), of which only the first N columns are active. *vs1* and *vs2* have elements of width $\text{SEW} \div 2$; *vd* has elements of width SEW.

+ $K_{\text{effective}} = \lambda \times 2 \times \text{LMUL}$. VL selects how many columns of B_{tile}^T , and therefore how many columns of C, are active; VL must be a multiple of $\lambda \times \text{LMUL}$. The remaining columns of the physical C tile are C tile tail elements. The signedness of each input operand is selected by `altfmt_A` and `altfmt_B` in `vtype`: 0 = signed, 1 = unsigned. Accumulation is modular (wraps at 2^{SEW}).

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- `vstart` $\neq 0$.
- VL is not a multiple of $\lambda \times \text{LMUL}$.
- $\text{EMUL_C} (= \text{VLEN} \div (\text{SEW} \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for EMUL_C.
- When `vm=1`: the selected row in [Table 88](#) is marked *reserved*, or the extension named by that row is not implemented.

Operation

```
if vstart != 0 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(2);
check_int_encoding_legality(2, g.EEW_C, vtype[altfmt_A], vtype[altfmt_B]);

let signed_A : bool = vtype[altfmt_A] == 0b0;
let signed_B : bool = vtype[altfmt_B] == 0b0;
```

```
int_gemm(g, signed_A, signed_B, vs1, vs2, vd);
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvmm (Matrix-multiplication instructions (integer))	0.1	Draft

36.12. Microscaling Subextensions

Each microscaling extension enables the `vm=0` / `v0.scale` encoding for a specific (input format, accumulator format, block size) combination. A floating-point microscaling extension implies its corresponding base extension: for example, `Zvvxofp4fp16mm` implies `Zvvpfp4fp16mm`, which provides the underlying `OFP4→FP16` instruction support. Only widening floating-point multiply-accumulate instructions define FP microscaling variants, and only for narrow input formats (`OFP4`, `OFP8`). The integer MX extensions (`Zvvxi*mm`, `Zvvxni*mm`) do not imply a base extension: `vfwimmacc.vv` and `vfqimmacc.vv` have no non-MX FP-accumulate form (the `vm=1` encoding of those opcodes is the existing integer-accumulate `vmmacc.vv` / `vqmmacc.vv`). The block size and implication relationships are listed in [Table 86](#).

Table 86. Microscaling extensions

Extension	BS	Implies	Multiplicand Types	Accumulator Type
<code>Zvvxofp4ofp8mm</code>	32	<code>Zvvpfp4ofp8mm</code>	<code>MXFP4</code> (E8M0-scaled <code>OFP4</code>), <code>MXFP4</code>	<code>OFP8</code>
<code>Zvvxofp4fp16mm</code>	32	<code>Zvvpfp4fp16mm</code>	<code>MXFP4</code> , <code>MXFP4</code>	IEEE binary16
<code>Zvvxofp4bfp16mm</code>	32	<code>Zvvpfp4bfp16mm</code>	<code>MXFP4</code> , <code>MXFP4</code>	BFloat16
<code>Zvvxofp8fp16mm</code>	32	<code>Zvvpfp8fp16mm</code>	<code>MXFP8</code> , <code>MXFP8</code>	IEEE binary16
<code>Zvvxofp8bfp16mm</code>	32	<code>Zvvpfp8bfp16mm</code>	<code>MXFP8</code> , <code>MXFP8</code>	BFloat16
<code>Zvvxofp8fp32mm</code>	32	<code>Zvvpfp8fp32mm</code>	<code>MXFP8</code> , <code>MXFP8</code>	IEEE binary32
<code>Zvvxofp4fp32mm</code>	32	<code>Zvvpfp4fp32mm</code>	<code>MXFP4</code> , <code>MXFP4</code>	IEEE binary32
<code>Zvvxofp8fp64mm</code>	32	<code>Zvvpfp8fp64mm</code>	<code>MXFP8</code> , <code>MXFP8</code>	IEEE binary64
<code>Zvvxnofp4ofp8mm</code>	16	<code>Zvvxofp4ofp8mm</code>	<code>MXFP4</code> (E8M0-scaled <code>OFP4</code>), <code>MXFP4</code>	<code>OFP8</code>
<code>Zvvxnofp4fp16mm</code>	16	<code>Zvvxofp4fp16mm</code>	<code>MXFP4</code> , <code>MXFP4</code>	IEEE binary16
<code>Zvvxnofp4bfp16mm</code>	16	<code>Zvvxofp4bfp16mm</code>	<code>MXFP4</code> , <code>MXFP4</code>	BFloat16
<code>Zvvxnofp8fp16mm</code>	16	<code>Zvvxofp8fp16mm</code>	<code>MXFP8</code> , <code>MXFP8</code>	IEEE binary16
<code>Zvvxnofp8bfp16mm</code>	16	<code>Zvvxofp8bfp16mm</code>	<code>MXFP8</code> , <code>MXFP8</code>	BFloat16

Extension	BS	Implies	Multiplicand Types	Accumulator Type
Zvvxnofp8fp32mm	16	Zvvxofp8fp32mm	MXFP8, MXFP8	IEEE binary32
Zvvxnofp4fp32mm	16	Zvvxofp4fp32mm	MXFP4, MXFP4	IEEE binary32
Zvvxnofp8fp64mm	16	Zvvxofp8fp64mm	MXFP8, MXFP8	IEEE binary64
Zvvxi8fp16mm	32	—	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary16
Zvvxi8bf16mm	32	—	MXINT8 (E8M0-scaled Int8), MXINT8	BFloat16
Zvvxi8fp32mm	32	—	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary32
Zvvxi8fp64mm	32	—	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary64
Zvvxi4fp16mm	32	—	MXINT4 (E8M0-scaled Int4), MXINT4	IEEE binary16
Zvvxi4bf16mm	32	—	MXINT4 (E8M0-scaled Int4), MXINT4	BFloat16
Zvvxi4fp32mm	32	—	MXINT4 (E8M0-scaled Int4), MXINT4	IEEE binary32
Zvvxni8fp16mm	16	Zvvxi8fp16mm	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary16
Zvvxni8bf16mm	16	Zvvxi8bf16mm	MXINT8 (E8M0-scaled Int8), MXINT8	BFloat16
Zvvxni8fp32mm	16	Zvvxi8fp32mm	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary32
Zvvxni8fp64mm	16	Zvvxi8fp64mm	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary64
Zvvxni4fp16mm	16	Zvvxi4fp16mm	MXINT4 (E8M0-scaled Int4), MXINT4	IEEE binary16
Zvvxni4bf16mm	16	Zvvxi4bf16mm	MXINT4 (E8M0-scaled Int4), MXINT4	BFloat16
Zvvxni4fp32mm	16	Zvvxi4fp32mm	MXINT4 (E8M0-scaled Int4), MXINT4	IEEE binary32



See [Section 36.7.3.4](#) for LMUL restrictions when **bs=1**.

36.13. Encoding map

The following tables are normative. They enumerate each computational instruction encoding point and map it to its input and output data types and governing individual extension or extensions.

A selected table row marked **reserved** raises an illegal-instruction exception. For a non-reserved row, every extension named by the applicable extension cell shall be implemented. Otherwise, the instruction raises an illegal-instruction exception.

The applicable extension cell is identified separately for ordinary unscaled, floating-point-input MX, and integer-input MX instructions in the subsections below.

For these checks, an extension implied by another implemented extension is considered implemented according to the extension dependency and implication rules.

36.13.1. Floating-point encoding map

Mixed-format inputs (`altfmt_A` $\neq$ `altfmt_B`) are permitted wherever both input formats are defined for a given (SEW, W) combination; those combinations are listed explicitly in the table below.

For an ordinary floating-point-input instruction with `vm=1`, the selected row is determined by the instruction, SEW, `altfmt_A`, `altfmt_B`, and `altfmt`. The **Extension** column is the applicable extension cell.

The encoding is legal only if the selected row is not marked **reserved** and every extension named in the **Extension** cell is implemented. A cell that names more than one extension requires all named extensions. For example, a mixed FP16 and BF16 row that names both `Zvvfp16fp32mm` and `Zvvbf16fp32mm` requires both extensions.

Where the table indicates that an `altfmt_A` or `altfmt_B` field is ignored, either value of that field selects the same row. The `bs` field is ignored when `vm=1`.

For `vm=1`, the **Extension** column identifies the extension or extensions required by the selected row.



For the 16-bit floating-point forms (`vfmacc.vv`, SEW=16) the legal accumulator format depends on whether the two inputs share a format, which is why a BF16 accumulator is reserved from two FP16 inputs yet legal from mixed FP16/BF16 inputs:

- When both inputs are FP16, the only legal accumulator format is FP16 (`Zvvfp16mm`); when both inputs are BF16, the only legal accumulator format is BF16 (`Zvvbf16mm`). The cross combinations — two FP16 inputs accumulated as BF16, or two BF16 inputs accumulated as FP16 — are **reserved**. Each single-format extension defines accumulation only in its own format, and no extension defines a same-input-format computation that reformats the accumulator to the other 16-bit format.
- When the inputs are mixed (one FP16 and one BF16, in either order), the row already requires both `Zvvfp16mm` and `Zvvbf16mm`. An implementation that supports the row therefore supports both 16-bit accumulator formats, so either FP16 or BF16 output is legal.

The asymmetry is thus a consequence of which extensions a row requires, not an independent output-format rule.

For `vfwmacc.vv`, `vfqmmacc.vv`, and `vf8wmacc.vv` with `vm=0`, the row is selected by the instruction, SEW, `altfmt_A`, `altfmt_B`, and `altfmt`. The governing MX cell is:

- MX BS=32 (`vm=0`) when `bs=0`; or
- MX BS=16 (`vm=0`) when `bs=1`.

The `vm=0` encoding is legal only if the selected MX cell names one or more extensions and every extension named by that cell is implemented. A selected MX cell that names no extension or is marked **reserved** raises an illegal-instruction exception.

Implementation of the extension or extensions named in the unscaled **Extension** column does not by itself authorize the corresponding `vm=0` encoding.

Table 87. Floating-point multiply-accumulate encoding map

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	Extension	MX BS=32 (vm=0)	MX BS=16 (vm=0)
vfm-macc.vv (W=1)											
vfm-macc	8	8	0	E4M3	0	E4M3	0	E4M3	Zvvo-fp8m	—	—
vfm-macc	8	8	0	E4M3	0	E4M3	1	E5M2	Zvvo-fp8m	—	—
vfm-macc	8	8	1	E5M2	1	E5M2	0	E4M3	Zvvo-fp8m	—	—
vfm-macc	8	8	1	E5M2	1	E5M2	1	E5M2	Zvvo-fp8m	—	—
vfm-macc	8	8	0	E4M3	1	E5M2	0	E4M3	Zvvo-fp8m	—	—
vfm-macc	8	8	0	E4M3	1	E5M2	1	E5M2	Zvvo-fp8m	—	—
vfm-macc	8	8	1	E5M2	0	E4M3	0	E4M3	Zvvo-fp8m	—	—
vfm-macc	8	8	1	E5M2	0	E4M3	1	E5M2	Zvvo-fp8m	—	—
vfm-macc	16	16	0	FP16	0	FP16	0	FP16	Zvv-fp16m	—	—
vfm-macc	16	16	0	FP16	0	FP16	1	BF16	<i>reserved</i>	—	—
vfm-macc	16	16	1	BF16	1	BF16	0	FP16	<i>reserved</i>	—	—
vfm-macc	16	16	1	BF16	1	BF16	1	BF16	Zvvb-f16m	—	—
vfm-macc	16	16	0	FP16	1	BF16	0	FP16	Zvv-fp16m, Zvvb-f16m	—	—
vfm-macc	16	16	0	FP16	1	BF16	1	BF16	Zvv-fp16m, Zvvb-f16m	—	—
vfm-macc	16	16	1	BF16	0	FP16	0	FP16	Zvv-fp16m, Zvvb-f16m	—	—
vfm-macc	16	16	1	BF16	0	FP16	1	BF16	Zvv-fp16m, Zvvb-f16m	—	—
vfm-macc	32	32	×	FP32	×	FP32	0	FP32	Zvv-fp32m	—	—
vfm-macc	32	32	×	FP32	×	FP32	1	—	<i>reserved</i>	—	—
vfm-macc	64	64	×	FP64	×	FP64	0	FP64	Zvv-fp64m	—	—
vfm-macc	64	64	×	FP64	×	FP64	1	—	<i>reserved</i>	—	—
vfwm-macc.vv (W=2)											

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	Extension	MX BS=32 (vm=0)	MX BS=16 (vm=0)
vfwmacc	8	4	0	E2M1	0	E2M1	0	E4M3	Zvvofp4ofp8mm	Zvvxofp4ofp8mm	Zvvxnopf4ofp8mm
vfwmacc	8	4	0	E2M1	0	E2M1	1	E5M2	Zvvofp4ofp8mm	Zvvxofp4ofp8mm	Zvvxnopf4ofp8mm
vfwmacc	8	4	1	—	1	—	×	—	<i>reserved</i>	—	—
vfwmacc	16	8	0	E4M3	0	E4M3	0	FP16	Zvvofp8fp16mm	Zvvxofp8fp16mm	Zvvxnopf8fp16mm
vfwmacc	16	8	0	E4M3	0	E4M3	1	BF16	Zvvofp8bfp16mm	Zvvxofp8bfp16mm	Zvvxnopf8bfp16mm
vfwmacc	16	8	1	E5M2	1	E5M2	0	FP16	Zvvofp8fp16mm	Zvvxofp8fp16mm	Zvvxnopf8fp16mm
vfwmacc	16	8	1	E5M2	1	E5M2	1	BF16	Zvvofp8bfp16mm	Zvvxofp8bfp16mm	Zvvxnopf8bfp16mm
vfwmacc	16	8	0	E4M3	1	E5M2	0	FP16	Zvvofp8fp16mm	Zvvxofp8fp16mm	Zvvxnopf8fp16mm
vfwmacc	16	8	0	E4M3	1	E5M2	1	BF16	Zvvofp8bfp16mm	Zvvxofp8bfp16mm	Zvvxnopf8bfp16mm
vfwmacc	16	8	1	E5M2	0	E4M3	0	FP16	Zvvofp8fp16mm	Zvvxofp8fp16mm	Zvvxnopf8fp16mm
vfwmacc	16	8	1	E5M2	0	E4M3	1	BF16	Zvvofp8bfp16mm	Zvvxofp8bfp16mm	Zvvxnopf8bfp16mm
vfwmacc	32	16	0	FP16	0	FP16	0	FP32	Zvvfp16fp32mm	—	—
vfwmacc	32	16	0	FP16	0	FP16	1	—	<i>reserved</i>	—	—
vfwmacc	32	16	1	BF16	1	BF16	0	FP32	Zvvbf16fp32mm	—	—
vfwmacc	32	16	1	BF16	1	BF16	1	—	<i>reserved</i>	—	—
vfwmacc	32	16	0	FP16	1	BF16	0	FP32	Zvvfp16fp32mm, Zvvbf16fp32mm	—	—
vfwmacc	32	16	1	BF16	0	FP16	0	FP32	Zvvfp16fp32mm, Zvvbf16fp32mm	—	—
vfwmacc	64	32	×	FP32	×	FP32	0	FP64	Zvvfp32fp64mm	—	—
vfwmacc	64	32	×	FP32	×	FP32	1	—	<i>reserved</i>	—	—
vfqmmacc.vv (W=4)											
vfqmmacc	8	2	×	—	×	—	×	—	<i>reserved</i>	—	—
vfqmmacc	16	4	0	E2M1	0	E2M1	0	FP16	Zvvofp4fp16mm	Zvvxofp4fp16mm	Zvvxnopf4fp16mm
vfqmmacc	16	4	0	E2M1	0	E2M1	1	BF16	Zvvofp4bfp16mm	Zvvxofp4bfp16mm	Zvvxnopf4bfp16mm
vfqmmacc	16	4	1	—	1	—	×	—	<i>reserved</i>	—	—

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	Extension	MX BS=32 (vm=0)	MX BS=16 (vm=0)
vfqmmacc	32	8	0	E4M3	0	E4M3	0	FP32	Zvvofp8fp32mm	Zvvxofp8fp32mm	Zvvxnofp8fp32mm
vfqmmacc	32	8	0	E4M3	0	E4M3	1	—	<i>reserved</i>	—	—
vfqmmacc	32	8	1	E5M2	1	E5M2	0	FP32	Zvvofp8fp32mm	Zvvxofp8fp32mm	Zvvxnofp8fp32mm
vfqmmacc	32	8	1	E5M2	1	E5M2	1	—	<i>reserved</i>	—	—
vfqmmacc	32	8	0	E4M3	1	E5M2	0	FP32	Zvvofp8fp32mm	Zvvxofp8fp32mm	Zvvxnofp8fp32mm
vfqmmacc	32	8	1	E5M2	0	E4M3	0	FP32	Zvvofp8fp32mm	Zvvxofp8fp32mm	Zvvxnofp8fp32mm
vfqmmacc	64	16	0	FP16	0	FP16	0	FP64	Zvvfp16fp64mm	—	—
vfqmmacc	64	16	0	FP16	0	FP16	1	—	<i>reserved</i>	—	—
vfqmmacc	64	16	1	BF16	1	BF16	0	FP64	Zvbf16fp64mm	—	—
vfqmmacc	64	16	1	BF16	1	BF16	1	—	<i>reserved</i>	—	—
vfqmmacc	64	16	0	FP16	1	BF16	0	FP64	Zvvfp16fp64mm, Zvbf16fp64mm	—	—
vfqmmacc	64	16	1	BF16	0	FP16	0	FP64	Zvvfp16fp64mm, Zvbf16fp64mm	—	—
vf8wmmacc.vv (W=8)											
vf8wmmacc	8	1	×	—	×	—	×	—	<i>reserved</i>	—	—
vf8wmmacc	16	2	×	—	×	—	×	—	<i>reserved</i>	—	—
vf8wmmacc	32	4	0	E2M1	0	E2M1	0	FP32	Zvvofp4fp32mm	Zvvxofp4fp32mm	Zvvxnofp4fp32mm
vf8wmmacc	32	4	0	E2M1	0	E2M1	1	—	<i>reserved</i>	—	—
vf8wmmacc	32	4	1	—	1	—	×	—	<i>reserved</i>	—	—
vf8wmmacc	64	8	0	E4M3	0	E4M3	0	FP64	Zvvofp8fp64mm	Zvvxofp8fp64mm	Zvvxnofp8fp64mm
vf8wmmacc	64	8	0	E4M3	0	E4M3	1	—	<i>reserved</i>	—	—
vf8wmmacc	64	8	1	E5M2	1	E5M2	0	FP64	Zvvofp8fp64mm	Zvvxofp8fp64mm	Zvvxnofp8fp64mm
vf8wmmacc	64	8	1	E5M2	1	E5M2	1	—	<i>reserved</i>	—	—
vf8wmmacc	64	8	0	E4M3	1	E5M2	0	FP64	Zvvofp8fp64mm	Zvvxofp8fp64mm	Zvvxnofp8fp64mm

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	Extension	MX BS=32 (vm=0)	MX BS=16 (vm=0)
vf8wmmacc	64	8	1	E5M2	0	E4M3	0	FP64	Zvvofp8fp64mm	Zvvxofp8fp64mm	Zvvxnofp8fp64mm

36.13.2. Integer encoding map

altfmt_A and **altfmt_B** select signed (0) or unsigned (1) interpretation; the accumulator is always signed. **altfmt** is unused for ordinary integer accumulation.

For the ordinary integer forms listed in this table, **vm=1**. For **vmmacc.vv**, **vm=0** is *reserved*.

For **vmmacc.vv**, **vqmmacc.vv**, and **v8wmmacc.vv**, **vm=0** is separately defined as **vwimmacc.vv**, **vfqimmacc.vv**, and **vf8wimmacc.vv**, respectively (integer inputs, FP accumulator, microscaling); see [Section 36.13.3](#).

For an ordinary integer instruction with **vm=1**, the selected row is determined by the instruction, **SEW**, **altfmt_A**, and **altfmt_B**. The **Extension** column is the applicable extension cell.

The encoding is legal only if the selected row is not marked *reserved* and the extension named in the **Extension** cell is implemented.

The **altfmt** and **bs** fields do not participate in the selection of an ordinary integer row and are ignored when **vm=1**.

Table 88. Integer multiply-accumulate encoding map

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	C Type	Extension
vmmacc.vv (W=1)								
vmmacc	8	8	0	Int8	0	Int8	Int8	Zvvi8mm
vmmacc	8	8	0	Int8	1	UInt8	Int8	Zvvi8mm
vmmacc	8	8	1	UInt8	0	Int8	Int8	Zvvi8mm
vmmacc	8	8	1	UInt8	1	UInt8	Int8	Zvvi8mm
vmmacc	16	16	0	Int16	0	Int16	Int16	Zvvi16mm
vmmacc	16	16	0	Int16	1	UInt16	Int16	Zvvi16mm
vmmacc	16	16	1	UInt16	0	Int16	Int16	Zvvi16mm
vmmacc	16	16	1	UInt16	1	UInt16	Int16	Zvvi16mm
vmmacc	32	32	0	Int32	0	Int32	Int32	Zvvi32mm
vmmacc	32	32	0	Int32	1	UInt32	Int32	Zvvi32mm
vmmacc	32	32	1	UInt32	0	Int32	Int32	Zvvi32mm
vmmacc	32	32	1	UInt32	1	UInt32	Int32	Zvvi32mm
vmmacc	64	64	0	Int64	0	Int64	Int64	Zvvi64mm
vmmacc	64	64	0	Int64	1	UInt64	Int64	Zvvi64mm
vmmacc	64	64	1	UInt64	0	Int64	Int64	Zvvi64mm
vmmacc	64	64	1	UInt64	1	UInt64	Int64	Zvvi64mm
vwmmacc.vv (W=2)								
vwmmacc	8	4	0	Int4	0	Int4	Int8	Zvvi4i8mm

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	C Type	Extension
vwmacc	8	4	0	Int4	1	UInt4	Int8	Zvvi4i8mm
vwmacc	8	4	1	UInt4	0	Int4	Int8	Zvvi4i8mm
vwmacc	8	4	1	UInt4	1	UInt4	Int8	Zvvi4i8mm
vwmacc	16	8	0	Int8	0	Int8	Int16	Zvvi8i16mm
vwmacc	16	8	0	Int8	1	UInt8	Int16	Zvvi8i16mm
vwmacc	16	8	1	UInt8	0	Int8	Int16	Zvvi8i16mm
vwmacc	16	8	1	UInt8	1	UInt8	Int16	Zvvi8i16mm
vwmacc	32	16	0	Int16	0	Int16	Int32	Zvvi16i32mm
vwmacc	32	16	0	Int16	1	UInt16	Int32	Zvvi16i32mm
vwmacc	32	16	1	UInt16	0	Int16	Int32	Zvvi16i32mm
vwmacc	32	16	1	UInt16	1	UInt16	Int32	Zvvi16i32mm
vwmacc	64	32	0	Int32	0	Int32	Int64	Zvvi32i64mm
vwmacc	64	32	0	Int32	1	UInt32	Int64	Zvvi32i64mm
vwmacc	64	32	1	UInt32	0	Int32	Int64	Zvvi32i64mm
vwmacc	64	32	1	UInt32	1	UInt32	Int64	Zvvi32i64mm
vqmmacc.vv (W=4)								
vqmmacc	8	2	×	—	×	—	—	<i>reserved</i>
vqmmacc	16	4	0	Int4	0	Int4	Int16	Zvvi4i16mm
vqmmacc	16	4	0	Int4	1	UInt4	Int16	Zvvi4i16mm
vqmmacc	16	4	1	UInt4	0	Int4	Int16	Zvvi4i16mm
vqmmacc	16	4	1	UInt4	1	UInt4	Int16	Zvvi4i16mm
vqmmacc	32	8	0	Int8	0	Int8	Int32	Zvvi8i32mm
vqmmacc	32	8	0	Int8	1	UInt8	Int32	Zvvi8i32mm
vqmmacc	32	8	1	UInt8	0	Int8	Int32	Zvvi8i32mm
vqmmacc	32	8	1	UInt8	1	UInt8	Int32	Zvvi8i32mm
vqmmacc	64	16	0	Int16	0	Int16	Int64	Zvvi16i64mm
vqmmacc	64	16	0	Int16	1	UInt16	Int64	Zvvi16i64mm
vqmmacc	64	16	1	UInt16	0	Int16	Int64	Zvvi16i64mm
vqmmacc	64	16	1	UInt16	1	UInt16	Int64	Zvvi16i64mm
v8wmmacc.vv (W=8)								
v8wmmacc	8	1	×	—	×	—	—	<i>reserved</i>
v8wmmacc	16	2	×	—	×	—	—	<i>reserved</i>
v8wmmacc	32	4	0	Int4	0	Int4	Int32	Zvvi4i32mm
v8wmmacc	32	4	0	Int4	1	UInt4	Int32	Zvvi4i32mm
v8wmmacc	32	4	1	UInt4	0	Int4	Int32	Zvvi4i32mm
v8wmmacc	32	4	1	UInt4	1	UInt4	Int32	Zvvi4i32mm
v8wmmacc	64	8	0	Int8	0	Int8	Int64	Zvvi8i64mm
v8wmmacc	64	8	0	Int8	1	UInt8	Int64	Zvvi8i64mm
v8wmmacc	64	8	1	UInt8	0	Int8	Int64	Zvvi8i64mm

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	C Type	Extension
v8wmmacc	64	8	1	UInt8	1	UInt8	Int64	Zvvi8i64mm

36.13.3. Integer MX encoding map (vm=0)

When `vm=0`, `vwmacc.vv`, `vqmmacc.vv`, and `v8wmmacc.vv` are decoded as `vfwimmacc.vv`, `vfqimmacc.vv`, and `vf8wimmacc.vv` respectively. MXINT inputs are always signed; `altfmt_A` and `altfmt_B` must be 0 (unsigned is *reserved*). `altfmt` selects the FP accumulator format. `vmmacc.vv` with `vm=0` remains *reserved*.

For `vfwimmacc.vv`, `vfqimmacc.vv`, and `vf8wimmacc.vv`, the row is selected by the instruction, `SEW`, `altfmt_A`, `altfmt_B`, and `altfmt`. The governing MX cell is:

- MX BS=32 when `bs=0`; or
- MX BS=16 when `bs=1`.

The encoding is legal only if the selected MX cell names an extension and that extension is implemented. A selected cell containing *reserved* raises an illegal-instruction exception.

Table 89. Integer MX multiply-accumulate encoding map (vm=0)

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	MX BS=32	MX BS=16
vfwimmacc.vv (W=2, vm=0 of vwmacc opcode)										
vfwimmacc	8	4	×	—	×	—	×	—	<i>reserved</i>	<i>reserved</i>
vfwimmacc	16	8	0	Int8	0	Int8	0	FP16	Zvvxi8fp16mm	Zvvxni8fp16mm
vfwimmacc	16	8	0	Int8	1	UInt8	0	FP16	<i>reserved</i>	<i>reserved</i>
vfwimmacc	16	8	1	UInt8	0	Int8	0	FP16	<i>reserved</i>	<i>reserved</i>
vfwimmacc	16	8	1	UInt8	1	UInt8	0	FP16	<i>reserved</i>	<i>reserved</i>
vfwimmacc	16	8	0	Int8	0	Int8	1	BF16	Zvvxi8bf16mm	Zvvxni8bf16mm
vfwimmacc	16	8	0	Int8	1	UInt8	1	BF16	<i>reserved</i>	<i>reserved</i>
vfwimmacc	16	8	1	UInt8	0	Int8	1	BF16	<i>reserved</i>	<i>reserved</i>
vfwimmacc	16	8	1	UInt8	1	UInt8	1	BF16	<i>reserved</i>	<i>reserved</i>
vfwimmacc	32	16	×	—	×	—	×	—	<i>reserved</i>	<i>reserved</i>
vfwimmacc	64	32	×	—	×	—	×	—	<i>reserved</i>	<i>reserved</i>
vfqimmacc.vv (W=4, vm=0 of vqmmacc opcode)										
vfqimmacc	8	2	×	—	×	—	×	—	<i>reserved</i>	<i>reserved</i>
vfqimmacc	16	4	0	Int4	0	Int4	0	FP16	Zvvxi4fp16mm	Zvvxni4fp16mm
vfqimmacc	16	4	0	Int4	1	UInt4	0	FP16	<i>reserved</i>	<i>reserved</i>
vfqimmacc	16	4	1	UInt4	0	Int4	0	FP16	<i>reserved</i>	<i>reserved</i>
vfqimmacc	16	4	1	UInt4	1	UInt4	0	FP16	<i>reserved</i>	<i>reserved</i>
vfqimmacc	16	4	0	Int4	0	Int4	1	BF16	Zvvxi4bf16mm	Zvvxni4bf16mm
vfqimmacc	16	4	0	Int4	1	UInt4	1	BF16	<i>reserved</i>	<i>reserved</i>

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	MX BS=32	MX BS=16
vfqimmac	16	4	1	UInt4	0	Int4	1	BF16	<i>reserved</i>	<i>reserved</i>
vfqimmac	16	4	1	UInt4	1	UInt4	1	BF16	<i>reserved</i>	<i>reserved</i>
vfqimmac	32	8	0	Int8	0	Int8	0	FP32	Zvvxi8fp32mm	Zvvxni8fp32mm
vfqimmac	32	8	0	Int8	1	UInt8	0	FP32	<i>reserved</i>	<i>reserved</i>
vfqimmac	32	8	1	UInt8	0	Int8	0	FP32	<i>reserved</i>	<i>reserved</i>
vfqimmac	32	8	1	UInt8	1	UInt8	0	FP32	<i>reserved</i>	<i>reserved</i>
vfqimmac	32	8	×	—	×	—	1	—	<i>reserved</i>	<i>reserved</i>
vfqimmac	64	16	×	—	×	—	×	—	<i>reserved</i>	<i>reserved</i>
vf8wimmac.vv (W=8, vm=0 of v8wmmacc opcode)										
vf8wimmac	8	1	×	—	×	—	×	—	<i>reserved</i>	<i>reserved</i>
vf8wimmac	16	2	×	—	×	—	×	—	<i>reserved</i>	<i>reserved</i>
vf8wimmac	32	4	0	Int4	0	Int4	0	FP32	Zvvxi4fp32mm	Zvvxni4fp32mm
vf8wimmac	32	4	0	Int4	1	UInt4	0	FP32	<i>reserved</i>	<i>reserved</i>
vf8wimmac	32	4	1	UInt4	0	Int4	0	FP32	<i>reserved</i>	<i>reserved</i>
vf8wimmac	32	4	1	UInt4	1	UInt4	0	FP32	<i>reserved</i>	<i>reserved</i>
vf8wimmac	32	4	×	—	×	—	1	—	<i>reserved</i>	<i>reserved</i>
vf8wimmac	64	8	0	Int8	0	Int8	0	FP64	Zvvxi8fp64mm	Zvvxni8fp64mm
vf8wimmac	64	8	0	Int8	1	UInt8	0	FP64	<i>reserved</i>	<i>reserved</i>
vf8wimmac	64	8	1	UInt8	0	Int8	0	FP64	<i>reserved</i>	<i>reserved</i>
vf8wimmac	64	8	1	UInt8	1	UInt8	0	FP64	<i>reserved</i>	<i>reserved</i>
vf8wimmac	64	8	×	—	×	—	1	—	<i>reserved</i>	<i>reserved</i>

[1] Open Compute Project, *OCF Microscaling Formats (MX) v1.0 Spec*, www.opencompute.org/documents/ocf-microscaling-formats-mx-v1-0-spec-final-pdf

[2] S = R: fully packed, no padding